LittleHeart: source code

UnivaBio 2026 · AI for Human Health

Project structure

ml/download_data.py: Downloads the CirCor and PhysioNet 2016 datasets from PhysioNet's open-data mirror.
ml/features.py: Turns a recording into a log-mel spectrogram and 4-second windows.
ml/train.py: Cross-validated training (clean or noise-robust), evaluation, ONNX export, demo and quiz sets.
ml/evaluation.py: Damaged-audio test conditions, training augmentation, error analysis, quiz selection.
ml/heartbeat_check.py: Safety check calibration: refuse recordings without a heartbeat rhythm.
ml/external_validation.py: Tests the exported model on the PhysioNet 2016 dataset it never saw.
ml/compare_models.py: Adds the clean-trained model's results for before/after comparison.
web/index.html: Page structure: screening, ear training, screening log, validation and education tabs.
web/js/i18n.js: Translations (English, Turkish, Portuguese, Spanish).
web/js/features.js: JavaScript port of features.py and the heartbeat check, so the browser computes identical numbers.
web/js/model.js: Runs the ONNX model in the browser and applies the referral rule.
web/js/draw.js: Spectrogram, murmur timeline, playback cursor and live waveform drawing.
web/js/app.js: Screening view: real cases, uploads, verdict, per-valve results.
web/js/guided.js: Guided recording with live heartbeat and sound-level check.
web/js/quiz.js: Train-your-ear mode.
web/js/log.js: Screening log stored on the device, with CSV export.
web/js/validation.js: Validation tab: metrics, error analysis, robustness, external test.
web/js/main.js: Start-up: tabs, language, modules, offline support.
web/sw.js: Service worker that keeps the app working offline.
web/css/style.css: Visual design, light/dark themes, phone and print layouts.
tests/test_pipeline.py: Automated tests, including Python vs JavaScript parity and translations.
tests/js_logmel.cjs: Node.js helper used by the parity test.
tests/i18n_check.cjs: Checks that every text exists in all four languages.
docs/make_pdfs.py: Generates this PDF and the one-page description.
Generated, not listed: web/model/murmur_cnn.onnx (trained network), config.json (spectrogram settings and thresholds), metrics.json/.js (validation results), external.json (external test), heartbeat_check.json, web/samples and web/quiz (recordings from the dataset).
Third-party: web/vendor/ort (ONNX Runtime Web, MIT licence).

ml/download_data.py

Downloads the CirCor and PhysioNet 2016 datasets from PhysioNet's open-data mirror.

```
"""
Downloads PhysioNet datasets (open access) from PhysioNet's open-data mirror
on AWS, which is much faster than the zip download.

Run:  python ml/download_data.py              CirCor DigiScope (training data, ~550 MB)
      python ml/download_data.py --external   PhysioNet 2016 challenge (external test, ~200 MB)
"""

import sys
import time
import urllib.error
import urllib.parse
import urllib.request
import xml.etree.ElementTree as ET
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

BUCKET = "https://physionet-open.s3.amazonaws.com"
DATA = Path(__file__).resolve().parent.parent / "data"
DATASETS = {
    "circor": ("circor-heart-sound/1.0.3/", DATA / "circor-1.0.3"),
    "external": ("challenge-2016/1.0.0/", DATA / "physionet2016"),
}
NS = "{http://s3.amazonaws.com/doc/2006-03-01/}"


def list_keys(prefix):
    keys, token = [], None
    while True:
        url = f"{BUCKET}/?list-type=2&prefix={prefix}"
        if token:
            url += "&continuation-token=" + urllib.parse.quote(token)
        root = ET.fromstring(urllib.request.urlopen(url).read())
        keys += [c.find(NS + "Key").text for c in root.iter(NS + "Contents")]
        token_el = root.find(NS + "NextContinuationToken")
        if token_el is None:
            return keys
        token = token_el.text


def download(name):
    prefix, out = DATASETS[name]

    def fetch(key):
        target = out / key[len(prefix):]
        if target.exists() and target.stat().st_size > 0:
            return
        target.parent.mkdir(parents=True, exist_ok=True)
        for attempt in range(6):
            try:
                target.write_bytes(urllib.request.urlopen(f"{BUCKET}/{key}").read())
                return
            except urllib.error.HTTPError as err:
                if err.code not in (500, 503) or attempt == 5:
                    raise
                time.sleep(2 ** attempt)  # the mirror throttles bursts of requests

    keys = [k for k in list_keys(prefix) if k.endswith((".wav", ".csv"))]
    if name == "external":
        keys = [k for k in keys if "/training-" in k]
    print(f"Downloading {len(keys)} files to {out}")
    with ThreadPoolExecutor(32) as pool:
        for i, _ in enumerate(pool.map(fetch, keys), 1):
            if i % 500 == 0:
                print(f"  {i}/{len(keys)}")
    print("Done")


if __name__ == "__main__":
    download("external" if "--external" in sys.argv else "circor")
```

ml/features.py

Turns a recording into a log-mel spectrogram and 4-second windows.

```
"""
Turns a heart sound recording into a log-mel spectrogram: a picture of
which frequencies are loud at each moment. The CNN looks at this picture.

web/js/features.js implements exactly the same steps in JavaScript, and
tests/parity check that both give the same numbers.
"""

import numpy as np
from scipy.io import wavfile

SAMPLE_RATE = 4000      # CirCor recordings are sampled at 4 kHz
N_FFT = 256             # 64 ms analysis frame
HOP = 64                # 16 ms between frames
N_MELS = 32
F_MIN = 25.0            # heart sounds and murmurs live roughly in 25-800 Hz
F_MAX = 1000.0
WINDOW_FRAMES = 247     # frames in one 4-second window
WINDOW_STEP = 125       # 2 seconds between window starts
EPS = 1e-6


def hz_to_mel(f):
    return 2595.0 * np.log10(1.0 + f / 700.0)


def mel_to_hz(m):
    return 700.0 * (10.0 ** (m / 2595.0) - 1.0)


def mel_filterbank():
    """Triangular filters, evenly spaced on the mel scale (how humans hear pitch)."""
    n_bins = N_FFT // 2 + 1
    bin_freqs = np.arange(n_bins) * SAMPLE_RATE / N_FFT
    mel_points = np.linspace(hz_to_mel(F_MIN), hz_to_mel(F_MAX), N_MELS + 2)
    hz_points = mel_to_hz(mel_points)
    fb = np.zeros((N_MELS, n_bins))
    for m in range(N_MELS):
        left, center, right = hz_points[m], hz_points[m + 1], hz_points[m + 2]
        rising = (bin_freqs - left) / (center - left)
        falling = (right - bin_freqs) / (right - center)
        fb[m] = np.maximum(0.0, np.minimum(rising, falling))
    return fb


MEL_FB = mel_filterbank()
HANN = 0.5 - 0.5 * np.cos(2 * np.pi * np.arange(N_FFT) / N_FFT)


def load_wav(path):
    rate, x = wavfile.read(path)
    if rate != SAMPLE_RATE:
        raise ValueError(f"{path}: expected {SAMPLE_RATE} Hz, got {rate}")
    if x.dtype == np.int16:
        x = x.astype(np.float64) / 32768.0
    return x.astype(np.float64)


def normalize(x):
    """Remove the DC offset and scale to unit loudness, so a quiet and a loud
    stethoscope give the same spectrogram."""
    x = x - x.mean()
    return x / (x.std() + 1e-8)


def log_mel(x):
    """Full-recording log-mel spectrogram, shape (N_MELS, n_frames)."""
    x = normalize(x)
    if len(x) < N_FFT:
        x = np.pad(x, (0, N_FFT - len(x)))
    n_frames = 1 + (len(x) - N_FFT) // HOP
    idx = np.arange(N_FFT)[None, :] + HOP * np.arange(n_frames)[:, None]
    frames = x[idx] * HANN
    power = np.abs(np.fft.rfft(frames, axis=1)) ** 2
    return np.log10(MEL_FB @ power.T + EPS).astype(np.float32)


def window_starts(n_frames):
    """Start frame of every 4-second window, 2 seconds apart."""
    if n_frames <= WINDOW_FRAMES:
        return [0]
    return list(range(0, n_frames - WINDOW_FRAMES + 1, WINDOW_STEP))


def cut_window(spec, start):
    """One 4-second window. Short recordings are repeated to fill it."""
    if spec.shape[1] < WINDOW_FRAMES:
```

```
        reps = int(np.ceil(WINDOW_FRAMES / spec.shape[1]))
        spec = np.tile(spec, (1, reps))
    return spec[:, start:start + WINDOW_FRAMES]
```

ml/train.py

Cross-validated training (clean or noise-robust), evaluation, ONNX export, demo and quiz sets.

```
"""
LittleHeart - training and evaluation.

1. Reads the CirCor DigiScope dataset (942 children, 3,163 heart sound recordings).
2. Holds out a few children as demo cases that the model never trains on.
3. Runs 5-fold cross-validation, split by CHILD (never by recording), comparing
   our CNN against a simple baseline using the official PhysioNet 2022 score.
   Inside every fold it also re-tests the model on damaged audio (noise,
   phone-like microphone, handling rumble) and keeps each child's answer for
   the error analysis and the "Train your ear" quiz (see evaluation.py).
4. Trains the final model on all remaining children and exports it to ONNX
   for the browser, plus the metrics, analysis, demo cases and quiz.

Run:  python ml/train.py --robust   (deployed model: trained with noise and phone-like damage, ~2 h)
      python ml/train.py            (original model trained on clean audio only, ~1.5 h)
      python ml/train.py --quick    (1 fold, 1 epoch, to test the pipeline)
Options: --out DIR writes to DIR instead of web/, --no-final skips the final model.
"""

import argparse
import csv
import json
import shutil
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from scipy.io import wavfile
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedKFold, train_test_split

import evaluation as E
import features as F

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = next((ROOT / "data").glob("*circor*1.0.3*"), ROOT / "data" / "circor")
CACHE = ROOT / "data" / "spectrograms.npz"
WEB_MODEL = ROOT / "web" / "model"
WEB_SAMPLES = ROOT / "web" / "samples"

CLASSES = ["Absent", "Present", "Unknown"]
ABSENT, PRESENT, UNKNOWN = 0, 1, 2
SEED = 7


# ---------------------------------------------------------------- data

def load_patients():
    rows = list(csv.DictReader(open(DATA_DIR / "training_data.csv", encoding="utf-8")))
    patients = []
    for r in rows:
        pid = r["Patient ID"]
        murmur_locs = set(r["Murmur locations"].split("+")) if r["Murmur locations"] not in ("", "nan") else set()
        recs = []
        for wav in sorted((DATA_DIR / "training_data").glob(f"{pid}_*.wav")):
            loc = wav.stem.split("_")[1]
            if r["Murmur"] == "Present":
                # A murmur is only audible at some valves; other sites are labelled absent.
                label = PRESENT if loc in murmur_locs else ABSENT
            elif r["Murmur"] == "Unknown":
                label = UNKNOWN
            else:
                label = ABSENT
            recs.append({"path": wav, "location": loc, "label": label})
        patients.append({
            "id": pid,
            "label": CLASSES.index(r["Murmur"]),
            "age": r["Age"],
            "sex": r["Sex"],
            "murmur_locations": sorted(murmur_locs),
            "timing": r["Systolic murmur timing"],
            "grade": r["Systolic murmur grading"],
            "pitch": r["Systolic murmur pitch"],
            "quality": r["Systolic murmur quality"],
            "shape": r["Systolic murmur shape"],
            "outcome": r["Outcome"],
            "recordings": recs,
        })
    return patients


def load_spectrograms(patients):
    if CACHE.exists():
        cached = np.load(CACHE, allow_pickle=True)
```

```
        specs = cached["specs"].item()
    else:
        specs = {}
    missing = [rec for p in patients for rec in p["recordings"] if rec["path"].name not in specs]
    if missing:
        print(f"Computing {len(missing)} spectrograms...")
        for rec in missing:
            specs[rec["path"].name] = F.log_mel(F.load_wav(rec["path"]))
        np.savez_compressed(CACHE, specs=np.array(specs, dtype=object))
    for p in patients:
        for rec in p["recordings"]:
            rec["spec"] = specs[rec["path"].name]


# ---------------------------------------------------------------- model

class MurmurCNN(nn.Module):
    """Small convolutional network: four conv blocks, then average over time
    and frequency, then 3 outputs (Absent / Present / Unknown)."""

    def __init__(self, mean, std):
        super().__init__()
        self.register_buffer("mean", torch.tensor(mean, dtype=torch.float32).view(1, 1, -1, 1))
        self.register_buffer("std", torch.tensor(std, dtype=torch.float32).view(1, 1, -1, 1))

        def block(c_in, c_out, pool=True):
            layers = [nn.Conv2d(c_in, c_out, 3, padding=1), nn.BatchNorm2d(c_out), nn.ReLU()]
            if pool:
                layers.append(nn.MaxPool2d(2))
            return layers

        self.body = nn.Sequential(*block(1, 16), *block(16, 32), *block(32, 64), *block(64, 64, pool=False))
        self.head = nn.Sequential(nn.Dropout(0.3), nn.Linear(64, 3))

    def forward(self, x):                  # x: (batch, 1, N_MELS, WINDOW_FRAMES) log-mel
        x = (x - self.mean) / self.std     # standardise each mel band
        x = self.body(x)
        x = x.mean(dim=(2, 3))             # global average pooling
        return self.head(x)


class WithSoftmax(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, x):
        return torch.softmax(self.model(x), dim=1)


def band_stats(recordings):
    allspec = np.concatenate([r["spec"] for r in recordings], axis=1)
    return allspec.mean(axis=1), allspec.std(axis=1) + 1e-6


def random_window(spec, rng):
    n = spec.shape[1]
    start = 0 if n <= F.WINDOW_FRAMES else int(rng.integers(0, n - F.WINDOW_FRAMES + 1))
    return F.cut_window(spec, start)


def train_cnn(train_recs, val_patients, epochs, rng, log=print, robust=False):
    torch.manual_seed(int(rng.integers(1e9)))
    mean, std = band_stats(train_recs)
    model = MurmurCNN(mean, std)
    opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=1e-3)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    # Murmurs are rarer and matter more, so their mistakes cost more.
    loss_fn = nn.CrossEntropyLoss(weight=torch.tensor([1.0, 3.0, 2.0]))
    labels = np.array([r["label"] for r in train_recs])

    best = (-1.0, None, None)
    for epoch in range(epochs):
        model.train()
        order = np.concatenate([rng.permutation(len(train_recs)) for _ in range(3)])
        total = 0.0
        for i in range(0, len(order), 64):
            batch = order[i:i + 64]
            if robust:
                x = np.stack([E.augmented_window(train_recs[j]["audio"], rng) for j in batch])[:, None]
            else:
                x = np.stack([random_window(train_recs[j]["spec"], rng) for j in batch])[:, None]
            x = x + rng.normal(0, 0.05, x.shape).astype(np.float32)
            y = torch.tensor(labels[batch])
            opt.zero_grad()
            loss = loss_fn(model(torch.from_numpy(x)), y)
            loss.backward()
            opt.step()
            total += loss.item() * len(batch)
        sched.step()

```

```
        probs = patient_probs(lambda w: predict_windows(model, w), val_patients)
        t, score = tune_thresholds(probs, [p["label"] for p in val_patients])
        log(f"    epoch {epoch + 1:2d}  loss {total / len(order):.3f}  val weighted acc {score:.3f}")
        if score > best[0]:
            best = (score, {k: v.clone() for k, v in model.state_dict().items()}, t)
    model.load_state_dict(best[1])
    model.eval()
    return model, best[2]


@torch.no_grad()
def predict_windows(model, windows):
    model.eval()
    x = torch.from_numpy(np.stack(windows)[:, None].astype(np.float32))
    return torch.softmax(model(x), dim=1).numpy()


def recording_probs(predict, spec):
    """Average the class probabilities of every 4-second window."""
    windows = [F.cut_window(spec, s) for s in F.window_starts(spec.shape[1])]
    return predict(windows).mean(axis=0)


def patient_probs(predict, patients):
    """For each child: the highest Present and Unknown probability over all valves."""
    out = []
    for p in patients:
        rec = np.array([recording_probs(predict, r["spec"]) for r in p["recordings"]])
        out.append((rec[:, PRESENT].max(), rec[:, UNKNOWN].max()))
    return np.array(out)


# ------------------------------------------------------------------ decisions and metrics

def decide(probs, t_present, t_unknown):
    """Present if any valve looks like a murmur, else Unknown if any recording
    is unclear, else Absent. Same rule order as the PhysioNet challenge."""
    out = np.full(len(probs), ABSENT)
    out[probs[:, 1] >= t_unknown] = UNKNOWN
    out[probs[:, 0] >= t_present] = PRESENT
    return out


def weighted_accuracy(y_true, y_pred):
    """Official PhysioNet 2022 murmur score: Present counts 5x, Unknown 3x, Absent 1x."""
    w = {PRESENT: 5, UNKNOWN: 3, ABSENT: 1}
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    num = sum(w[c] * np.sum((y_true == c) & (y_pred == c)) for c in w)
    den = sum(w[c] * np.sum(y_true == c) for c in w)
    return float(num / den)


def tune_thresholds(probs, y_true):
    grid = np.round(np.arange(0.05, 0.96, 0.05), 2)
    best = (-1.0, (0.5, 0.5))
    for tp in grid:
        for tu in grid:
            s = weighted_accuracy(y_true, decide(probs, tp, tu))
            if s > best[0]:
                best = (s, (float(tp), float(tu)))
    return best[1], best[0]


def screening_metrics(y_true, y_pred, present_prob):
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    known = y_true != UNKNOWN
    is_murmur = y_true[known] == PRESENT
    flagged = y_pred[known] == PRESENT
    confusion = [[int(np.sum((y_true == t) & (y_pred == p))) for p in range(3)] for t in range(3)]
    return {
        "weighted_accuracy": weighted_accuracy(y_true, y_pred),
        "sensitivity": float(np.mean(flagged[is_murmur])),
        "specificity": float(np.mean(~flagged[~is_murmur])),
        "auc": float(roc_auc_score(is_murmur, present_prob[known])),
        "confusion": confusion,  # rows = true, cols = predicted (Absent, Present, Unknown)
    }


# ------------------------------------------------------------------ baseline

def baseline_features(spec):
    return np.concatenate([spec.mean(axis=1), spec.std(axis=1)])


def train_baseline(train_recs):
    X = np.stack([baseline_features(r["spec"]) for r in train_recs])
    y = np.array([r["label"] for r in train_recs])
    clf = LogisticRegression(max_iter=3000, C=0.5, class_weight="balanced")
    mean, std = X.mean(0), X.std(0) + 1e-6
    clf.fit((X - mean) / std, y)
    return lambda spec: clf.predict_proba(((baseline_features(spec) - mean) / std)[None])[0]
```

```


def baseline_patient_probs(predict_rec, patients):
    out = []
    for p in patients:
        rec = np.array([predict_rec(r["spec"]) for r in p["recordings"]])
        out.append((rec[:, PRESENT].max(), rec[:, UNKNOWN].max()))
    return np.array(out)


# ---------------------------------------------------------------- main

def pick_demo_patients(patients, rng):
    """A few children per class that are never used for training or CV."""
    chosen = []
    for label, n in [(PRESENT, 4), (ABSENT, 4), (UNKNOWN, 2)]:
        pool = [p for p in patients if p["label"] == label and 3 <= len(p["recordings"]) <= 5]
        chosen += list(rng.choice(pool, n, replace=False))
    return chosen


def summarize(fold_metrics):
    keys = ["weighted_accuracy", "sensitivity", "specificity", "auc"]
    return {k: {"mean": float(np.mean([m[k] for m in fold_metrics])),
                "std": float(np.std([m[k] for m in fold_metrics]))} for k in keys}


def evaluate_fold(model, tp, tu, test_p, noise_rng, results):
    """Scores the fold's unseen children on clean and damaged audio and keeps
    each child's and recording's answer for the error analysis and quiz."""
    predict = lambda w: predict_windows(model, w)  # noqa: E731
    y = [p["label"] for p in test_p]
    for cond in E.CONDITIONS:
        child_probs, passed, total = [], 0, 0
        for p in test_p:
            rec_probs = []
            for r in p["recordings"]:
                spec = r["spec"] if cond == "clean" else F.log_mel(E.degrade(F.load_wav(r["path"]), cond,
    noise_rng))
                rp = recording_probs(predict, spec)
                ok = E.beat_ok(spec)
                rec_probs.append(rp)
                passed += ok
                total += 1
                if cond == "clean":
                    results["recordings"].append({
                        "file": r["path"].name, "location": r["location"], "label": int(r["label"]),
                        "present_prob": float(rp[PRESENT]), "ai_flags": bool(rp[PRESENT] >= tp), "beat_ok": ok,
                        "grade": p["grade"], "timing": p["timing"], "pitch": p["pitch"],
                        "quality": p["quality"], "shape": p["shape"]})
            rec_probs = np.array(rec_probs)
            child_probs.append((rec_probs[:, PRESENT].max(), rec_probs[:, UNKNOWN].max()))
        child_probs = np.array(child_probs)
        pred = decide(child_probs, tp, tu)
        m = screening_metrics(y, pred, child_probs[:, 0])
        m["beat_pass"] = passed / total
        results["conditions"][cond].append(m)
        if cond == "clean":
            for p, lab in zip(test_p, pred):
                results["children"].append({"id": p["id"], "label": int(p["label"]), "pred": int(lab),
                                            "grade": p["grade"], "timing": p["timing"], "age": p["age"]})
    return results["conditions"]["clean"][-1]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true")
    ap.add_argument("--robust", action="store_true", help="train with noise and phone-like damage")
    ap.add_argument("--out", default=str(ROOT / "web"), help="folder for model/, samples/ and quiz/")
    ap.add_argument("--no-final", action="store_true", help="only cross-validate, do not export a model")
    args = ap.parse_args()
    n_folds, epochs = (5, 1) if args.quick else (5, 20)
    rng = np.random.default_rng(SEED)
    noise_rng = np.random.default_rng(SEED + 1)   # separate, so training randomness is unchanged
    t0 = time.time()

    patients = load_patients()
    load_spectrograms(patients)
    if args.robust:
        for p in patients:
            for r in p["recordings"]:
                r["audio"] = wavfile.read(r["path"])[1]
    demo = pick_demo_patients(patients, rng)
    demo_ids = {p["id"] for p in demo}
    pool = [p for p in patients if p["id"] not in demo_ids]
    n_recs = sum(len(p["recordings"]) for p in patients)
    print(f"{len(patients)} children, {n_recs} recordings; {len(demo)} held out as demo cases; "
          f"{'robust' if args.robust else 'clean-only'} training")

    y_pool = np.array([p["label"] for p in pool])
    folds = StratifiedKFold(n_folds, shuffle=True, random_state=SEED).split(np.zeros(len(pool)), y_pool)
```

```
    base_folds = []
    results = {"conditions": {c: [] for c in E.CONDITIONS}, "children": [], "recordings": []}

    for k, (tr_idx, te_idx) in enumerate(folds):
        train_p = [pool[i] for i in tr_idx]
        test_p = [pool[i] for i in te_idx]
        inner_train, inner_val = train_test_split(
            train_p, test_size=0.15, random_state=SEED, stratify=[p["label"] for p in train_p])
        train_recs = [r for p in inner_train for r in p["recordings"]]
        y_test = [p["label"] for p in test_p]
        print(f"Fold {k + 1}/{n_folds}: train {len(inner_train)}, val {len(inner_val)}, test {len(test_p)}
children")

        model, (tp, tu) = train_cnn(train_recs, inner_val, epochs, rng, robust=args.robust)
        m = evaluate_fold(model, tp, tu, test_p, noise_rng, results)

        base = train_baseline(train_recs)
        bt, _ = tune_thresholds(baseline_patient_probs(base, inner_val), [p["label"] for p in inner_val])
        bprobs = baseline_patient_probs(base, test_p)
        base_folds.append(screening_metrics(y_test, decide(bprobs, *bt), bprobs[:, 0]))
        damaged = " ".join(f"{c} {results['conditions'][c][-1]['weighted_accuracy']:.3f}" for c in E.CONDITIONS[1:
])
        print(f"  CNN weighted acc {m['weighted_accuracy']:.3f} (sens {m['sensitivity']:.2f}, "
              f"spec {m['specificity']:.2f}, AUC {m['auc']:.3f})  |  baseline {base_folds[-1][
'weighted_accuracy']:.3f}"
              f"  |  {damaged}  ({(time.time() - t0) / 60:.0f} min)", flush=True)
        if args.quick:
            break

    cond_summary = {c: summarize_robust(ms) for c, ms in results["conditions"].items()}
    cnn_summary, base_summary = summarize(results["conditions"]["clean"]), summarize(base_folds)
    oof_true = np.array([c["label"] for c in results["children"]])
    oof_pred = np.array([c["pred"] for c in results["children"]])
    confusion = [[int(np.sum((oof_true == t) & (oof_pred == p))) for p in range(3)] for t in range(3)]
    analysis = {
        "by_grade": E.breakdown(results["children"], "grade", ["I/VI", "II/VI", "III/VI"], PRESENT, UNKNOWN,
ABSENT),
        "by_timing": E.breakdown(results["children"], "timing",
                                 ["Early-systolic", "Mid-systolic", "Late-systolic", "Holosystolic"],
                                 PRESENT, UNKNOWN, ABSENT),
        "by_age": E.by_age(results["children"], PRESENT, ABSENT),
        "robustness": [{"condition": c, "name": E.CONDITION_NAMES[c], **cond_summary[c]} for c in E.CONDITIONS],
    }
    print("CNN     ", {k: round(v["mean"], 3) for k, v in cnn_summary.items()})
    print("Baseline", {k: round(v["mean"], 3) for k, v in base_summary.items()})
    print(json.dumps(analysis, indent=1), flush=True)

    out = Path(args.out)
    quiz = E.pick_quiz(results["recordings"], PRESENT, ABSENT, np.random.default_rng(SEED + 2))
    model = None
    if not args.no_final:
        print("Training final model on all non-demo children...", flush=True)
        final_train, final_val = train_test_split(pool, test_size=0.15, random_state=SEED, stratify=y_pool)
        model, (tp, tu) = train_cnn([r for p in final_train for r in p["recordings"]], final_val, epochs, rng,
                                    robust=args.robust)
    export(out, model, tp, tu, cnn_summary, base_summary, confusion, n_folds, len(pool), demo, len(patients),
           n_recs, analysis, quiz, args.robust)
    print(f"Done in {(time.time() - t0) / 60:.1f} min")


def summarize_robust(fold_metrics):
    keys = ["weighted_accuracy", "sensitivity", "specificity", "auc", "beat_pass"]
    return {k: float(np.mean([m[k] for m in fold_metrics])) for k in keys}


def export(out, model, tp, tu, cnn_summary, base_summary, confusion, n_folds, n_pool, demo, n_patients, n_recs,
           analysis, quiz, robust):
    model_dir, samples_dir, quiz_dir = out / "model", out / "samples", out / "quiz"
    for d in (model_dir, samples_dir, quiz_dir):
        d.mkdir(parents=True, exist_ok=True)
    if model is not None:
        dummy = torch.zeros(1, 1, F.N_MELS, F.WINDOW_FRAMES)
        torch.onnx.export(WithSoftmax(model).eval(), dummy, str(model_dir / "murmur_cnn.onnx"),
                          input_names=["logmel"], output_names=["probs"],
                          dynamic_axes={"logmel": {0: "batch"}, "probs": {0: "batch"}},
                          opset_version=17, dynamo=False)
        config = {
            "features": {
                "sample_rate": F.SAMPLE_RATE, "n_fft": F.N_FFT, "hop": F.HOP, "n_mels": F.N_MELS,
                "window_frames": F.WINDOW_FRAMES, "window_step": F.WINDOW_STEP, "eps": F.EPS,
                "mel_filterbank": np.round(F.MEL_FB, 7).tolist(),
            },
            "classes": CLASSES,
            "thresholds": {"present": tp, "unknown": tu},
        }
        (model_dir / "config.json").write_text(json.dumps(config))

    demo_cases = []
    for p in demo:
        files = []
        for r in p["recordings"]:
```

```
            shutil.copy(r["path"], samples_dir / r["path"].name)
            files.append({"file": r["path"].name, "location": r["location"]})
        demo_cases.append({"id": p["id"], "truth": CLASSES[p["label"]], "age": p["age"], "sex": p["sex"],
                           "murmur_locations": p["murmur_locations"], "timing": p["timing"],
                           "grade": p["grade"], "recordings": files})

    quiz_items = []
    for r in quiz:
        shutil.copy(DATA_DIR / "training_data" / r["file"], quiz_dir / r["file"])
        quiz_items.append({"file": r["file"], "location": r["location"], "murmur": r["label"] == PRESENT,
                           "grade": r["grade"], "timing": r["timing"], "pitch": r["pitch"],
                           "quality": r["quality"], "shape": r["shape"],
                           "ai_present_prob": round(r["present_prob"], 3), "ai_flags": r["ai_flags"]})

    metrics = {
        "dataset": {"children": n_patients, "recordings": n_recs, "cv_children": n_pool, "demo_children": len(
demo)},
        "training": "robust" if robust else "clean",
        "folds": n_folds,
        "cnn": cnn_summary,
        "baseline": base_summary,
        "confusion": confusion,
        "challenge_reference": {"top_hidden_test": 0.780, "note": "Best team, PhysioNet 2022 hidden test set"},
        "analysis": analysis,
        "demo_cases": demo_cases,
        "quiz": quiz_items,
    }
    (model_dir / "metrics.json").write_text(json.dumps(metrics, indent=1))
    (model_dir / "metrics.js").write_text("const METRICS = " + json.dumps(metrics) + ";\n")
    print(f"Exported to {out}: model={'yes' if model is not None else 'no'}, thresholds ({tp}, {tu}), "
          f"{len(demo_cases)} demo cases, {len(quiz_items)} quiz recordings")


if __name__ == "__main__":
    main()
```

ml/evaluation.py

Damaged-audio test conditions, training augmentation, error analysis, quiz selection.

```
"""
Helpers for the extra evaluation that train.py runs inside cross-validation:

- degraded test audio (noise, phone-like microphone, handling rumble)
- training-time augmentation for the noise-robust model
- missed murmurs by grade / timing / age
- picking recordings for the "Train your ear" quiz
"""

import json
from pathlib import Path

import numpy as np
from scipy.signal import butter, sosfilt

import features as F
from heartbeat_check import beat_measures

ROOT = Path(__file__).resolve().parent.parent
CHECK = json.loads((ROOT / "web" / "model" / "heartbeat_check.json").read_text())

# Test conditions. "noise*" and "phone" are the same kinds of damage the robust
# model sees in training; "rumble" (low-frequency handling noise) is not, so it
# shows whether robustness carries over to damage the model was never shown.
CONDITIONS = ["clean", "noise20", "noise10", "noise5", "phone", "rumble"]
CONDITION_NAMES = {
    "clean": "Original recordings",
    "noise20": "Background noise, 20 dB SNR",
    "noise10": "Background noise, 10 dB SNR",
    "noise5": "Background noise, 5 dB SNR",
    "phone": "Phone-like microphone (no sound below 150 Hz)",
    "rumble": "Handling rumble, 10 dB SNR (never seen in training)",
}
PHONE = butter(4, 150, "highpass", fs=F.SAMPLE_RATE, output="sos")
RUMBLE_SHAPE = butter(2, [5, 60], "bandpass", fs=F.SAMPLE_RATE, output="sos")


def add_noise(x, snr_db, rng, noise=None):
    noise = rng.normal(size=len(x)) if noise is None else noise
    scale = np.sqrt(np.mean(x ** 2) / (np.mean(noise ** 2) + 1e-12)) / 10 ** (snr_db / 20)
    return x + noise * scale


def degrade(x, condition, rng):
    if condition == "clean":
        return x
    if condition == "phone":
        return sosfilt(PHONE, x)
    if condition == "rumble":
        return add_noise(x, 10, rng, sosfilt(RUMBLE_SHAPE, rng.normal(size=len(x))))
    return add_noise(x, {"noise20": 20, "noise10": 10, "noise5": 5}[condition], rng)


WINDOW_SAMPLES = F.N_FFT + (F.WINDOW_FRAMES - 1) * F.HOP   # 4 seconds
MARGIN = 1000                                               # lets the filter settle


def augmented_window(audio, rng):
    """A random 4-second training window, damaged at random like a cheap
    microphone in a noisy room would: a missing low end (high-pass at a random
    60-250 Hz) and/or background noise (3-30 dB SNR). 40% stay clean."""
    need = WINDOW_SAMPLES + MARGIN
    if len(audio) < need:
        audio = np.tile(audio, int(np.ceil(need / len(audio))))
    start = int(rng.integers(0, len(audio) - need + 1))
    x = audio[start:start + need].astype(np.float64) / 32768.0
    if rng.random() < 0.6:
        if rng.random() < 0.5:
            x = sosfilt(butter(4, rng.uniform(60, 250), "highpass", fs=F.SAMPLE_RATE, output="sos"), x)
        if rng.random() < 0.6:
            x = add_noise(x, rng.uniform(3, 30), rng)
    return F.log_mel(x[MARGIN:])


def beat_ok(spec):
    rhythm, depth = beat_measures(spec)
    return bool(rhythm >= CHECK["rhythm_min"] and depth >= CHECK["depth_min"])


def breakdown(children, key, order, present, unknown, absent):
    """For children with a murmur: how many were referred, asked to re-record, or missed."""
    out = []
    for value in order:
        group = [c for c in children if c["label"] == present and c[key] == value]
        if group:
            out.append({"group": value, "n": len(group),
                        "referred": sum(c["pred"] == present for c in group),
```

```
                        "rerecord": sum(c["pred"] == unknown for c in group),
                        "missed": sum(c["pred"] == absent for c in group)})
    return out


def by_age(children, present, absent):
    out = []
    for age in ["Neonate", "Infant", "Child", "Adolescent"]:
        m = [c for c in children if c["age"] == age and c["label"] == present]
        n = [c for c in children if c["age"] == age and c["label"] == absent]
        if m or n:
            out.append({"group": age, "murmur": len(m), "no_murmur": len(n),
                        "sensitivity": sum(c["pred"] == present for c in m) / len(m) if m else None,
                        "specificity": sum(c["pred"] != present for c in n) / len(n) if n else None})
    return out


def pick_quiz(recordings, present, absent, rng):
    """9 murmur recordings (3 per grade) and 9 without, all with a clear heartbeat.
    Each carries the answer of the cross-validation model that never heard it."""
    usable = [r for r in recordings if r["beat_ok"]]
    picks = []
    for grade in ["III/VI", "II/VI", "I/VI"]:
        pool = [r for r in usable if r["label"] == present and r["grade"] == grade]
        picks += list(rng.choice(pool, 3, replace=False))
    pool = [r for r in usable if r["label"] == absent and r["grade"] in ("nan", "")]
    picks += list(rng.choice(pool, 9, replace=False))
    return [picks[i] for i in rng.permutation(len(picks))]
```

ml/heartbeat_check.py

Safety check calibration: refuse recordings without a heartbeat rhythm.

```
"""
Safety check: does a recording contain a heartbeat rhythm at all?

The CNN was only trained on heart sounds, so it can give confident but
meaningless answers for speech, noise or a phone lying on a table. Before
trusting the CNN, we look for a repeating beat between 40 and 220 beats per
minute in the loudness of the low frequencies (where S1 and S2 live).

The threshold is calibrated so that 98% of real CirCor recordings pass,
then tested on signals that are not heart sounds.

Run:  python ml/heartbeat_check.py      (after ml/train.py has cached spectrograms)
"""

import json
from pathlib import Path

import numpy as np

import features as F

ROOT = Path(__file__).resolve().parent.parent
FRAME_RATE = F.SAMPLE_RATE / F.HOP           # 62.5 spectrogram frames per second
LOW_BANDS = 12                               # mel bands up to roughly 200 Hz
MIN_LAG = int(FRAME_RATE * 60 / 220)         # 220 bpm
MAX_LAG = int(FRAME_RATE * 60 / 40)          # 40 bpm


def beat_measures(spec):
    """Returns (rhythm, depth) of the low-frequency loudness curve.
    rhythm: highest autocorrelation at a heartbeat-like delay (0 = none, 1 = perfectly regular).
    depth:  how much the loudness rises and falls (standard deviation, in log10 units);
            heartbeats are loud clicks with quiet gaps, a steady tone is flat."""
    env = spec[:LOW_BANDS].mean(axis=0).astype(np.float64)
    env = env - env.mean()
    denom = np.dot(env, env)
    depth = float(np.sqrt(denom / max(1, len(env))))
    if denom <= 0 or len(env) <= MAX_LAG + 1:
        return 0.0, depth
    best = 0.0
    for lag in range(MIN_LAG, MAX_LAG + 1):
        best = max(best, np.dot(env[:-lag], env[lag:]) / denom)
    return float(best), depth


def synthetic_negatives(rng, seconds=15):
    t = np.arange(F.SAMPLE_RATE * seconds) / F.SAMPLE_RATE
    n = len(t)
    syllables = np.zeros(n)
    pos = 0
    while pos < n:  # irregular syllables (0.08-0.35 s) separated by irregular gaps
        length = int(rng.uniform(0.08, 0.35) * F.SAMPLE_RATE)
        syllables[pos:pos + length] = rng.uniform(0.5, 1.0)
        pos += length + int(rng.uniform(0.03, 0.4) * F.SAMPLE_RATE)
    pitch = 120 + 30 * np.sin(2 * np.pi * 0.3 * t)
    return {
        "white noise": rng.normal(size=n),
        "pink-ish noise": np.cumsum(rng.normal(size=n)) * 0.01 + rng.normal(size=n),
        "steady tone 440 Hz": np.sin(2 * np.pi * 440 * t),
        "mains hum 50 Hz": np.sin(2 * np.pi * 50 * t) + 0.1 * rng.normal(size=n),
        "speech-like babble": syllables * np.sin(2 * np.pi * np.cumsum(pitch) / F.SAMPLE_RATE) + 0.05 * rng.
normal(size=n),
        "silence with hiss": 0.001 * rng.normal(size=n),
    }


def main():
    specs = np.load(ROOT / "data" / "spectrograms.npz", allow_pickle=True)["specs"].item()
    real = np.array([beat_measures(s) for s in specs.values()])
    # The rhythm limit lets 97% of real recordings through, the depth limit 99%.
    t_rhythm = float(np.round(np.percentile(real[:, 0], 3), 3))
    t_depth = float(np.round(np.percentile(real[:, 1], 1), 3))
    passes = lambda m: m[0] >= t_rhythm and m[1] >= t_depth
    real_pass = float(np.mean([passes(m) for m in real]))
    print(f"{len(real)} real recordings: median rhythm {np.median(real[:, 0]):.3f}, depth {np.median(real[:, 1]):.
3f}")
    print(f"limits: rhythm >= {t_rhythm}, depth >= {t_depth}; real recordings passing: {real_pass:.1%}")

    negatives = {}
    trials = [synthetic_negatives(np.random.default_rng(seed)) for seed in range(10)]
    for name in trials[0]:
        rejected = sum(not passes(beat_measures(F.log_mel(t[name]))) for t in trials)
        negatives[name] = {"rejected": rejected, "trials": len(trials)}
        print(f"  {name:20s} rejected {rejected}/{len(trials)}")

    out = {
        "rhythm_min": t_rhythm,
```

```
        "depth_min": t_depth,
        "low_bands": LOW_BANDS,
        "min_lag": MIN_LAG,
        "max_lag": MAX_LAG,
        "real_pass_rate": real_pass,
        "real_recordings": int(len(real)),
        "negatives": negatives,
    }
    (ROOT / "web" / "model" / "heartbeat_check.json").write_text(json.dumps(out, indent=1))
    print("Saved web/model/heartbeat_check.json")


if __name__ == "__main__":
    main()
```

ml/external_validation.py

Tests the exported model on the PhysioNet 2016 dataset it never saw.

```
"""
External validation on data LittleHeart has never seen: the PhysioNet/CinC
Challenge 2016 heart sound database (3,240 recordings from six research
groups in different countries, mostly adults, different stethoscopes).

Important: its labels are "normal" vs "abnormal heart" (confirmed by
cardiac diagnosis), not "murmur" vs "no murmur". Many abnormal hearts have
a murmur, but not all, so this tests whether the murmur score transfers to
new devices and populations, not the exact screening task.

Runs the exported ONNX model with the same steps as the browser.

Run:  python ml/download_data.py --external
      python ml/external_validation.py [model folder, default web/model]
"""

import csv
import json
import sys
from pathlib import Path

import numpy as np
import onnxruntime as ort
from scipy.io import wavfile
from scipy.signal import resample_poly
from sklearn.metrics import roc_auc_score

import evaluation as E
import features as F

ROOT = Path(__file__).resolve().parent.parent
DATA = ROOT / "data" / "physionet2016"


def load_4k(path):
    rate, x = wavfile.read(path)
    x = x.astype(np.float64) / 32768.0
    if rate != F.SAMPLE_RATE:
        x = resample_poly(x, F.SAMPLE_RATE, rate)
    return x


def recording_present_prob(session, spec):
    windows = np.stack([F.cut_window(spec, s) for s in F.window_starts(spec.shape[1])])[:, None]
    probs = session.run(None, {"logmel": windows.astype(np.float32)})[0]
    return float(probs.mean(axis=0)[1])


def summarize(rows, threshold):
    ok = [r for r in rows if r["beat_ok"]]
    y = np.array([r["abnormal"] for r in ok])
    p = np.array([r["present_prob"] for r in ok])
    flagged = p >= threshold
    out = {"recordings": len(rows), "abnormal_share": float(np.mean([r["abnormal"] for r in rows])),
           "beat_pass": len(ok) / len(rows)}
    if 0 < y.sum() < len(y):
        out |= {"auc": float(roc_auc_score(y, p)),
                "sensitivity": float(flagged[y == 1].mean()),
                "specificity": float((~flagged[y == 0]).mean())}
    return out


def main():
    model_dir = Path(sys.argv[1]) if len(sys.argv) > 1 else ROOT / "web" / "model"
    session = ort.InferenceSession(str(model_dir / "murmur_cnn.onnx"))
    threshold = json.loads((model_dir / "config.json").read_text())["thresholds"]["present"]

    rows = []
    for folder in sorted(DATA.glob("training-*")):
        for name, label in csv.reader(open(folder / "REFERENCE.csv")):
            spec = F.log_mel(load_4k(folder / f"{name}.wav"))
            rows.append({"subset": folder.name[-1], "abnormal": label.strip() == "1",
                         "beat_ok": E.beat_ok(spec), "present_prob": recording_present_prob(session, spec)})
        print(f"{folder.name}: {sum(r['subset'] == folder.name[-1] for r in rows)} recordings", flush=True)

    result = {
        "dataset": "PhysioNet/CinC Challenge 2016 training set (normal vs abnormal heart sounds)",
        "threshold": threshold,
        "overall": summarize(rows, threshold),
        "subsets": {s: summarize([r for r in rows if r["subset"] == s], threshold)
                    for s in sorted({r["subset"] for r in rows})},
    }
    (model_dir / "external.json").write_text(json.dumps(result, indent=1))
    print(json.dumps(result, indent=1))


if __name__ == "__main__":
```

```
88      main()
```

ml/compare_models.py

Adds the clean-trained model's results for before/after comparison.

```
"""
Adds the clean-trained model's results to the deployed (robust) model's
metrics, so the Validation tab can show before -> after for damaged audio
and for the external dataset.

Inputs:
  clean metrics.json   from: python ml/train.py --no-final --out data/eval_clean
  clean external.json  from: python ml/external_validation.py <folder with the clean-trained ONNX model>
Run after the deployed model was trained (python ml/train.py --robust) and
externally tested (python ml/external_validation.py).

Usage: python ml/compare_models.py data/eval_clean/model/metrics.json data/external_clean_model.json
"""

import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DEPLOYED = ROOT / "web" / "model"


def main():
    clean = json.loads(Path(sys.argv[1]).read_text())
    clean_external = json.loads(Path(sys.argv[2]).read_text())
    metrics = json.loads((DEPLOYED / "metrics.json").read_text())
    metrics["comparison"] = {
        "note": "Same cross-validation folds, model trained on clean audio only",
        "cnn": clean["cnn"],
        "robustness": clean["analysis"]["robustness"],
        "external": {"overall": clean_external["overall"], "subsets": clean_external["subsets"]},
    }
    (DEPLOYED / "metrics.json").write_text(json.dumps(metrics, indent=1))
    (DEPLOYED / "metrics.js").write_text("const METRICS = " + json.dumps(metrics) + ";\n")
    for c, r in zip(clean["analysis"]["robustness"], metrics["analysis"]["robustness"]):
        print(f"{r['condition']:8s} score {c['weighted_accuracy']:.3f} -> {r['weighted_accuracy']:.3f}   "
              f"sens {c['sensitivity']:.2f} -> {r['sensitivity']:.2f}   spec {c['specificity']:.2f} -> {r['specificity']:.2f}")


if __name__ == "__main__":
    main()
```

web/index.html

Page structure: screening, ear training, screening log, validation and education tabs.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>LittleHeart</title>
  <meta name="description" content="AI heart murmur screening for children, running privately in the browser.">
  <meta name="theme-color" content="#c2364f">
  <link rel="manifest" href="manifest.webmanifest">
  <link rel="icon" href="icons/icon.svg" type="image/svg+xml">
  <link rel="apple-touch-icon" href="icons/icon-192.png">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <div class="wrap header-row">
      <div class="brand">
        <svg viewBox="0 0 32 32" class="logo" aria-hidden="true"><path d="M16 28s-11-7-11-15a6 6 0 0 1 11-3 6 6 0 0 1 11 3c0 8-11 15-11 15z"/><path d="M6 16h6l2-4 3 8 2-4h7" class="logo-line"/></svg>
        <span>LittleHeart</span>
        <select id="lang" class="lang" aria-label="Language">
          <option value="en">EN</option>
          <option value="tr">TR</option>
          <option value="pt">PT</option>
          <option value="es">ES</option>
        </select>
      </div>
      <nav class="tabs" role="tablist">
        <button class="tab active" data-tab="screen" role="tab" data-i18n="tab.screen">Screen a child</button>
        <button class="tab" data-tab="train" role="tab" data-i18n="tab.train">Train your ear</button>
        <button class="tab" data-tab="log" role="tab" data-i18n="tab.log">Screening log</button>
        <button class="tab" data-tab="validation" role="tab" data-i18n="tab.validation">Validation</button>
        <button class="tab" data-tab="learn" role="tab" data-i18n="tab.learn">About murmurs</button>
      </nav>
    </div>
  </header>

  <main class="wrap">
    <!-- ================== SCREEN ================== -->
    <section id="tab-screen" class="panel active">
      <div class="intro">
        <h1 data-i18n="screen.title">Hear the heart problems that hide in children.</h1>
        <p data-i18n="screen.lead">LittleHeart listens to stethoscope recordings from the four heart valve areas and flags children whose heart
          sounds contain a murmur, so a health worker knows who needs an echocardiogram. The AI runs on this device; no recording is uploaded.</p>
      </div>

      <div class="screen-grid">
        <div class="card input-card">
          <h2 data-i18n="screen.step1">1. Add recordings</h2>
          <div class="source-tabs" role="tablist">
            <button class="source active" data-source="record" data-i18n="source.record">Record</button>
            <button class="source" data-source="demo" data-i18n="source.demo">Real cases</button>
            <button class="source" data-source="upload" data-i18n="source.upload">Upload</button>
          </div>

          <div class="source-panel active" id="source-record">
            <p class="muted small" data-i18n="record.lead">Use a digital stethoscope, or a stethoscope with a phone adapter. LittleHeart guides you through the four
              listening sites and checks the sound quality live.</p>
            <div id="guide-idle">
              <button class="btn wide" id="guide-start" data-i18n="record.start">● Start guided screening</button>
              <details class="one-site">
                <summary data-i18n="record.oneSite">Record a single site instead</summary>
                <div class="record-row">
                  <select id="record-site" aria-label="Valve area">
                    <option value="AV" data-i18n="site.AV.long">Aortic (AV)</option>
                    <option value="PV" data-i18n="site.PV.long">Pulmonary (PV)</option>
                    <option value="TV" data-i18n="site.TV.long">Tricuspid (TV)</option>
                    <option value="MV" data-i18n="site.MV.long">Mitral (MV)</option>
                  </select>
                  <button class="btn" id="record-one" data-i18n="record.one">● Record 15 s</button>
                </div>
              </details>
            </div>
            <div id="guide-live" class="guide-live" hidden>
              <div class="guide-step" id="guide-step"></div>
              <div class="guide-instruction" id="guide-instruction"></div>
              <canvas id="live-wave" class="live-wave" aria-hidden="true"></canvas>
              <div class="live-row">
                <span id="beat-chip" class="chip-status"></span>
                <span id="level-meter" class="level"><span></span></span>
                <span id="countdown" class="countdown"></span>
              </div>
              <div class="guide-actions">
                <button class="btn-ghost" id="guide-redo" data-i18n="record.redo">Restart this site</button>
```

```
                <button class="btn-ghost" id="guide-skip" data-i18n="record.skip">Skip site</button>
                <button class="btn-ghost" id="guide-stop" data-i18n="record.stop">Stop</button>
              </div>
            </div>
            <p id="record-status" class="small muted" aria-live="polite"></p>
          </div>

          <div class="source-panel" id="source-demo">
            <p class="muted small" data-i18n="demo.lead">Children from the CirCor dataset that the AI never saw during training. The cardiologist's finding is revealed after the analysis.</p>
            <div id="demo-cases" class="demo-cases"></div>
          </div>

          <div class="source-panel" id="source-upload">
            <p class="muted small" data-i18n-html="upload.lead">Choose WAV/MP3 recordings. Name files like <code>name_AV.wav</code>, <code>name_MV.wav</code> to set the valve automatically.</p>
            <label class="file-drop" id="file-drop">
              <input type="file" id="file-input" accept="audio/*,.wav" multiple>
              <span data-i18n-html="upload.drop"><b>Choose files</b> or drop them here</span>
            </label>
          </div>

          <h2 class="mt" data-i18n="screen.step2">2. Listening sites</h2>
          <div class="chest-wrap">
            <svg viewBox="30 20 160 170" class="chest" id="chest" aria-label="Chest diagram with the four auscultation sites">
              <path class="torso" d="M40 20 Q110 0 180 20 L195 70 Q200 150 185 220 L35 220 Q20 150 25 70 Z"/>
              <path class="sternum" d="M110 30 L110 150"/>
              <path class="heart-shape" d="M95 85 Q80 70 95 62 Q110 58 112 75 Q125 60 140 70 Q150 85 135 110 L112 135 Q95 115 95 85Z"/>
              <g class="site" data-site="AV" transform="translate(92 62)"><circle r="13"/><text>AV</text></g>
              <g class="site" data-site="PV" transform="translate(128 62)"><circle r="13"/><text>PV</text></g>
              <g class="site" data-site="TV" transform="translate(118 118)"><circle r="13"/><text>TV</text></g>
              <g class="site" data-site="MV" transform="translate(150 132)"><circle r="13"/><text>MV</text></g>
            </svg>
            <ul class="site-legend">
              <li><b>AV</b> <span data-i18n="site.AV.where">aortic · 2nd space, right of sternum</span></li>
              <li><b>PV</b> <span data-i18n="site.PV.where">pulmonary · 2nd space, left of sternum</span></li>
              <li><b>TV</b> <span data-i18n="site.TV.where">tricuspid · lower left sternum</span></li>
              <li><b>MV</b> <span data-i18n="site.MV.where">mitral · apex, 5th space</span></li>
            </ul>
          </div>
        </div>

        <div class="results-col">
          <div class="card verdict-card" id="verdict-card">
            <h2 data-i18n="screen.step3">3. Result</h2>
            <div id="verdict" class="verdict empty"></div>
            <div id="truth" class="truth" hidden></div>
            <div class="verdict-actions" id="verdict-actions" hidden>
              <input id="child-id" class="child-id" data-i18n-placeholder="log.childId" placeholder="Child ID or initials" maxlength="40">
              <button class="btn-ghost" id="save-btn" data-i18n="log.save">Save to log</button>
              <button class="btn-ghost" id="print-btn" data-i18n="result.print">Print screening note</button>
              <button class="btn-ghost" id="clear-btn" data-i18n="result.new">New child</button>
            </div>
            <p class="print-only small" id="print-meta"></p>
          </div>

          <div id="site-results" class="site-results"></div>
        </div>
      </div>

      <p class="disclaimer" data-i18n="screen.disclaimer">LittleHeart is a research prototype for screening support, not a medical device and not a diagnosis.
        It was trained on children in north-east Brazil recorded with one stethoscope model, and in a test on other devices and mostly adult patients
        it worked much less well. A child who seems unwell should see a clinician whatever the result.</p>
    </section>

    <!-- ==================== TRAIN YOUR EAR ==================== -->
    <section id="tab-train" class="panel">
      <div class="intro">
        <h1 data-i18n="train.title">Can you hear a murmur?</h1>
        <p data-i18n="train.lead">Practise on 18 real children's heart sounds. Listen, decide, then see the cardiologist's answer and what the AI said.
          The AI answers come from a model that never heard that child, so you are competing fairly.</p>
      </div>
      <div class="train-grid">
        <div class="card" id="quiz-card"></div>
        <div class="train-side">
          <div class="card">
            <h2 data-i18n="train.score">Score</h2>
            <div class="scoreboard" id="scoreboard"></div>
          </div>
          <div class="card">
            <h2 data-i18n="train.howTitle">How to listen</h2>
            <ul class="tips">
              <li data-i18n="train.tip1">Find the rhythm first: "lub" (S1) then "dub" (S2), then a longer pause.</li>
              <li data-i18n="train.tip2">Now listen only to the short gap between lub and dub. Is it silent, or
```

```
is there a whoosh?</li>
              <li data-i18n="train.tip3">On the spectrogram, a murmur is a haze of colour between the bright vertical beats.</li>
              <li data-i18n="train.tip4">Use headphones. Laptop speakers cannot play the deepest heart sounds.</li>
            </ul>
          </div>
        </div>
      </div>
    </section>

    <!-- ================== SCREENING LOG ================== -->
    <section id="tab-log" class="panel">
      <div class="intro">
        <h1 data-i18n="log.title">Screening log</h1>
        <p data-i18n="log.lead">Keep track of a school or village screening day: who was screened, who needs an echocardiogram and who must be recorded again.
          The log is stored only on this device.</p>
      </div>
      <div class="grid four" id="log-stats"></div>
      <div class="card">
        <div class="log-toolbar">
          <div class="filter" role="group">
            <button class="chip active" data-filter="all" data-i18n="log.all">All</button>
            <button class="chip" data-filter="1" data-i18n="log.referred">Refer</button>
            <button class="chip" data-filter="2" data-i18n="log.rerecord">Record again</button>
            <button class="chip" data-filter="0" data-i18n="log.clear">No murmur</button>
          </div>
          <div class="log-actions">
            <button class="btn-ghost" id="log-export" data-i18n="log.export">Export CSV</button>
            <button class="btn-ghost" id="log-wipe" data-i18n="log.wipe">Delete all</button>
          </div>
        </div>
        <div class="table-wrap"><table class="log-table" id="log-table"></table></div>
      </div>
    </section>

    <!-- ================== VALIDATION ================== -->
    <section id="tab-validation" class="panel">
      <div class="intro">
        <h1 data-i18n="val.title">How well does it work?</h1>
        <p id="val-intro"></p>
      </div>

      <div class="grid four" id="val-cards"></div>

      <div class="grid two">
        <div class="card">
          <h2 data-i18n="val.compare">Compared with other approaches</h2>
          <p class="muted small" data-i18n="val.compareLead">Official PhysioNet 2022 murmur score (weighted accuracy). Higher is better.</p>
          <div id="compare" class="compare"></div>
          <p class="muted small" id="compare-note"></p>
        </div>
        <div class="card">
          <h2 data-i18n="val.confusion">Every child in cross-validation</h2>
          <p class="muted small" data-i18n="val.confusionLead">Rows: cardiologist's finding. Columns: LittleHeart's answer.</p>
          <div id="confusion" class="confusion"></div>
        </div>
      </div>

      <div class="grid two">
        <div class="card">
          <h2 data-i18n="val.missTitle">Which murmurs does it miss?</h2>
          <p class="muted small" data-i18n="val.missLead">Children with a murmur in cross-validation, by loudness grade (I = faintest, III = loud).</p>
          <div id="by-grade" class="stack-chart"></div>
          <div class="legend-row">
            <span><i class="sw bad"></i><span data-i18n="val.legendRefer">referred</span></span>
            <span><i class="sw warn"></i><span data-i18n="val.legendRerecord">asked to re-record</span></span>
            <span><i class="sw miss"></i><span data-i18n="val.legendMissed">missed</span></span>
          </div>
          <p class="small" id="grade-note"></p>
        </div>
        <div class="card">
          <h2 data-i18n="val.ageTitle">By age group</h2>
          <div class="table-wrap"><table class="data-table" id="by-age"></table></div>
        </div>
      </div>

      <div class="card">
        <h2 data-i18n="val.robustTitle">Does it survive real-world sound?</h2>
        <p class="muted small" id="robust-lead"></p>
        <div class="table-wrap"><table class="data-table" id="robustness"></table></div>
        <p class="small" id="robust-note"></p>
      </div>

      <div class="card">
        <h2 data-i18n="val.externalTitle">Test on a completely different dataset</h2>
        <p class="muted small" data-i18n="val.externalLead">PhysioNet/CinC Challenge 2016: 3,240 recordings
```

```
collected by six research groups in different countries,
250           mostly from adults, with different stethoscopes. LittleHeart never trained on any of it. Its labels are
"normal" or "abnormal heart",
251           which is not the same as "murmur": for example, coronary artery disease is abnormal but usually has no
murmur.</p>
252         <div class="table-wrap"><table class="data-table" id="external"></table></div>
253         <p class="small" id="external-note"></p>
254       </div>
255
256       <div class="grid two">
257         <div class="card">
258           <h2 data-i18n="val.howTitle">How the AI works</h2>
259           <ol class="steps">
260             <li data-i18n-html="val.how1"><b>Spectrogram.</b> Each recording is turned into a log-mel spectrogram:
 a picture of which pitches (25–1000 Hz) are loud every 16 ms.</li>
261             <li data-i18n-html="val.how2"><b>4-second windows.</b> The picture is cut into overlapping 4-second
windows, about 4–8 heartbeats each.</li>
262             <li data-i18n-html="val.how3"><b>CNN.</b> A small convolutional neural network (4 layers, about 61,
000 weights) scores each window: murmur, no murmur, or unclear.</li>
263             <li data-i18n-html="val.how4"><b>Trained for the real world.</b> During training, recordings are
randomly damaged with background noise and phone-like microphones, so the AI learns to cope with them.</li>
264             <li data-i18n-html="val.how5"><b>Per child.</b> Window scores are averaged per valve. If any valve
looks like a murmur, the child is referred; if a recording is unclear, it asks for a re-recording.</li>
265           </ol>
266         </div>
267         <div class="card">
268           <h2 data-i18n="val.testedTitle">How we tested it</h2>
269           <ul class="checks">
270             <li data-i18n-html="val.tested1"><b>Split by child, not recording.</b> All recordings of a child are
either in training or in testing, never both.</li>
271             <li data-i18n-html="val.tested2"><b>Cross-validation.</b> Every child is tested once by a model that
never heard them.</li>
272             <li data-i18n-html="val.tested3"><b>Thresholds chosen on separate data.</b> Decision cut-offs come
from a validation split inside the training folds.</li>
273             <li data-i18n-html="val.tested4"><b>Baseline.</b> A logistic regression on the average spectrum shows
how much the CNN adds.</li>
274             <li data-i18n-html="val.tested5"><b>Untouched demo cases.</b> The 10 real cases on the first tab were
picked at random and removed before any training.</li>
275             <li data-i18n-html="val.tested6"><b>External test.</b> A second, unrelated dataset checks how far the
results carry over.</li>
276           </ul>
277         </div>
278       </div>
279
280       <div class="card safety-card">
281         <h2 data-i18n="val.safetyTitle">Safety check: is it really a heartbeat?</h2>
282         <p class="muted small" data-i18n="val.safetyLead">The CNN has only ever heard heart sounds, so it could
give a confident but meaningless answer for speech or noise.
283           Before trusting it, LittleHeart checks that the low-pitched loudness rises and falls with a regular
rhythm between 40 and 220 beats per minute.
284           Recordings that fail are never called "murmur" or "no murmur"; the health worker is asked to record
again.</p>
285         <div id="safety" class="safety"></div>
286       </div>
287
288       <div class="card">
289         <h2 data-i18n="val.statusTitle">Honest status</h2>
290         <div class="status-grid">
291           <div><h3 class="ok-text" data-i18n="val.done">Working now</h3><ul>
292             <li data-i18n="val.done1">Cross-validated murmur model on 942 children, trained to cope with noise
and phone-like microphones</li>
293             <li data-i18n="val.done2">Guided recording with a live heartbeat and sound-level check</li>
294             <li data-i18n="val.done3">Per-valve explanation: spectrogram with murmur timeline</li>
295             <li data-i18n="val.done4">Ear-training mode, screening log with CSV export, 4 languages</li>
296             <li data-i18n="val.done5">Works offline after the first visit and can be installed like an app</li>
297             <li data-i18n="val.done6">Python and JavaScript code verified to give identical numbers</li>
298           </ul></div>
299           <div><h3 class="warn-text" data-i18n="val.notyet">Not done yet</h3><ul>
300             <li data-i18n="val.notyet1">Testing with real low-cost stethoscope adapters</li>
301             <li data-i18n="val.notyet2">Validation with children in a new region, ideally a prospective study</li>
302             <li data-i18n="val.notyet3">Telling innocent from pathological murmurs (needs echocardiography-
labelled data)</li>
303             <li data-i18n="val.notyet4">Review by paediatric cardiologists and regulatory approval</li>
304           </ul></div>
305           <div><h3 class="bad-text" data-i18n="val.limits">Known limits</h3><ul id="limits"></ul></div>
306         </div>
307       </div>
308     </section>
309
310     <!-- ================== LEARN ================== -->
311     <section id="tab-learn" class="panel">
312       <div class="intro">
313         <h1 data-i18n="learn.title">Why listen to children's hearts?</h1>
314         <p data-i18n="learn.lead">Congenital heart disease is the most common birth defect, and rheumatic heart
disease still damages the hearts of millions of
315           children and young people. Both are often first noticed as a murmur, but echocardiography and
cardiologists are scarce where these diseases are most common.</p>
316       </div>
317       <div class="grid three">
318         <article class="card"><h2 data-i18n="learn.q1">What is a murmur?</h2>
319           <p data-i18n="learn.a1">A whooshing sound between the normal "lub-dub" heart sounds, caused by
```

```
turbulent blood flow. Many murmurs in children are innocent, but some come from holes in the heart or damaged valves.</p></article>
        <article class="card"><h2 data-i18n="learn.q2">Who is LittleHeart for?</h2>
          <p data-i18n="learn.a2">Community health workers, school screening programmes and primary care clinics that have a stethoscope but no cardiologist, and need to decide which children to send for an echocardiogram.</p></article>
        <article class="card"><h2 data-i18n="learn.q3">Why AI?</h2>
          <p data-i18n="learn.a3">Hearing a murmur takes years of practice, and even trained listeners disagree. An AI that has heard thousands of recordings can give every health worker a consistent second opinion.</p></article>
        <article class="card"><h2 data-i18n="learn.q4">The four sites</h2>
          <p data-i18n="learn.a4">Each valve is best heard at one spot: aortic and pulmonary in the 2nd rib space either side of the breastbone, tricuspid at its lower left edge, and mitral at the apex of the heart.</p></article>
        <article class="card"><h2 data-i18n="learn.q5">Recording tips</h2>
          <p data-i18n="learn.a5">Use a quiet room, keep the child calm and still, press the stethoscope firmly on bare skin, and record at least 10 seconds per site. Crying and talking make recordings unclear.</p></article>
        <article class="card"><h2 data-i18n="learn.q6">What happens next?</h2>
          <p data-i18n="learn.a6">A "refer" result means the child should have an echocardiogram, the ultrasound test that shows the heart's structure. A "no murmur" result does not rule out heart disease.</p></article>
      </div>
      <p class="muted small" data-i18n="learn.data">Data: CirCor DigiScope Phonocardiogram Dataset (Oliveira et al., IEEE JBHI 2022) and PhysioNet/CinC Challenge 2016 (Liu et al. 2016), PhysioNet. Recordings used under their open licences.</p>
    </section>
  </main>

  <footer class="wrap footer">
    <span data-i18n="footer">LittleHeart · UnivaBio 2026: AI for Human Health · Research prototype, not a medical device</span>
    <span id="offline-status" class="offline-status" hidden></span>
  </footer>

  <script src="vendor/ort/ort.wasm.min.js"></script>
  <script src="model/metrics.js"></script>
  <script src="js/i18n.js"></script>
  <script src="js/features.js"></script>
  <script src="js/model.js"></script>
  <script src="js/draw.js"></script>
  <script src="js/app.js"></script>
  <script src="js/guided.js"></script>
  <script src="js/quiz.js"></script>
  <script src="js/log.js"></script>
  <script src="js/validation.js"></script>
  <script src="js/main.js"></script>
</body>
</html>
```

web/js/i18n.js

Translations (English, Turkish, Portuguese, Spanish).

```
// Translations. English page text lives in index.html; English text built in
// JavaScript lives in EN below. Other languages translate both. Placeholders
// like {n} are filled in by t(key, {n: 3}).

const I18N = (() => {
  const EN = {
    "site.AV": "Aortic", "site.PV": "Pulmonary", "site.TV": "Tricuspid", "site.MV": "Mitral", "site.Phc": "Other site",
    "label.0": "No murmur", "label.1": "Murmur", "label.2": "Unclear",
    "age.Neonate": "Newborn", "age.Infant": "Infant", "age.Child": "Child", "age.Adolescent": "Adolescent", "age.nan": "Young adult",
    "sex.Female": "female", "sex.Male": "male",
    "timing.Early-systolic": "early-systolic", "timing.Mid-systolic": "mid-systolic",
    "timing.Late-systolic": "late-systolic", "timing.Holosystolic": "holosystolic",
    "demo.case": "Case {n}", "demo.recs": "{n} recordings",
    "verdict.empty.title": "No recordings yet",
    "verdict.empty.text": "Start a guided recording, pick a real case or upload recordings.",
    "verdict.busy.title": "Analysing",
    "verdict.busy.text": "Turning sound into a spectrogram and running the AI on this device.",
    "verdict.pending": "Still analysing {n} recording(s).",
    "verdict.refer.title": "Murmur suspected: refer for echocardiogram",
    "verdict.refer.text": "Strongest at the {site} area ({p} murmur likelihood). Arrange an echocardiogram to check the heart's structure.",
    "verdict.nobeat.title": "No heartbeat found: check and record again",
    "verdict.nobeat.text": "LittleHeart could not find a regular heartbeat, so it will not guess. Press the stethoscope firmly on bare skin at the marked spot and keep the room quiet.",
    "verdict.unclear.title": "Recording unclear: record again",
    "verdict.unclear.text": "The AI could not judge {sites} reliably. Re-record in a quieter moment; if it stays unclear, refer to a clinician.",
    "verdict.unclear.nobeat": "No heartbeat rhythm was found at {sites}.",
    "verdict.clear.title": "No murmur detected",
    "verdict.clear.text": "Highest murmur likelihood {p} (referral threshold {th}). This does not rule out heart disease; follow usual care.",
    "truth.label": "Cardiologist's finding:", "truth.murmur": "murmur heard at {sites}", "truth.grade": "grade {g}",
    "truth.none": "no murmur", "truth.unknown": "the expert could not decide (unclear recording)",
    "truth.agree": "LittleHeart agreed ✓", "truth.disagree": "LittleHeart disagreed ✗",
    "card.busy": "analysing…", "card.errorBadge": "error", "card.error": "Could not read this audio file.",
    "card.nobeat": "No heartbeat found", "card.specAria": "Spectrogram", "card.timelineAria": "Murmur likelihood over time",
    "card.play": "Play recording", "card.beatOk": "heartbeat ✓", "card.beatNo": "no heartbeat rhythm",
    "card.murmur": "murmur", "card.unclear": "unclear",
    "card.timelineCaption": "murmur likelihood per 4 s window · dashed = referral threshold",
    "print.meta": "LittleHeart screening note · {date} · {n} recording(s) · Research prototype, not a diagnosis. Referral decisions remain with a clinician.",
    "live.searching": "Listening for a heartbeat…", "live.quiet": "Too quiet: press the stethoscope on bare skin",
    "live.beat": "Heartbeat found ✓",
    "guide.step": "Site {n} of {total}: {site}", "guide.single": "Single site: {site}",
    "guide.place.AV": "Place the stethoscope in the 2nd rib space, just right of the breastbone.",
    "guide.place.PV": "Place the stethoscope in the 2nd rib space, just left of the breastbone.",
    "guide.place.TV": "Place the stethoscope at the lower left edge of the breastbone.",
    "guide.place.MV": "Place the stethoscope at the apex: 5th rib space, below the middle of the left collarbone.",
    "guide.ready": "Get ready… recording starts in {s} s", "guide.recording": "Recording… {s} s left",
    "guide.takeName": "Recording at {site}", "guide.done": "All sites recorded. The result appears on the right.",
    "guide.denied": "Microphone permission was denied or no microphone was found.",
    "guide.stopped": "Recording stopped.",
    "quiz.truthNone": "No murmur", "quiz.truthMurmur": "Murmur",
    "quiz.pitch.Low": "low pitch", "quiz.pitch.Medium": "medium pitch", "quiz.pitch.High": "high pitch",
    "quiz.quality.Blowing": "blowing", "quiz.quality.Harsh": "harsh", "quiz.quality.Musical": "musical",
    "quiz.none": "No practice recordings are available.",
    "quiz.progress": "Recording {n} of {total}", "quiz.question": "Do you hear a murmur?",
    "quiz.hint": "Play it as often as you like. Headphones help.",
    "quiz.yes": "Murmur", "quiz.no": "No murmur", "quiz.right": "Correct!", "quiz.wrong": "Not quite.",
    "quiz.cardiologist": "Cardiologist:", "quiz.ai": "AI:",
    "quiz.aiYes": "murmur likelihood {p}, so it said murmur", "quiz.aiNo": "murmur likelihood {p}, so it said no murmur",
    "quiz.lookMurmur": "Look for the haze of colour between the bright vertical heartbeats: that is the murmur.",
    "quiz.lookClear": "Between the bright vertical heartbeats the picture stays dark: no extra sound.",
    "quiz.next": "Next recording", "quiz.finish": "See my result",
    "quiz.you": "You", "quiz.aiName": "LittleHeart AI",
    "quiz.scoreNote": "Same recordings for both. The AI never trained on these children.",
    "quiz.endTitle": "Practice complete",
    "quiz.endText": "You got {you} of {total} right. The AI got {ai} of {total}.",
    "quiz.endNote": "Faint murmurs are hard even for doctors. Practise again, or try the real cases on the first tab.",
    "quiz.again": "Practise again",
    "log.unnamed": "Child {n}", "log.save": "Save to log", "log.saved": "Saved ✓", "log.saveFailed": "Could not save",
    "log.status0": "No murmur", "log.status1": "Refer", "log.status2": "Record again",
    "log.statScreened": "Children screened", "log.statRefer": "Refer for echocardiogram",
    "log.statRerecord": "Record again", "log.statClear": "No murmur",
    "log.noneFilter": "No children match this filter.",
    "log.empty": "No screenings saved yet. After a result, type the child's ID and press \"Save to log\".",
    "log.colChild": "Child", "log.colDate": "Date", "log.colResult": "Result", "log.colSites": "Sites",
    "log.demo": "demo", "log.delete": "Delete", "log.confirmWipe": "Delete all saved screenings on this device?",
    "val.intro": "LittleHeart was tested with {folds}-fold cross-validation on {children} children ({recordings}
```

```
recordings in the full dataset). Each child was judged by a model that had never heard them.",
    "val.card1": "Challenge score", "val.card1Text": "PhysioNet 2022 weighted accuracy: murmurs count 5×, unclear 3×, normal 1×.",
    "val.card2": "Murmurs caught", "val.card2Text": "Children with a murmur that LittleHeart referred.",
    "val.card3": "Normal hearts cleared", "val.card3Text": "Children without a murmur that were not referred.",
    "val.card4Text": "How well murmur likelihood ranks children with and without a murmur (1.0 = perfect).",
    "val.barBaseline": "Simple baseline (ours)", "val.barClean": "CNN, clean training (ours)",
    "val.barOurs": "LittleHeart CNN (ours)", "val.barTop": "Best team, PhysioNet 2022*",
    "val.compareNote": "*Not a like-for-like comparison. The challenge's best score (team HearHeart, among 40 ranked murmur entries) was measured on a hidden test set we cannot access, and scores on that set are usually lower than cross-validation on the public data. It shows we are in the right range, not that we match the winner.",
    "val.gradeNote": "Of the {missed} murmurs that were missed completely, {faint} ({share}) were the faintest grade, I/VI. Loud grade III/VI murmurs were referred {loud} of the time.",
    "val.ageCol": "Age group", "val.ageMurmur": "With murmur", "val.ageCaught": "Caught",
    "val.ageNone": "Without murmur", "val.ageCleared": "Cleared",
    "val.robustLead": "Every cross-validation fold was re-tested with damaged copies of its unseen recordings.",
    "val.robustLeadCompare": "Every cross-validation fold was re-tested with damaged copies of its unseen recordings. Grey: model trained on clean audio only. Bold: the deployed model, trained with randomly damaged audio.",
    "val.robCond": "Test condition", "val.robScore": "Challenge score", "val.robBeat": "Heartbeat check passed",
    "val.robustNote": "Training with damaged audio raised the share of murmurs caught through a phone-like microphone from {before} to {after}. On handling rumble, a kind of damage it never saw in training, it did slightly worse (challenge score {rb} → {ra}), so robustness does not automatically carry over to new kinds of noise.",
    "val.extSet": "Data set", "val.extN": "Recordings", "val.extAbn": "Abnormal", "val.extSens": "Abnormal flagged",
    "val.extSpec": "Normal cleared", "val.extSubset": "Group {k}", "val.extAll": "All",
    "val.externalNote": "Overall AUC {auc}, and only {beat} of these recordings passed the heartbeat check. The model does not yet carry over well to adult recordings from other devices, which is why LittleHeart must be validated locally before it is used anywhere new. Grey numbers: model trained on clean audio only.",
    "val.safetyReal": "Real heart recordings ({n})", "val.accepted": "{p} accepted", "val.rejected": "{k}/{n} rejected",
    "val.limit1": "Misses about 1 in {n} murmurs in cross-validation, mostly faint ones",
    "val.limit2": "Trained on one region (Brazil), mostly children, one stethoscope",
    "val.limit3": "On a different dataset of mostly adult recordings from other devices, AUC fell to {auc}",
    "val.limit4": "Cross-validation on public data is not the challenge's hidden test set",
    "val.limit5": "Rhythmic sounds that are not heartbeats (e.g. a metronome) can pass the safety check",
    "val.limit6": "Scores above do not include the heartbeat check, which asks for a re-recording of about 4% of real recordings",
    "cond.clean": "Original recordings", "cond.noise20": "Background noise, 20 dB signal-to-noise",
    "cond.noise10": "Background noise, 10 dB signal-to-noise", "cond.noise5": "Background noise, 5 dB signal-to-noise",
    "cond.phone": "Phone-like microphone (nothing below 150 Hz)",
    "cond.rumble": "Handling rumble, 10 dB (never seen in training)",
    "neg.white noise": "White noise (synthetic)", "neg.pink-ish noise": "Pink noise (synthetic)",
    "neg.steady tone 440 Hz": "Steady tone 440 Hz (synthetic)", "neg.mains hum 50 Hz": "Mains hum 50 Hz (synthetic)",
    "neg.speech-like babble": "Speech-like babble (synthetic)", "neg.silence with hiss": "Silence with hiss (synthetic)",
    "load.failTitle": "AI model failed to load",
    "load.failText": "Open this page through a web server (see README), not by double-clicking the file.",
    "offline.ready": "✓ Works offline",
  };

  const TR = {
    "tab.screen": "Çocuk tara", "tab.train": "Kula■■n■ e■it", "tab.log": "Tarama kayd■", "tab.validation": "Do■rulama", "tab.learn": "Üfürümler hakk■nda",
    "screen.title": "Çocuklarda gizlenen kalp sorunlar■n■ duyun.",
    "screen.lead": "LittleHeart, dört kalp kapakç■■■ bölgesinden al■nan steteskop kay■tlar■n■ dinler ve kalp sesinde üfürüm olan çocuklar■ i■aretler. Böylece sa■l■k çal■■an■ kimin ekokardiyografiye ihtiyac■ oldu■unu bilir. Yapay zeka bu cihazda çal■■■r; hiçbir kay■t yüklenmez.",
    "screen.step1": "1. Kay■t ekle", "source.record": "Kaydet", "source.demo": "Gerçek vakalar", "source.upload": "Yükle",
    "record.lead": "Dijital steteskop ya da telefon adaptörlü bir steteskop kullan■n. LittleHeart sizi dört dinleme noktas■nda yönlendirir ve ses kalitesini canl■ kontrol eder.",
    "record.start": "● Rehberli taramay■ ba■lat", "record.oneSite": "Bunun yerine tek bir nokta kaydet",
    "site.AV.long": "Aort (AV)", "site.PV.long": "Pulmoner (PV)", "site.TV.long": "Triküspit (TV)", "site.MV.long": "Mitral (MV)",
    "record.one": "● 15 sn kaydet", "record.redo": "Bu noktay■ yeniden ba■lat", "record.skip": "Noktay■ atla", "record.stop": "Durdur",
    "demo.lead": "CirCor veri setinden, yapay zekan■n e■itimde hiç görmedi■i çocuklar. Kardiyolo■un bulgusu analizden sonra gösterilir.",
    "upload.lead": "WAV/MP3 kay■tlar■ seçin. Kapakç■■■n otomatik atanmas■ için dosyalar■ <code>isim_AV.wav</code>, <code>isim_MV.wav</code> gibi adland■r■n.",
    "upload.drop": "<b>Dosya seçin</b> ya da buraya b■rak■n",
    "screen.step2": "2. Dinleme noktalar■",
    "site.AV.where": "aort · 2. aral■k, gö■üs kemi■inin sa■■", "site.PV.where": "pulmoner · 2. aral■k, gö■üs kemi■inin solu",
    "site.TV.where": "triküspit · gö■üs kemi■inin sol alt kenar■", "site.MV.where": "mitral · kalp tepesi, 5. aral■k",
    "screen.step3": "3. Sonuç", "log.childId": "Çocuk kimli■i ya da ba■ harfleri", "log.save": "Kayda ekle",
    "result.print": "Tarama notunu yazd■r", "result.new": "Yeni çocuk",
    "screen.disclaimer": "LittleHeart tarama deste■i için bir ara■t■rma prototipidir; t■bbi cihaz ya da tan■ de■ildir. Kuzeydo■u Brezilya'daki çocuklarla, tek bir steteskop modeliyle e■itildi ve ba■ka cihazlarla, ço■unlukla yeti■kinlerden al■nan kay■tlarda çok daha zay■f çal■■t■. Kendini iyi hissetmeyen bir çocuk, sonuç ne olursa olsun bir hekime görünmelidir.",
    "train.title": "Üfürümü duyabiliyor musunuz?",
    "train.lead": "18 gerçek çocu■un kalp sesiyle pratik yap■n. Dinleyin, karar verin, sonra kardiyolo■un cevab■n■ ve yapay zekan■n ne dedi■ini görün. Yapay zekan■n cevaplar■ o çocu■u hiç duymam■■ bir modelden gelir, yani adil bir yar■■.",
```

```
    "train.score": "Skor", "train.howTitle": "Nas■l dinlenir?",
    "train.tip1": "Önce ritmi bulun: \"lup\" (S1), sonra \"dup\" (S2), ard■ndan daha uzun bir sessizlik.",
    "train.tip2": "■imdi yaln■zca lup ile dup aras■ndaki k■sa bo■lu■u dinleyin. Sessiz mi, yoksa bir h■■■rt■ m■ var?",
    "train.tip3": "Spektrogramda üfürüm, parlak dikey vuru■lar aras■ndaki renkli bir pus gibi görünür.",
    "train.tip4": "Kulakl■k kullan■n. Dizüstü bilgisayar hoparlörleri en kal■n kalp seslerini çalamaz.",
    "log.title": "Tarama kayd■",
    "log.lead": "Bir okul ya da köy tarama gününü takip edin: kim tarand■, kimin ekokardiyografiye ihtiyac■ var, kimin yeniden kaydedilmesi gerekiyor. Kay■t yaln■zca bu cihazda saklan■r.",
    "log.all": "Tümü", "log.referred": "Sevk", "log.rerecord": "Yeniden kay■t", "log.clear": "Üfürüm yok",
    "log.export": "CSV d■■a aktar", "log.wipe": "Tümünü sil",
    "val.title": "Ne kadar iyi çal■■■yor?", "val.compare": "Di■er yakla■■mlarla kar■■la■t■rma",
    "val.compareLead": "Resmi PhysioNet 2022 üfürüm skoru (a■■rl■kl■ do■ruluk). Yüksek olan daha iyi.",
    "val.confusion": "Çapraz do■rulamadaki her çocuk", "val.confusionLead": "Sat■rlar: kardiyolo■un bulgusu. Sütunlar: LittleHeart'■n cevab■.",
    "val.missTitle": "Hangi üfürümleri kaç■r■yor?", "val.missLead": "Çapraz do■rulamada üfürümü olan çocuklar, ■iddet derecesine göre (I = en hafif, III = yüksek).",
    "val.legendRefer": "sevk edildi", "val.legendRerecord": "yeniden kay■t istendi", "val.legendMissed": "kaç■r■ld■",
    "val.ageTitle": "Ya■ grubuna göre", "val.robustTitle": "Gerçek dünya seslerine dayan■yor mu?",
    "val.externalTitle": "Tamamen farkl■ bir veri setinde test",
    "val.externalLead": "PhysioNet/CinC Challenge 2016: farkl■ ülkelerdeki alt■ ara■t■rma grubunun, ço■unlukla yeti■kinlerden ve farkl■ steteskoplarla toplad■■■ 3.240 kay■t. LittleHeart bunlar■n hiçbiriyle e■itilmedi. Etiketleri \"normal\" ya da \"anormal kalp\"tir; bu \"üfürüm\" ile ayn■ ■ey de■ildir: örne■in koroner arter hastal■■■ anormaldir ama genellikle üfürüm yapmaz.",
    "val.howTitle": "Yapay zeka nas■l çal■■■yor?",
    "val.how1": "<b>Spektrogram.</b> Her kay■t bir log-mel spektrograma dönü■türülür: her 16 ms'de hangi seslerin (25–1000 Hz) yüksek oldu■unu gösteren bir resim.",
    "val.how2": "<b>4 saniyelik pencereler.</b> Resim, her biri 4–8 kalp at■■■ içeren, üst üste binen 4 saniyelik pencerelere bölünür.",
    "val.how3": "<b>CNN.</b> Küçük bir evri■imli sinir a■■ (4 katman, yakla■■k 61.000 a■■rl■k) her pencereyi puanlar: üfürüm, üfürüm yok ya da belirsiz.",
    "val.how4": "<b>Gerçek dünya için e■itildi.</b> E■itim s■ras■nda kay■tlar rastgele arka plan gürültüsü ve telefon mikrofonu benzeri bozulmalarla zorla■t■r■l■r; böylece yapay zeka bunlarla ba■a çı■kmay■ ö■renir.",
    "val.how5": "<b>Çocuk baz■nda.</b> Pencere puanlar■n■n her kapakç■k için ortalamas■ al■n■r. Herhangi bir kapakç■k üfürümlü görünürse çocuk sevk edilir; bir kay■t belirsizse yeniden kay■t istenir.",
    "val.testedTitle": "Nas■l test ettik?",
    "val.tested1": "<b>Kay■t de■il çocuk baz■nda bölme.</b> Bir çocu■un bütün kay■tlar■ ya e■itimde ya testte yer al■r, asla ikisinde birden.",
    "val.tested2": "<b>Çapraz do■rulama.</b> Her çocuk, onu hiç duymam■■ bir model taraf■ndan bir kez test edilir.",
    "val.tested3": "<b>E■ikler ayr■ veride seçildi.</b> Karar e■ikleri, e■itim katlar■n■n içindeki ayr■ bir do■rulama bölümünden gelir.",
    "val.tested4": "<b>Temel model.</b> Ortalama spektrum üzerinde bir lojistik regresyon, CNN'in ne kadar katk■ sa■lad■■■n■ gösterir.",
    "val.tested5": "<b>Dokunulmam■■ demo vakalar■.</b> ■lk sekmedeki 10 gerçek vaka rastgele seçildi ve her türlü e■itimden önce ayr■ld■.",
    "val.tested6": "<b>D■■ test.</b> ■lgisiz ikinci bir veri seti, sonuçlar■n ne kadar genellenebildi■ini kontrol eder.",
    "val.safetyTitle": "Güvenlik kontrolü: gerçekten kalp at■■■ m■?",
    "val.safetyLead": "CNN yaln■zca kalp seslerini duydu; konu■ma ya da gürültü için emin görünen ama anlams■z bir cevap verebilir. Ona güvenmeden önce LittleHeart, dü■ük frekansl■ ses ■iddetinin dakikada 40–220 at■ml■k düzenli bir ritimle inip ç■kt■■■n■ kontrol eder. Bu kontrolü geçemeyen kay■tlara asla \"üfürüm\" ya da \"üfürüm yok\" denmez; sa■l■k çal■■an■ndan yeniden kay■t istenir.",
    "val.statusTitle": "Dürüst durum", "val.done": "■u an çal■■anlar",
    "val.done1": "942 çocukla çapraz do■rulanm■■, gürültü ve telefon benzeri mikrofonlarla ba■a çı■kacak ■ekilde e■itilmi■ üfürüm modeli",
    "val.done2": "Canl■ kalp at■■■ ve ses seviyesi kontrolüyle rehberli kay■t",
    "val.done3": "Kapakç■k baz■nda aç■klama: üfürüm zaman çizelgeli spektrogram",
    "val.done4": "Kulak e■itimi modu, CSV d■■a aktarmal■ tarama kayd■, 4 dil",
    "val.done5": "■lk ziyaretten sonra internetsiz çal■■■r ve uygulama gibi yüklenebilir",
    "val.done6": "Python ve JavaScript kodunun ayn■ say■lar■ verdi■i do■ruland■",
    "val.notyet": "Henüz yap■lmad■",
    "val.notyet1": "Gerçek, dü■ük maliyetli steteskop adaptörleriyle test",
    "val.notyet2": "Yeni bir bölgedeki çocuklarla do■rulama, ideal olarak ileriye dönük bir çal■■ma",
    "val.notyet3": "Masum ve hastal■kl■ üfürümleri ay■rt etme (ekokardiyografi etiketli veri gerekir)",
    "val.notyet4": "Çocuk kardiyologlar■n■n incelemesi ve yasal onay",
    "val.limits": "Bilinen s■n■rlar",
    "learn.title": "Neden çocuklar■n kalbini dinlemeliyiz?",
    "learn.lead": "Do■u■tan kalp hastal■■■ en s■k görülen do■um kusurudur; romatizmal kalp hastal■■■ da hâlâ milyonlarca çocu■un ve gencin kalbine zarar veriyor. ■kisi de ço■u zaman ilk olarak bir üfürümle fark edilir, ama bu hastal■klar■n en yayg■n oldu■u yerlerde ekokardiyografi ve kardiyolog azd■r.",
    "learn.q1": "Üfürüm nedir?", "learn.a1": "Normal \"lup-dup\" kalp sesleri aras■nda, türbülansl■ kan ak■■■n■n yol açt■■■ h■■■rt■l■ bir ses. Çocuklardaki üfürümlerin ço■u masumdur, ama baz■lar■ kalpteki deliklerden ya da hasarl■ kapakç■klardan kaynaklan■r.",
    "learn.q2": "LittleHeart kimin için?", "learn.a2": "Steteskobu olan ama kardiyolo■u olmayan, hangi çocuklar■ ekokardiyografiye gönderece■ine karar vermesi gereken toplum sa■l■■■ çal■■anlar■, okul tarama programlar■ ve birinci basamak klinikleri.",
    "learn.q3": "Neden yapay zeka?", "learn.a3": "Üfürümü duymak y■llar süren pratik ister ve deneyimli dinleyiciler bile anla■amaz. Binlerce kay■t dinlemi■ bir yapay zeka her sa■l■k çal■■an■na tutarl■ bir ikinci görü■ verebilir.",
    "learn.q4": "Dört nokta", "learn.a4": "Her kapakç■k en iyi bir noktada duyulur: aort ve pulmoner gö■üs kemi■inin iki yan■nda 2. kaburga aral■■■nda, triküspit gö■üs kemi■inin sol alt kenar■nda, mitral ise kalbin tepesinde.",
    "learn.q5": "Kay■t ipuçlar■", "learn.a5": "Sessiz bir oda kullan■n, çocu■u sakin ve hareketsiz tutun, steteskobu ç■plak cilde s■k■ca bast■r■n ve her noktada en az 10 saniye kaydedin. A■lama ve konu■ma kay■tlar■ belirsiz hale getirir.",
    "learn.q6": "Sonra ne olur?", "learn.a6": "\"Sevk\" sonucu, çocu■un kalbin yap■s■n■ gösteren ultrason testi olan ekokardiyografiyi yapt■rmas■ gerekti■i anlam■na gelir. \"Üfürüm yok\" sonucu kalp hastal■■■n■ d■■lamaz.",
    "learn.data": "Veri: CirCor DigiScope Fonokardiyogram Veri Seti (Oliveira vd., IEEE JBHI 2022) ve PhysioNet/CinC Challenge 2016 (Liu vd. 2016), PhysioNet. Kay■tlar aç■k lisanslar■yla kullan■lm■■t■r.",
```

```
    "footer": "LittleHeart · UnivaBio 2026: ■nsan Sa■l■■■ için Yapay Zeka · Ara■t■rma prototipi, t■bbi cihaz de■ildir",
    "site.AV": "Aort", "site.PV": "Pulmoner", "site.TV": "Triküspit", "site.MV": "Mitral", "site.Phc": "Di■er nokta",
    "label.0": "Üfürüm yok", "label.1": "Üfürüm", "label.2": "Belirsiz",
    "age.Neonate": "Yenido■an", "age.Infant": "Bebek", "age.Child": "Çocuk", "age.Adolescent": "Ergen", "age.nan": "Genç yeti■kin",
    "sex.Female": "k■z", "sex.Male": "erkek",
    "timing.Early-systolic": "erken sistolik", "timing.Mid-systolic": "orta sistolik",
    "timing.Late-systolic": "geç sistolik", "timing.Holosystolic": "holosistolik",
    "demo.case": "Vaka {n}", "demo.recs": "{n} kay■t",
    "verdict.empty.title": "Henüz kay■t yok",
    "verdict.empty.text": "Rehberli kay■t ba■lat■n, gerçek bir vaka seçin ya da kay■t yükleyin.",
    "verdict.busy.title": "Analiz ediliyor",
    "verdict.busy.text": "Ses spektrograma dönü■türülüyor ve yapay zeka bu cihazda çal■■t■r■l■yor.",
    "verdict.pending": "{n} kay■t hâlâ analiz ediliyor.",
    "verdict.refer.title": "Üfürüm ■üphesi: ekokardiyografiye sevk edin",
    "verdict.refer.text": "En belirgin {site} bölgesinde ({p} üfürüm olas■l■■■). Kalbin yap■s■n■ kontrol etmek için ekokardiyografi ayarlay■n.",
    "verdict.nobeat.title": "Kalp at■■■ bulunamad■: kontrol edip yeniden kaydedin",
    "verdict.nobeat.text": "LittleHeart düzenli bir kalp at■■■ bulamad■, bu yüzden tahmin yürütmeyecek. Steteskobu i■aretli noktada ç■plak cilde s■k■ca bast■r■n ve oday■ sessiz tutun.",
    "verdict.unclear.title": "Kay■t belirsiz: yeniden kaydedin",
    "verdict.unclear.text": "Yapay zeka {sites} için güvenilir bir karar veremedi. Daha sessiz bir anda yeniden kaydedin; belirsizlik sürerse bir hekime yönlendirin.",
    "verdict.unclear.nobeat": "{sites} noktas■nda kalp ritmi bulunamad■.",
    "verdict.clear.title": "Üfürüm saptanmad■",
    "verdict.clear.text": "En yüksek üfürüm olas■l■■■ {p} (sevk e■i■i {th}). Bu, kalp hastal■■■n■ d■■lamaz; ola■an bak■ma devam edin.",
    "truth.label": "Kardiyolo■un bulgusu:", "truth.murmur": "{sites} noktalar■nda üfürüm duyuldu", "truth.grade": "{g}. derece",
    "truth.none": "üfürüm yok", "truth.unknown": "uzman karar veremedi (belirsiz kay■t)",
    "truth.agree": "LittleHeart ayn■ fikirde ✓", "truth.disagree": "LittleHeart farkl■ dü■ündü ✗",
    "card.busy": "analiz ediliyor…", "card.errorBadge": "hata", "card.error": "Bu ses dosyas■ okunamad■.",
    "card.nobeat": "Kalp at■■■ yok", "card.specAria": "Spektrogram", "card.timelineAria": "Zaman içinde üfürüm olas■l■■■",
    "card.play": "Kayd■ çal", "card.beatOk": "kalp at■■■ ✓", "card.beatNo": "kalp ritmi yok",
    "card.murmur": "üfürüm", "card.unclear": "belirsiz",
    "card.timelineCaption": "4 sn'lik pencere ba■■na üfürüm olas■l■■■ · kesikli çizgi = sevk e■i■i",
    "print.meta": "LittleHeart tarama notu · {date} · {n} kay■t · Ara■t■rma prototipidir, tan■ de■ildir. Sevk karar■ hekime aittir.",
    "live.searching": "Kalp at■■■ dinleniyor…", "live.quiet": "Çok sessiz: steteskobu ç■plak cilde bast■r■n",
    "live.beat": "Kalp at■■■ bulundu ✓",
    "guide.step": "Nokta {n}/{total}: {site}", "guide.single": "Tek nokta: {site}",
    "guide.place.AV": "Steteskobu 2. kaburga aral■■■na, gö■üs kemi■inin hemen sa■■na yerle■tirin.",
    "guide.place.PV": "Steteskobu 2. kaburga aral■■■na, gö■üs kemi■inin hemen soluna yerle■tirin.",
    "guide.place.TV": "Steteskobu gö■üs kemi■inin sol alt kenar■na yerle■tirin.",
    "guide.place.MV": "Steteskobu kalp tepesine yerle■tirin: 5. kaburga aral■■■, sol köprücük kemi■inin ortas■n■n alt■.",
    "guide.ready": "Haz■rlan■n… kay■t {s} sn içinde ba■l■yor", "guide.recording": "Kaydediliyor… {s} sn kald■",
    "guide.takeName": "{site} kayd■", "guide.done": "Bütün noktalar kaydedildi. Sonuç sa■da görünüyor.",
    "guide.denied": "Mikrofon izni reddedildi ya da mikrofon bulunamad■.", "guide.stopped": "Kay■t durduruldu.",
    "quiz.truthNone": "Üfürüm yok", "quiz.truthMurmur": "Üfürüm",
    "quiz.pitch.Low": "dü■ük perde", "quiz.pitch.Medium": "orta perde", "quiz.pitch.High": "yüksek perde",
    "quiz.quality.Blowing": "üfleyici", "quiz.quality.Harsh": "sert", "quiz.quality.Musical": "müzikal",
    "quiz.none": "Pratik kayd■ bulunmuyor.",
    "quiz.progress": "Kay■t {n}/{total}", "quiz.question": "Üfürüm duyuyor musunuz?",
    "quiz.hint": "■stedi■iniz kadar çal■n. Kulakl■k yard■mc■ olur.",
    "quiz.yes": "Üfürüm var", "quiz.no": "Üfürüm yok", "quiz.right": "Do■ru!", "quiz.wrong": "Tam de■il.",
    "quiz.cardiologist": "Kardiyolog:", "quiz.ai": "Yapay zeka:",
    "quiz.aiYes": "üfürüm olas■l■■■ {p}, bu yüzden üfürüm var dedi", "quiz.aiNo": "üfürüm olas■l■■■ {p}, bu yüzden üfürüm yok dedi",
    "quiz.lookMurmur": "Parlak dikey kalp at■■lar■ aras■ndaki renkli pusa bak■n: üfürüm odur.",
    "quiz.lookClear": "Parlak dikey kalp at■■lar■ aras■nda resim karanl■k kal■yor: fazladan ses yok.",
    "quiz.next": "Sonraki kay■t", "quiz.finish": "Sonucumu gör",
    "quiz.you": "Siz", "quiz.aiName": "LittleHeart yapay zekas■",
    "quiz.scoreNote": "■kiniz için de ayn■ kay■tlar. Yapay zeka bu çocuklarla hiç e■itilmedi.",
    "quiz.endTitle": "Pratik tamamland■",
    "quiz.endText": "{total} kayd■n {you} tanesini do■ru bildiniz. Yapay zeka {total} kayd■n {ai} tanesini bildi.",
    "quiz.endNote": "Hafif üfürümler hekimler için bile zordur. Tekrar pratik yap■n ya da ilk sekmedeki gerçek vakalar■ deneyin.",
    "quiz.again": "Tekrar pratik yap",
    "log.unnamed": "Çocuk {n}", "log.saved": "Kaydedildi ✓", "log.saveFailed": "Kaydedilemedi",
    "log.status0": "Üfürüm yok", "log.status1": "Sevk", "log.status2": "Yeniden kay■t",
    "log.statScreened": "Taranan çocuk", "log.statRefer": "Ekokardiyografiye sevk",
    "log.statRerecord": "Yeniden kay■t", "log.statClear": "Üfürüm yok",
    "log.noneFilter": "Bu filtreye uyan çocuk yok.",
    "log.empty": "Henüz kaydedilmi■ tarama yok. Bir sonuçtan sonra çocu■un kimli■ini yaz■p \"Kayda ekle\"ye bas■n.",
    "log.colChild": "Çocuk", "log.colDate": "Tarih", "log.colResult": "Sonuç", "log.colSites": "Noktalar",
    "log.demo": "demo", "log.delete": "Sil", "log.confirmWipe": "Bu cihazdaki bütün kay■tl■ taramalar silinsin mi?",
    "val.intro": "LittleHeart {children} çocuk üzerinde {folds} katl■ çapraz do■rulamayla test edildi (tüm veri setinde {recordings} kay■t). Her çocuk, onu hiç duymam■■ bir model taraf■ndan de■erlendirildi.",
    "val.card1": "Yar■■ma skoru", "val.card1Text": "PhysioNet 2022 a■■rl■kl■ do■ruluk: üfürümler 5×, belirsizler 3×, normaller 1× say■l■r.",
    "val.card2": "Yakalanan üfürümler", "val.card2Text": "Üfürümü olan ve LittleHeart'■n sevk etti■i çocuklar.",
    "val.card3": "Temize ç■kan normal kalpler", "val.card3Text": "Üfürümü olmayan ve sevk edilmeyen çocuklar.",
    "val.card4Text": "Üfürüm olas■l■■■n■n, üfürümü olan ve olmayan çocuklar■ ne kadar iyi s■ralad■■■ (1,0 = mükemmel).",
```

```
    "val.barBaseline": "Basit temel model (bizim)", "val.barClean": "CNN, temiz e■itim (bizim)",
    "val.barOurs": "LittleHeart CNN (bizim)", "val.barTop": "En iyi tak■m, PhysioNet 2022*",
    "val.compareNote": "*Birebir kar■■la■t■rma de■ildir. Yar■■man■n en iyi skoru (40 s■ralanm■■ üfürüm kat■l■m■ aras■nda HearHeart tak■m■) eri■emedi■imiz gizli bir test setinde ölçüldü ve o setteki skorlar genellikle aç■k verideki çapraz do■rulamadan dü■üktür. Do■ru aral■kta oldu■umuzu gösterir, birinciyle e■it oldu■umuzu de■il.",
    "val.gradeNote": "Tamamen kaç■r■lan {missed} üfürümün {faint} tanesi ({share}) en hafif derece olan I/VI idi. Yüksek sesli III/VI derece üfürümlerin {loud} kadar■ sevk edildi.",
    "val.ageCol": "Ya■ grubu", "val.ageMurmur": "Üfürümlü", "val.ageCaught": "Yakalanan",
    "val.ageNone": "Üfürümsüz", "val.ageCleared": "Temize ç■kan",
    "val.robustLead": "Çapraz do■rulaman■n her kat■, görülmemi■ kay■tlar■n■n bozulmu■ kopyalar■yla yeniden test edildi.",
    "val.robustLeadCompare": "Çapraz do■rulaman■n her kat■, görülmemi■ kay■tlar■n■n bozulmu■ kopyalar■yla yeniden test edildi. Gri: yaln■zca temiz sesle e■itilen model. Kal■n: rastgele bozulmu■ seslerle e■itilen, yay■ndaki model.",
    "val.robCond": "Test ko■ulu", "val.robScore": "Yar■■ma skoru", "val.robBeat": "Kalp at■■■ kontrolünü geçen",
    "val.robustNote": "Bozulmu■ seslerle e■itim, telefon benzeri mikrofonda yakalanan üfürüm oran■n■ {before} de■erinden {after} de■erine ç■kard■. E■itimde hiç görmedi■i bir bozulma türü olan tutma gürültüsünde ise biraz daha kötü sonuç verdi (yar■■ma skoru {rb} → {ra}); yani dayan■kl■l■k yeni gürültü türlerine kendili■inden ta■■nm■yor.",
    "val.extSet": "Veri seti", "val.extN": "Kay■t", "val.extAbn": "Anormal", "val.extSens": "■■aretlenen anormal",
    "val.extSpec": "Temize ç■kan normal", "val.extSubset": "Grup {k}", "val.extAll": "Tümü",
    "val.externalNote": "Genel AUC {auc} ve bu kay■tlar■n yaln■zca {beat} kadar■ kalp at■■■ kontrolünü geçti. Model, ba■ka cihazlarla yeti■kinlerden al■nan kay■tlara henüz iyi genellenmiyor; bu yüzden LittleHeart yeni bir yerde kullan■lmadan önce yerel olarak do■rulanmal■d■r. Gri say■lar: yaln■zca temiz sesle e■itilen model.",
    "val.safetyReal": "Gerçek kalp kay■tlar■ ({n})", "val.accepted": "{p} kabul edildi", "val.rejected": "{k}/{n} reddedildi",
    "val.limit1": "Çapraz do■rulamada yakla■■k {n} üfürümden 1'ini kaç■r■yor, ço■unlukla hafif olanlar■",
    "val.limit2": "Tek bir bölgede (Brezilya), ço■unlukla çocuklarla, tek bir steteskopla e■itildi",
    "val.limit3": "Ba■ka cihazlarla, ço■unlukla yeti■kinlerden al■nan farkl■ bir veri setinde AUC {auc} de■erine dü■tü",
    "val.limit4": "Aç■k verideki çapraz do■rulama, yar■■man■n gizli test seti de■ildir",
    "val.limit5": "Kalp at■■■ olmayan ritmik sesler (örne■in metronom) güvenlik kontrolünü geçebilir",
    "val.limit6": "Yukar■daki skorlar, gerçek kay■tlar■n yakla■■k %4'ü için yeniden kay■t isteyen kalp at■■■ kontrolünü içermez",
    "cond.clean": "Orijinal kay■tlar", "cond.noise20": "Arka plan gürültüsü, 20 dB sinyal-gürültü oran■",
    "cond.noise10": "Arka plan gürültüsü, 10 dB sinyal-gürültü oran■", "cond.noise5": "Arka plan gürültüsü, 5 dB sinyal-gürültü oran■",
    "cond.phone": "Telefon benzeri mikrofon (150 Hz alt■ yok)",
    "cond.rumble": "Tutma gürültüsü, 10 dB (e■itimde hiç görülmedi)",
    "neg.white noise": "Beyaz gürültü (sentetik)", "neg.pink-ish noise": "Pembe gürültü (sentetik)",
    "neg.steady tone 440 Hz": "Sabit ton 440 Hz (sentetik)", "neg.mains hum 50 Hz": "■ebeke u■ultusu 50 Hz (sentetik)",
    "neg.speech-like babble": "Konu■ma benzeri ses (sentetik)", "neg.silence with hiss": "H■■■rt■l■ sessizlik (sentetik)",
    "load.failTitle": "Yapay zeka modeli yüklenemedi",
    "load.failText": "Bu sayfay■ dosyaya çift t■klayarak de■il, bir web sunucusu üzerinden aç■n (README'ye bak■n).",
    "offline.ready": "✓ ■nternetsiz çal■■■r",
  };

  const PT = {
    "tab.screen": "Triar uma criança", "tab.train": "Treine o ouvido", "tab.log": "Registro de triagem", "tab.validation": "Validação", "tab.learn": "Sobre sopros",
    "screen.title": "Ouça os problemas cardíacos que se escondem nas crianças.",
    "screen.lead": "O LittleHeart escuta gravações de estetoscópio dos quatro focos valvares e sinaliza as crianças cujo som cardíaco tem um sopro, para que o profissional de saúde saiba quem precisa de ecocardiograma. A IA roda neste aparelho; nenhuma gravação é enviada.",
    "screen.step1": "1. Adicionar gravações", "source.record": "Gravar", "source.demo": "Casos reais", "source.upload": "Enviar",
    "record.lead": "Use um estetoscópio digital ou um estetoscópio com adaptador para celular. O LittleHeart guia você pelos quatro focos de ausculta e verifica a qualidade do som ao vivo.",
    "record.start": "● Iniciar triagem guiada", "record.oneSite": "Gravar apenas um foco",
    "site.AV.long": "Aórtico (AV)", "site.PV.long": "Pulmonar (PV)", "site.TV.long": "Tricúspide (TV)", "site.MV.long": "Mitral (MV)",
    "record.one": "● Gravar 15 s", "record.redo": "Recomeçar este foco", "record.skip": "Pular foco", "record.stop": "Parar",
    "demo.lead": "Crianças do conjunto de dados CirCor que a IA nunca viu no treinamento. O achado do cardiologista é mostrado após a análise.",
    "upload.lead": "Escolha gravações WAV/MP3. Nomeie os arquivos como <code>nome_AV.wav</code>, <code>nome_MV.wav</code> para definir o foco automaticamente.",
    "upload.drop": "<b>Escolha arquivos</b> ou solte-os aqui",
    "screen.step2": "2. Focos de ausculta",
    "site.AV.where": "aórtico · 2º espaço, à direita do esterno", "site.PV.where": "pulmonar · 2º espaço, à esquerda do esterno",
    "site.TV.where": "tricúspide · borda esternal inferior esquerda", "site.MV.where": "mitral · ápice, 5º espaço",
    "screen.step3": "3. Resultado", "log.childId": "ID ou iniciais da criança", "log.save": "Salvar no registro",
    "result.print": "Imprimir nota de triagem", "result.new": "Nova criança",
    "screen.disclaimer": "O LittleHeart é um protótipo de pesquisa para apoio à triagem, não é um dispositivo médico nem um diagnóstico. Foi treinado com crianças do Nordeste do Brasil gravadas com um único modelo de estetoscópio e, num teste com outros aparelhos e pacientes principalmente adultos, funcionou bem pior. Uma criança que parece doente deve ver um profissional de saúde, qualquer que seja o resultado.",
    "train.title": "Você consegue ouvir um sopro?",
    "train.lead": "Pratique com sons cardíacos de 18 crianças reais. Ouça, decida e depois veja a resposta do cardiologista e o que a IA disse. As respostas da IA vêm de um modelo que nunca ouviu aquela criança, então a disputa é justa.",
    "train.score": "Placar", "train.howTitle": "Como escutar",
    "train.tip1": "Primeiro encontre o ritmo: \"tum\" (B1), depois \"tá\" (B2) e então uma pausa mais longa.",
    "train.tip2": "Agora escute só o pequeno intervalo entre o tum e o tá. Está silencioso ou há um sopro?",
    "train.tip3": "No espectrograma, um sopro aparece como uma névoa de cor entre as batidas verticais brilhantes.",
```

```
    "train.tip4": "Use fones de ouvido. Alto-falantes de notebook não reproduzem os sons cardíacos mais graves.",
    "log.title": "Registro de triagem",
    "log.lead": "Acompanhe um dia de triagem numa escola ou comunidade: quem foi triado, quem precisa de ecocardiograma e quem deve ser gravado de novo. O registro fica salvo apenas neste aparelho.",
    "log.all": "Todos", "log.referred": "Encaminhar", "log.rerecord": "Gravar de novo", "log.clear": "Sem sopro",
    "log.export": "Exportar CSV", "log.wipe": "Apagar tudo",
    "val.title": "Quão bem funciona?", "val.compare": "Comparado com outras abordagens",
    "val.compareLead": "Pontuação oficial de sopro do PhysioNet 2022 (acurácia ponderada). Maior é melhor.",
    "val.confusion": "Cada criança na validação cruzada", "val.confusionLead": "Linhas: achado do cardiologista. Colunas: resposta do LittleHeart.",
    "val.missTitle": "Quais sopros ele deixa passar?", "val.missLead": "Crianças com sopro na validação cruzada, por grau de intensidade (I = mais fraco, III = forte).",
    "val.legendRefer": "encaminhadas", "val.legendRerecord": "pedido para regravar", "val.legendMissed": "não detectadas",
    "val.ageTitle": "Por faixa etária", "val.robustTitle": "Resiste ao som do mundo real?",
    "val.externalTitle": "Teste num conjunto de dados totalmente diferente",
    "val.externalLead": "PhysioNet/CinC Challenge 2016: 3.240 gravações coletadas por seis grupos de pesquisa em diferentes países, principalmente de adultos, com estetoscópios diferentes. O LittleHeart nunca treinou com nenhuma delas. Os rótulos são \"coração normal\" ou \"anormal\", o que não é o mesmo que \"sopro\": por exemplo, a doença arterial coronariana é anormal, mas geralmente não tem sopro.",
    "val.howTitle": "Como a IA funciona",
    "val.how1": "<b>Espectrograma.</b> Cada gravação vira um espectrograma log-mel: uma imagem de quais frequências (25–1000 Hz) estão fortes a cada 16 ms.",
    "val.how2": "<b>Janelas de 4 segundos.</b> A imagem é cortada em janelas sobrepostas de 4 segundos, com cerca de 4–8 batimentos cada.",
    "val.how3": "<b>CNN.</b> Uma pequena rede neural convolucional (4 camadas, cerca de 61.000 pesos) pontua cada janela: sopro, sem sopro ou incerto.",
    "val.how4": "<b>Treinada para o mundo real.</b> Durante o treinamento, as gravações são danificadas aleatoriamente com ruído de fundo e microfones parecidos com os de celular, para a IA aprender a lidar com eles.",
    "val.how5": "<b>Por criança.</b> As pontuações das janelas são promediadas por foco. Se algum foco parecer ter sopro, a criança é encaminhada; se uma gravação for incerta, pede-se para gravar de novo.",
    "val.testedTitle": "Como testamos",
    "val.tested1": "<b>Divisão por criança, não por gravação.</b> Todas as gravações de uma criança ficam no treino ou no teste, nunca nos dois.",
    "val.tested2": "<b>Validação cruzada.</b> Cada criança é testada uma vez por um modelo que nunca a ouviu.",
    "val.tested3": "<b>Limiares escolhidos em dados separados.</b> Os pontos de corte vêm de uma divisão de validação dentro das dobras de treino.",
    "val.tested4": "<b>Modelo de referência.</b> Uma regressão logística sobre o espectro médio mostra quanto a CNN acrescenta.",
    "val.tested5": "<b>Casos de demonstração intocados.</b> Os 10 casos reais da primeira aba foram sorteados e separados antes de qualquer treino.",
    "val.tested6": "<b>Teste externo.</b> Um segundo conjunto de dados, sem relação, verifica até onde os resultados se mantêm.",
    "val.safetyTitle": "Verificação de segurança: é mesmo um batimento?",
    "val.safetyLead": "A CNN só ouviu sons cardíacos, então poderia dar uma resposta confiante mas sem sentido para fala ou ruído. Antes de confiar nela, o LittleHeart verifica se a intensidade dos sons graves sobe e desce num ritmo regular entre 40 e 220 batimentos por minuto. Gravações que falham nunca são chamadas de \"sopro\" ou \"sem sopro\"; pede-se ao profissional para gravar de novo.",
    "val.statusTitle": "Situação honesta", "val.done": "Funcionando agora",
    "val.done1": "Modelo de sopro com validação cruzada em 942 crianças, treinado para lidar com ruído e microfones parecidos com os de celular",
    "val.done2": "Gravação guiada com verificação ao vivo de batimento e volume",
    "val.done3": "Explicação por foco: espectrograma com linha do tempo do sopro",
    "val.done4": "Modo de treino auditivo, registro de triagem com exportação CSV, 4 idiomas",
    "val.done5": "Funciona offline após a primeira visita e pode ser instalado como um app",
    "val.done6": "Código Python e JavaScript verificado para dar números idênticos",
    "val.notyet": "Ainda não feito",
    "val.notyet1": "Testes com adaptadores reais de estetoscópio de baixo custo",
    "val.notyet2": "Validação com crianças de outra região, idealmente um estudo prospectivo",
    "val.notyet3": "Distinguir sopros inocentes de patológicos (requer dados rotulados por ecocardiograma)",
    "val.notyet4": "Revisão por cardiologistas pediátricos e aprovação regulatória",
    "val.limits": "Limites conhecidos",
    "learn.title": "Por que escutar o coração das crianças?",
    "learn.lead": "A cardiopatia congênita é o defeito de nascença mais comum, e a doença cardíaca reumática ainda danifica o coração de milhões de crianças e jovens. Ambas costumam ser notadas primeiro como um sopro, mas ecocardiografia e cardiologistas são escassos justamente onde essas doenças são mais comuns.",
    "learn.q1": "O que é um sopro?", "learn.a1": "Um som de \"chiado\" entre os sons normais \"tum-tá\" do coração, causado por fluxo sanguíneo turbulento. Muitos sopros em crianças são inocentes, mas alguns vêm de buracos no coração ou de válvulas danificadas.",
    "learn.q2": "Para quem é o LittleHeart?", "learn.a2": "Agentes comunitários de saúde, programas de triagem escolar e unidades de atenção básica que têm estetoscópio mas não têm cardiologista, e precisam decidir quais crianças encaminhar para ecocardiograma.",
    "learn.q3": "Por que IA?", "learn.a3": "Ouvir um sopro exige anos de prática, e até profissionais experientes discordam. Uma IA que ouviu milhares de gravações pode dar a cada profissional uma segunda opinião consistente.",
    "learn.q4": "Os quatro focos", "learn.a4": "Cada válvula é mais bem ouvida num ponto: aórtico e pulmonar no 2º espaço intercostal de cada lado do esterno, tricúspide na borda esternal inferior esquerda e mitral no ápice do coração.",
    "learn.q5": "Dicas de gravação", "learn.a5": "Use um ambiente silencioso, mantenha a criança calma e parada, pressione o estetoscópio com firmeza sobre a pele e grave pelo menos 10 segundos por foco. Choro e fala deixam as gravações incertas.",
    "learn.q6": "E depois?", "learn.a6": "Um resultado \"encaminhar\" significa que a criança deve fazer um ecocardiograma, o ultrassom que mostra a estrutura do coração. Um resultado \"sem sopro\" não descarta doença cardíaca.",
    "learn.data": "Dados: CirCor DigiScope Phonocardiogram Dataset (Oliveira et al., IEEE JBHI 2022) e PhysioNet/CinC Challenge 2016 (Liu et al. 2016), PhysioNet. Gravações usadas sob suas licenças abertas.",
    "footer": "LittleHeart · UnivaBio 2026: IA para a Saúde Humana · Protótipo de pesquisa, não é um dispositivo médico",
    "site.AV": "Aórtico", "site.PV": "Pulmonar", "site.TV": "Tricúspide", "site.MV": "Mitral", "site.Phc": "Outro foco",
    "label.0": "Sem sopro", "label.1": "Sopro", "label.2": "Incerto",
    "age.Neonate": "Recém-nascido", "age.Infant": "Lactente", "age.Child": "Criança", "age.Adolescent":
```

```
"Adolescente", "age.nan": "Adulto jovem",
    "sex.Female": "menina", "sex.Male": "menino",
    "timing.Early-systolic": "protossistólico", "timing.Mid-systolic": "mesossistólico",
    "timing.Late-systolic": "telessistólico", "timing.Holosystolic": "holossistólico",
    "demo.case": "Caso {n}", "demo.recs": "{n} gravações",
    "verdict.empty.title": "Nenhuma gravação ainda",
    "verdict.empty.text": "Inicie uma gravação guiada, escolha um caso real ou envie gravações.",
    "verdict.busy.title": "Analisando",
    "verdict.busy.text": "Transformando o som em espectrograma e rodando a IA neste aparelho.",
    "verdict.pending": "Ainda analisando {n} gravação(ões).",
    "verdict.refer.title": "Suspeita de sopro: encaminhar para ecocardiograma",
    "verdict.refer.text": "Mais forte no foco {site} ({p} de probabilidade de sopro). Providencie um ecocardiograma para avaliar a estrutura do coração.",
    "verdict.nobeat.title": "Nenhum batimento encontrado: verifique e grave de novo",
    "verdict.nobeat.text": "O LittleHeart não encontrou um batimento regular, então não vai arriscar um palpite. Pressione o estetoscópio com firmeza sobre a pele no ponto marcado e mantenha o ambiente silencioso.",
    "verdict.unclear.title": "Gravação incerta: grave de novo",
    "verdict.unclear.text": "A IA não conseguiu avaliar {sites} com segurança. Grave de novo num momento mais calmo; se continuar incerto, encaminhe a um profissional.",
    "verdict.unclear.nobeat": "Nenhum ritmo cardíaco foi encontrado em {sites}.",
    "verdict.clear.title": "Nenhum sopro detectado",
    "verdict.clear.text": "Maior probabilidade de sopro {p} (limiar de encaminhamento {th}). Isso não descarta doença cardíaca; siga o cuidado habitual.",
    "truth.label": "Achado do cardiologista:", "truth.murmur": "sopro ouvido em {sites}", "truth.grade": "grau {g}",
    "truth.none": "sem sopro", "truth.unknown": "o especialista não conseguiu decidir (gravação incerta)",
    "truth.agree": "O LittleHeart concordou ✓", "truth.disagree": "O LittleHeart discordou ✗",
    "card.busy": "analisando…", "card.errorBadge": "erro", "card.error": "Não foi possível ler este arquivo de áudio.",
    "card.nobeat": "Sem batimento", "card.specAria": "Espectrograma", "card.timelineAria": "Probabilidade de sopro ao longo do tempo",
    "card.play": "Tocar gravação", "card.beatOk": "batimento ✓", "card.beatNo": "sem ritmo cardíaco",
    "card.murmur": "sopro", "card.unclear": "incerto",
    "card.timelineCaption": "probabilidade de sopro por janela de 4 s · tracejado = limiar de encaminhamento",
    "print.meta": "Nota de triagem LittleHeart · {date} · {n} gravação(ões) · Protótipo de pesquisa, não é diagnóstico. A decisão de encaminhar cabe a um profissional.",
    "live.searching": "Procurando o batimento…", "live.quiet": "Muito baixo: pressione o estetoscópio sobre a pele",
    "live.beat": "Batimento encontrado ✓",
    "guide.step": "Foco {n} de {total}: {site}", "guide.single": "Foco único: {site}",
    "guide.place.AV": "Coloque o estetoscópio no 2º espaço intercostal, logo à direita do esterno.",
    "guide.place.PV": "Coloque o estetoscópio no 2º espaço intercostal, logo à esquerda do esterno.",
    "guide.place.TV": "Coloque o estetoscópio na borda inferior esquerda do esterno.",
    "guide.place.MV": "Coloque o estetoscópio no ápice: 5º espaço intercostal, abaixo do meio da clavícula esquerda.",
    "guide.ready": "Prepare-se… a gravação começa em {s} s", "guide.recording": "Gravando… faltam {s} s",
    "guide.takeName": "Gravação em {site}", "guide.done": "Todos os focos gravados. O resultado aparece à direita.",
    "guide.denied": "A permissão do microfone foi negada ou nenhum microfone foi encontrado.", "guide.stopped": "Gravação interrompida.",
    "quiz.truthNone": "Sem sopro", "quiz.truthMurmur": "Sopro",
    "quiz.pitch.Low": "tom grave", "quiz.pitch.Medium": "tom médio", "quiz.pitch.High": "tom agudo",
    "quiz.quality.Blowing": "aspirativo", "quiz.quality.Harsh": "rude", "quiz.quality.Musical": "musical",
    "quiz.none": "Não há gravações de prática disponíveis.",
    "quiz.progress": "Gravação {n} de {total}", "quiz.question": "Você ouve um sopro?",
    "quiz.hint": "Toque quantas vezes quiser. Fones de ouvido ajudam.",
    "quiz.yes": "Sopro", "quiz.no": "Sem sopro", "quiz.right": "Correto!", "quiz.wrong": "Não foi desta vez.",
    "quiz.cardiologist": "Cardiologista:", "quiz.ai": "IA:",
    "quiz.aiYes": "probabilidade de sopro {p}, então disse sopro", "quiz.aiNo": "probabilidade de sopro {p}, então disse sem sopro",
    "quiz.lookMurmur": "Procure a névoa de cor entre as batidas verticais brilhantes: aquilo é o sopro.",
    "quiz.lookClear": "Entre as batidas verticais brilhantes a imagem fica escura: nenhum som extra.",
    "quiz.next": "Próxima gravação", "quiz.finish": "Ver meu resultado",
    "quiz.you": "Você", "quiz.aiName": "IA do LittleHeart",
    "quiz.scoreNote": "As mesmas gravações para os dois. A IA nunca treinou com essas crianças.",
    "quiz.endTitle": "Prática concluída",
    "quiz.endText": "Você acertou {you} de {total}. A IA acertou {ai} de {total}.",
    "quiz.endNote": "Sopros fracos são difíceis até para médicos. Pratique de novo ou experimente os casos reais na primeira aba.",
    "quiz.again": "Praticar de novo",
    "log.unnamed": "Criança {n}", "log.saved": "Salvo ✓", "log.saveFailed": "Não foi possível salvar",
    "log.status0": "Sem sopro", "log.status1": "Encaminhar", "log.status2": "Gravar de novo",
    "log.statScreened": "Crianças triadas", "log.statRefer": "Encaminhar para ecocardiograma",
    "log.statRerecord": "Gravar de novo", "log.statClear": "Sem sopro",
    "log.noneFilter": "Nenhuma criança corresponde a este filtro.",
    "log.empty": "Nenhuma triagem salva ainda. Após um resultado, digite o ID da criança e toque em \"Salvar no registro\".",
    "log.colChild": "Criança", "log.colDate": "Data", "log.colResult": "Resultado", "log.colSites": "Focos",
    "log.demo": "demo", "log.delete": "Apagar", "log.confirmWipe": "Apagar todas as triagens salvas neste aparelho?",
    "val.intro": "O LittleHeart foi testado com validação cruzada em {folds} dobras em {children} crianças ({recordings} gravações no conjunto completo). Cada criança foi avaliada por um modelo que nunca a tinha ouvido.",
    "val.card1": "Pontuação do desafio", "val.card1Text": "Acurácia ponderada do PhysioNet 2022: sopros contam 5×, incertos 3×, normais 1×.",
    "val.card2": "Sopros detectados", "val.card2Text": "Crianças com sopro que o LittleHeart encaminhou.",
    "val.card3": "Corações normais liberados", "val.card3Text": "Crianças sem sopro que não foram encaminhadas.",
    "val.card4Text": "Quão bem a probabilidade de sopro ordena crianças com e sem sopro (1,0 = perfeito).",
    "val.barBaseline": "Referência simples (nossa)", "val.barClean": "CNN, treino com som limpo (nossa)",
    "val.barOurs": "CNN do LittleHeart (nossa)", "val.barTop": "Melhor equipe, PhysioNet 2022*",
    "val.compareNote": "*Não é uma comparação direta. A melhor pontuação do desafio (equipe HearHeart, entre 40 participações de sopro classificadas) foi medida num conjunto de teste oculto ao qual não temos acesso, e as
```

```
pontuações nesse conjunto costumam ser menores que na validação cruzada com os dados públicos. Mostra que estamos na faixa certa, não que igualamos o vencedor.",
438     "val.gradeNote": "Dos {missed} sopros totalmente não detectados, {faint} ({share}) eram do grau mais fraco, I/VI. Sopros fortes de grau III/VI foram encaminhados em {loud} das vezes.",
439     "val.ageCol": "Faixa etária", "val.ageMurmur": "Com sopro", "val.ageCaught": "Detectados",
440     "val.ageNone": "Sem sopro", "val.ageCleared": "Liberados",
441     "val.robustLead": "Cada dobra da validação cruzada foi testada de novo com cópias danificadas de suas gravações não vistas.",
442     "val.robustLeadCompare": "Cada dobra da validação cruzada foi testada de novo com cópias danificadas de suas gravações não vistas. Cinza: modelo treinado só com som limpo. Negrito: o modelo em uso, treinado com som danificado aleatoriamente.",
443     "val.robCond": "Condição de teste", "val.robScore": "Pontuação do desafio", "val.robBeat": "Passou na verificação de batimento",
444     "val.robustNote": "O treino com som danificado elevou a proporção de sopros detectados com microfone parecido com o de celular de {before} para {after}. Com ruído de manuseio, um tipo de dano que nunca viu no treino, ficou um pouco pior (pontuação do desafio {rb} → {ra}); a robustez não se transfere automaticamente para novos tipos de ruído.",
445     "val.extSet": "Conjunto", "val.extN": "Gravações", "val.extAbn": "Anormais", "val.extSens": "Anormais sinalizados",
446     "val.extSpec": "Normais liberados", "val.extSubset": "Grupo {k}", "val.extAll": "Todos",
447     "val.externalNote": "AUC geral {auc}, e só {beat} dessas gravações passaram na verificação de batimento. O modelo ainda não se transfere bem para gravações de adultos feitas com outros aparelhos, por isso o LittleHeart precisa ser validado localmente antes de ser usado em qualquer lugar novo. Números em cinza: modelo treinado só com som limpo.",
448     "val.safetyReal": "Gravações cardíacas reais ({n})", "val.accepted": "{p} aceitas", "val.rejected": "{k}/{n} rejeitados",
449     "val.limit1": "Deixa passar cerca de 1 em cada {n} sopros na validação cruzada, principalmente os fracos",
450     "val.limit2": "Treinado numa só região (Brasil), principalmente com crianças, com um só estetoscópio",
451     "val.limit3": "Num conjunto diferente, de gravações principalmente de adultos com outros aparelhos, a AUC caiu para {auc}",
452     "val.limit4": "A validação cruzada nos dados públicos não é o conjunto de teste oculto do desafio",
453     "val.limit5": "Sons rítmicos que não são batimentos (por exemplo, um metrônomo) podem passar na verificação de segurança",
454     "val.limit6": "As pontuações acima não incluem a verificação de batimento, que pede nova gravação para cerca de 4% das gravações reais",
455     "cond.clean": "Gravações originais", "cond.noise20": "Ruído de fundo, 20 dB de relação sinal-ruído",
456     "cond.noise10": "Ruído de fundo, 10 dB de relação sinal-ruído", "cond.noise5": "Ruído de fundo, 5 dB de relação sinal-ruído",
457     "cond.phone": "Microfone tipo celular (nada abaixo de 150 Hz)",
458     "cond.rumble": "Ruído de manuseio, 10 dB (nunca visto no treino)",
459     "neg.white noise": "Ruído branco (sintético)", "neg.pink-ish noise": "Ruído rosa (sintético)",
460     "neg.steady tone 440 Hz": "Tom constante 440 Hz (sintético)", "neg.mains hum 50 Hz": "Zumbido da rede 50 Hz (sintético)",
461     "neg.speech-like babble": "Som parecido com fala (sintético)", "neg.silence with hiss": "Silêncio com chiado (sintético)",
462     "load.failTitle": "Falha ao carregar o modelo de IA",
463     "load.failText": "Abra esta página por um servidor web (veja o README), não clicando duas vezes no arquivo.",
464     "offline.ready": "✓ Funciona offline",
465   };
466
467   const ES = {
468     "tab.screen": "Evaluar a un niño", "tab.train": "Entrena tu oído", "tab.log": "Registro de cribado", "tab.validation": "Validación", "tab.learn": "Sobre los soplos",
469     "screen.title": "Escuche los problemas cardíacos que se esconden en los niños.",
470     "screen.lead": "LittleHeart escucha grabaciones de estetoscopio de los cuatro focos valvulares y señala a los niños cuyo sonido cardíaco tiene un soplo, para que el trabajador de salud sepa quién necesita un ecocardiograma. La IA funciona en este dispositivo; no se sube ninguna grabación.",
471     "screen.step1": "1. Añadir grabaciones", "source.record": "Grabar", "source.demo": "Casos reales", "source.upload": "Subir",
472     "record.lead": "Use un estetoscopio digital o un estetoscopio con adaptador para teléfono. LittleHeart le guía por los cuatro focos de auscultación y comprueba la calidad del sonido en directo.",
473     "record.start": "● Iniciar cribado guiado", "record.oneSite": "Grabar solo un foco",
474     "site.AV.long": "Aórtico (AV)", "site.PV.long": "Pulmonar (PV)", "site.TV.long": "Tricuspídeo (TV)", "site.MV.long": "Mitral (MV)",
475     "record.one": "● Grabar 15 s", "record.redo": "Repetir este foco", "record.skip": "Saltar foco", "record.stop": "Detener",
476     "demo.lead": "Niños del conjunto de datos CirCor que la IA nunca vio durante el entrenamiento. El hallazgo del cardiólogo se muestra después del análisis.",
477     "upload.lead": "Elija grabaciones WAV/MP3. Nombre los archivos como <code>nombre_AV.wav</code>, <code>nombre_MV.wav</code> para asignar el foco automáticamente.",
478     "upload.drop": "<b>Elija archivos</b> o suéltelos aquí",
479     "screen.step2": "2. Focos de auscultación",
480     "site.AV.where": "aórtico · 2.º espacio, a la derecha del esternón", "site.PV.where": "pulmonar · 2.º espacio, a la izquierda del esternón",
481     "site.TV.where": "tricuspídeo · borde esternal inferior izquierdo", "site.MV.where": "mitral · ápex, 5.º espacio",
482     "screen.step3": "3. Resultado", "log.childId": "ID o iniciales del niño", "log.save": "Guardar en el registro",
483     "result.print": "Imprimir nota de cribado", "result.new": "Nuevo niño",
484     "screen.disclaimer": "LittleHeart es un prototipo de investigación para apoyar el cribado; no es un dispositivo médico ni un diagnóstico. Se entrenó con niños del noreste de Brasil grabados con un solo modelo de estetoscopio y, en una prueba con otros dispositivos y pacientes sobre todo adultos, funcionó mucho peor. Un niño que parece enfermo debe ver a un profesional sanitario sea cual sea el resultado.",
485     "train.title": "¿Puede oír un soplo?",
486     "train.lead": "Practique con los sonidos cardíacos de 18 niños reales. Escuche, decida y luego vea la respuesta del cardiólogo y lo que dijo la IA. Las respuestas de la IA vienen de un modelo que nunca escuchó a ese niño, así que la competencia es justa.",
487     "train.score": "Puntuación", "train.howTitle": "Cómo escuchar",
488     "train.tip1": "Primero encuentre el ritmo: \"lub\" (R1), luego \"dub\" (R2) y después una pausa más larga.",
489     "train.tip2": "Ahora escuche solo el breve espacio entre lub y dub. ¿Hay silencio o un soplido?",
490     "train.tip3": "En el espectrograma, un soplo aparece como una neblina de color entre los latidos verticales
```

```
brillantes.",
    "train.tip4": "Use auriculares. Los altavoces del portátil no reproducen los sonidos cardíacos más graves.",
    "log.title": "Registro de cribado",
    "log.lead": "Lleve el control de una jornada de cribado en una escuela o comunidad: quién fue evaluado, quién necesita un ecocardiograma y a quién hay que grabar de nuevo. El registro se guarda solo en este dispositivo.",
    "log.all": "Todos", "log.referred": "Derivar", "log.rerecord": "Grabar de nuevo", "log.clear": "Sin soplo",
    "log.export": "Exportar CSV", "log.wipe": "Borrar todo",
    "val.title": "¿Qué tan bien funciona?", "val.compare": "Comparado con otros enfoques",
    "val.compareLead": "Puntuación oficial de soplos de PhysioNet 2022 (exactitud ponderada). Más alto es mejor.",
    "val.confusion": "Cada niño en la validación cruzada", "val.confusionLead": "Filas: hallazgo del cardiólogo. Columnas: respuesta de LittleHeart.",
    "val.missTitle": "¿Qué soplos se le escapan?", "val.missLead": "Niños con soplo en la validación cruzada, por grado de intensidad (I = el más débil, III = fuerte).",
    "val.legendRefer": "derivados", "val.legendRerecord": "se pidió regrabar", "val.legendMissed": "no detectados",
    "val.ageTitle": "Por grupo de edad", "val.robustTitle": "¿Resiste el sonido del mundo real?",
    "val.externalTitle": "Prueba con un conjunto de datos totalmente distinto",
    "val.externalLead": "PhysioNet/CinC Challenge 2016: 3.240 grabaciones recogidas por seis grupos de investigación de distintos países, sobre todo de adultos, con estetoscopios diferentes. LittleHeart nunca se entrenó con ninguna. Sus etiquetas son \"corazón normal\" o \"anormal\", que no es lo mismo que \"soplo\": por ejemplo, la enfermedad coronaria es anormal pero suele no tener soplo.",
    "val.howTitle": "Cómo funciona la IA",
    "val.how1": "<b>Espectrograma.</b> Cada grabación se convierte en un espectrograma log-mel: una imagen de qué frecuencias (25–1000 Hz) suenan fuerte cada 16 ms.",
    "val.how2": "<b>Ventanas de 4 segundos.</b> La imagen se corta en ventanas superpuestas de 4 segundos, de unos 4–8 latidos cada una.",
    "val.how3": "<b>CNN.</b> Una pequeña red neuronal convolucional (4 capas, unos 61.000 pesos) puntúa cada ventana: soplo, sin soplo o dudoso.",
    "val.how4": "<b>Entrenada para el mundo real.</b> Durante el entrenamiento, las grabaciones se dañan al azar con ruido de fondo y micrófonos parecidos a los de un teléfono, para que la IA aprenda a manejarlos.",
    "val.how5": "<b>Por niño.</b> Las puntuaciones de las ventanas se promedian por foco. Si algún foco parece tener soplo, se deriva al niño; si una grabación es dudosa, se pide grabar de nuevo.",
    "val.testedTitle": "Cómo lo probamos",
    "val.tested1": "<b>División por niño, no por grabación.</b> Todas las grabaciones de un niño están en entrenamiento o en prueba, nunca en ambos.",
    "val.tested2": "<b>Validación cruzada.</b> Cada niño se evalúa una vez con un modelo que nunca lo escuchó.",
    "val.tested3": "<b>Umbrales elegidos con datos aparte.</b> Los puntos de corte salen de una división de validación dentro de los pliegues de entrenamiento.",
    "val.tested4": "<b>Modelo de referencia.</b> Una regresión logística sobre el espectro medio muestra cuánto aporta la CNN.",
    "val.tested5": "<b>Casos de demostración intactos.</b> Los 10 casos reales de la primera pestaña se eligieron al azar y se apartaron antes de cualquier entrenamiento.",
    "val.tested6": "<b>Prueba externa.</b> Un segundo conjunto de datos, sin relación, comprueba hasta dónde se mantienen los resultados.",
    "val.safetyTitle": "Control de seguridad: ¿es de verdad un latido?",
    "val.safetyLead": "La CNN solo ha oído sonidos cardíacos, así que podría dar una respuesta segura pero sin sentido ante voz o ruido. Antes de confiar en ella, LittleHeart comprueba que la intensidad de los graves sube y baja con un ritmo regular de 40 a 220 latidos por minuto. Las grabaciones que no pasan nunca se califican como \"soplo\" o \"sin soplo\"; se pide al trabajador de salud que grabe de nuevo.",
    "val.statusTitle": "Estado honesto", "val.done": "Funciona ahora",
    "val.done1": "Modelo de soplos con validación cruzada en 942 niños, entrenado para manejar ruido y micrófonos parecidos a los de un teléfono",
    "val.done2": "Grabación guiada con control en directo del latido y del volumen",
    "val.done3": "Explicación por foco: espectrograma con línea de tiempo del soplo",
    "val.done4": "Modo de entrenamiento auditivo, registro de cribado con exportación CSV, 4 idiomas",
    "val.done5": "Funciona sin conexión tras la primera visita y se puede instalar como una app",
    "val.done6": "Código Python y JavaScript verificado para dar números idénticos",
    "val.notyet": "Aún no hecho",
    "val.notyet1": "Pruebas con adaptadores reales de estetoscopio de bajo coste",
    "val.notyet2": "Validación con niños de otra región, idealmente un estudio prospectivo",
    "val.notyet3": "Distinguir soplos inocentes de patológicos (requiere datos etiquetados con ecocardiograma)",
    "val.notyet4": "Revisión por cardiólogos pediátricos y aprobación regulatoria",
    "val.limits": "Límites conocidos",
    "learn.title": "¿Por qué escuchar el corazón de los niños?",
    "learn.lead": "La cardiopatía congénita es el defecto de nacimiento más común, y la cardiopatía reumática todavía daña el corazón de millones de niños y jóvenes. Ambas suelen notarse primero como un soplo, pero la ecocardiografía y los cardiólogos escasean justo donde estas enfermedades son más frecuentes.",
    "learn.q1": "¿Qué es un soplo?", "learn.a1": "Un sonido silbante entre los ruidos normales \"lub-dub\" del corazón, causado por un flujo de sangre turbulento. Muchos soplos en niños son inocentes, pero algunos se deben a agujeros en el corazón o a válvulas dañadas.",
    "learn.q2": "¿Para quién es LittleHeart?", "learn.a2": "Agentes comunitarios de salud, programas de cribado escolar y centros de atención primaria que tienen estetoscopio pero no cardiólogo, y deben decidir qué niños enviar a ecocardiograma.",
    "learn.q3": "¿Por qué IA?", "learn.a3": "Oír un soplo requiere años de práctica, e incluso los oyentes con experiencia discrepan. Una IA que ha escuchado miles de grabaciones puede dar a cada trabajador de salud una segunda opinión coherente.",
    "learn.q4": "Los cuatro focos", "learn.a4": "Cada válvula se oye mejor en un punto: aórtico y pulmonar en el 2.º espacio intercostal a cada lado del esternón, tricuspídeo en su borde inferior izquierdo y mitral en el ápex del corazón.",
    "learn.q5": "Consejos para grabar", "learn.a5": "Use una sala tranquila, mantenga al niño calmado y quieto, presione el estetoscopio con firmeza sobre la piel y grabe al menos 10 segundos por foco. El llanto y el habla vuelven dudosas las grabaciones.",
    "learn.q6": "¿Y después?", "learn.a6": "Un resultado de \"derivar\" significa que el niño debe hacerse un ecocardiograma, la ecografía que muestra la estructura del corazón. Un resultado \"sin soplo\" no descarta una enfermedad cardíaca.",
    "learn.data": "Datos: CirCor DigiScope Phonocardiogram Dataset (Oliveira et al., IEEE JBHI 2022) y PhysioNet/CinC Challenge 2016 (Liu et al. 2016), PhysioNet. Grabaciones usadas bajo sus licencias abiertas.",
    "footer": "LittleHeart · UnivaBio 2026: IA para la Salud Humana · Prototipo de investigación, no es un dispositivo médico",
    "site.AV": "Aórtico", "site.PV": "Pulmonar", "site.TV": "Tricuspídeo", "site.MV": "Mitral", "site.Phc": "Otro foco",
```

```
    "label.0": "Sin soplo", "label.1": "Soplo", "label.2": "Dudoso",
    "age.Neonate": "Recién nacido", "age.Infant": "Lactante", "age.Child": "Niño", "age.Adolescent": "Adolescente", "age.nan": "Adulto joven",
    "sex.Female": "niña", "sex.Male": "niño",
    "timing.Early-systolic": "protosistólico", "timing.Mid-systolic": "mesosistólico",
    "timing.Late-systolic": "telesistólico", "timing.Holosystolic": "holosistólico",
    "demo.case": "Caso {n}", "demo.recs": "{n} grabaciones",
    "verdict.empty.title": "Aún no hay grabaciones",
    "verdict.empty.text": "Inicie una grabación guiada, elija un caso real o suba grabaciones.",
    "verdict.busy.title": "Analizando",
    "verdict.busy.text": "Convirtiendo el sonido en espectrograma y ejecutando la IA en este dispositivo.",
    "verdict.pending": "Aún analizando {n} grabación(es).",
    "verdict.refer.title": "Sospecha de soplo: derivar a ecocardiograma",
    "verdict.refer.text": "Más fuerte en el foco {site} ({p} de probabilidad de soplo). Organice un ecocardiograma para revisar la estructura del corazón.",
    "verdict.nobeat.title": "No se encontró latido: compruebe y grabe de nuevo",
    "verdict.nobeat.text": "LittleHeart no encontró un latido regular, así que no va a adivinar. Presione el estetoscopio con firmeza sobre la piel en el punto marcado y mantenga la sala en silencio.",
    "verdict.unclear.title": "Grabación dudosa: grabe de nuevo",
    "verdict.unclear.text": "La IA no pudo evaluar {sites} con fiabilidad. Grabe de nuevo en un momento más tranquilo; si sigue siendo dudoso, derive a un profesional.",
    "verdict.unclear.nobeat": "No se encontró ritmo cardíaco en {sites}.",
    "verdict.clear.title": "No se detectó soplo",
    "verdict.clear.text": "Mayor probabilidad de soplo {p} (umbral de derivación {th}). Esto no descarta una enfermedad cardíaca; siga la atención habitual.",
    "truth.label": "Hallazgo del cardiólogo:", "truth.murmur": "soplo oído en {sites}", "truth.grade": "grado {g}",
    "truth.none": "sin soplo", "truth.unknown": "el especialista no pudo decidir (grabación dudosa)",
    "truth.agree": "LittleHeart coincidió ✓", "truth.disagree": "LittleHeart no coincidió ✗",
    "card.busy": "analizando…", "card.errorBadge": "error", "card.error": "No se pudo leer este archivo de audio.",
    "card.nobeat": "Sin latido", "card.specAria": "Espectrograma", "card.timelineAria": "Probabilidad de soplo a lo largo del tiempo",
    "card.play": "Reproducir grabación", "card.beatOk": "latido ✓", "card.beatNo": "sin ritmo cardíaco",
    "card.murmur": "soplo", "card.unclear": "dudoso",
    "card.timelineCaption": "probabilidad de soplo por ventana de 4 s · discontinua = umbral de derivación",
    "print.meta": "Nota de cribado LittleHeart · {date} · {n} grabación(es) · Prototipo de investigación, no es un diagnóstico. La decisión de derivar corresponde a un profesional.",
    "live.searching": "Buscando el latido…", "live.quiet": "Demasiado bajo: presione el estetoscopio sobre la piel",
    "live.beat": "Latido encontrado ✓",
    "guide.step": "Foco {n} de {total}: {site}", "guide.single": "Un solo foco: {site}",
    "guide.place.AV": "Coloque el estetoscopio en el 2.º espacio intercostal, justo a la derecha del esternón.",
    "guide.place.PV": "Coloque el estetoscopio en el 2.º espacio intercostal, justo a la izquierda del esternón.",
    "guide.place.TV": "Coloque el estetoscopio en el borde inferior izquierdo del esternón.",
    "guide.place.MV": "Coloque el estetoscopio en el ápex: 5.º espacio intercostal, debajo de la mitad de la clavícula izquierda.",
    "guide.ready": "Prepárese… la grabación empieza en {s} s", "guide.recording": "Grabando… quedan {s} s",
    "guide.takeName": "Grabación en {site}", "guide.done": "Todos los focos grabados. El resultado aparece a la derecha.",
    "guide.denied": "Se denegó el permiso del micrófono o no se encontró ningún micrófono.", "guide.stopped": "Grabación detenida.",
    "quiz.truthNone": "Sin soplo", "quiz.truthMurmur": "Soplo",
    "quiz.pitch.Low": "tono grave", "quiz.pitch.Medium": "tono medio", "quiz.pitch.High": "tono agudo",
    "quiz.quality.Blowing": "aspirativo", "quiz.quality.Harsh": "rudo", "quiz.quality.Musical": "musical",
    "quiz.none": "No hay grabaciones de práctica disponibles.",
    "quiz.progress": "Grabación {n} de {total}", "quiz.question": "¿Oye un soplo?",
    "quiz.hint": "Reprodúzcala las veces que quiera. Los auriculares ayudan.",
    "quiz.yes": "Soplo", "quiz.no": "Sin soplo", "quiz.right": "¡Correcto!", "quiz.wrong": "No exactamente.",
    "quiz.cardiologist": "Cardiólogo:", "quiz.ai": "IA:",
    "quiz.aiYes": "probabilidad de soplo {p}, así que dijo soplo", "quiz.aiNo": "probabilidad de soplo {p}, así que dijo sin soplo",
    "quiz.lookMurmur": "Busque la neblina de color entre los latidos verticales brillantes: eso es el soplo.",
    "quiz.lookClear": "Entre los latidos verticales brillantes la imagen queda oscura: no hay sonido extra.",
    "quiz.next": "Siguiente grabación", "quiz.finish": "Ver mi resultado",
    "quiz.you": "Usted", "quiz.aiName": "IA de LittleHeart",
    "quiz.scoreNote": "Las mismas grabaciones para ambos. La IA nunca se entrenó con estos niños.",
    "quiz.endTitle": "Práctica completada",
    "quiz.endText": "Acertó {you} de {total}. La IA acertó {ai} de {total}.",
    "quiz.endNote": "Los soplos débiles son difíciles incluso para los médicos. Practique de nuevo o pruebe los casos reales de la primera pestaña.",
    "quiz.again": "Practicar de nuevo",
    "log.unnamed": "Niño {n}", "log.saved": "Guardado ✓", "log.saveFailed": "No se pudo guardar",
    "log.status0": "Sin soplo", "log.status1": "Derivar", "log.status2": "Grabar de nuevo",
    "log.statScreened": "Niños evaluados", "log.statRefer": "Derivar a ecocardiograma",
    "log.statRerecord": "Grabar de nuevo", "log.statClear": "Sin soplo",
    "log.noneFilter": "Ningún niño coincide con este filtro.",
    "log.empty": "Aún no hay cribados guardados. Tras un resultado, escriba el ID del niño y pulse \"Guardar en el registro\".",
    "log.colChild": "Niño", "log.colDate": "Fecha", "log.colResult": "Resultado", "log.colSites": "Focos",
    "log.demo": "demo", "log.delete": "Borrar", "log.confirmWipe": "¿Borrar todos los cribados guardados en este dispositivo?",
    "val.intro": "LittleHeart se probó con validación cruzada de {folds} pliegues en {children} niños ({recordings} grabaciones en el conjunto completo). Cada niño fue evaluado por un modelo que nunca lo había oído.",
    "val.card1": "Puntuación del desafío", "val.card1Text": "Exactitud ponderada de PhysioNet 2022: los soplos cuentan 5×, los dudosos 3×, los normales 1×.",
    "val.card2": "Soplos detectados", "val.card2Text": "Niños con soplo que LittleHeart derivó.",
    "val.card3": "Corazones normales descartados", "val.card3Text": "Niños sin soplo que no fueron derivados.",
    "val.card4Text": "Qué tan bien la probabilidad de soplo ordena a niños con y sin soplo (1,0 = perfecto).",
    "val.barBaseline": "Referencia simple (nuestra)", "val.barClean": "CNN, entrenamiento limpio (nuestra)",
    "val.barOurs": "CNN de LittleHeart (nuestra)", "val.barTop": "Mejor equipo, PhysioNet 2022*",
```

```
    "val.compareNote": "*No es una comparación directa. La mejor puntuación del desafío (equipo HearHeart, entre 40 participaciones de soplos clasificadas) se midió en un conjunto de prueba oculto al que no tenemos acceso, y las puntuaciones en ese conjunto suelen ser menores que la validación cruzada con los datos públicos. Muestra que estamos en el rango correcto, no que igualemos al ganador.",
    "val.gradeNote": "De los {missed} soplos que se escaparon por completo, {faint} ({share}) eran del grado más débil, I/VI. Los soplos fuertes de grado III/VI se derivaron el {loud} de las veces.",
    "val.ageCol": "Grupo de edad", "val.ageMurmur": "Con soplo", "val.ageCaught": "Detectados",
    "val.ageNone": "Sin soplo", "val.ageCleared": "Descartados",
    "val.robustLead": "Cada pliegue de la validación cruzada se volvió a probar con copias dañadas de sus grabaciones no vistas.",
    "val.robustLeadCompare": "Cada pliegue de la validación cruzada se volvió a probar con copias dañadas de sus grabaciones no vistas. Gris: modelo entrenado solo con sonido limpio. Negrita: el modelo en uso, entrenado con sonido dañado al azar.",
    "val.robCond": "Condición de prueba", "val.robScore": "Puntuación del desafío", "val.robBeat": "Pasó el control de latido",
    "val.robustNote": "Entrenar con sonido dañado elevó la proporción de soplos detectados con un micrófono tipo teléfono de {before} a {after}. Con ruido de manipulación, un tipo de daño que nunca vio en el entrenamiento, funcionó algo peor (puntuación del desafío {rb} → {ra}); la robustez no se traslada automáticamente a nuevos tipos de ruido.",
    "val.extSet": "Conjunto", "val.extN": "Grabaciones", "val.extAbn": "Anormales", "val.extSens": "Anormales señalados",
    "val.extSpec": "Normales descartados", "val.extSubset": "Grupo {k}", "val.extAll": "Todos",
    "val.externalNote": "AUC global {auc}, y solo {beat} de estas grabaciones pasaron el control de latido. El modelo todavía no se transfiere bien a grabaciones de adultos hechas con otros dispositivos, por eso LittleHeart debe validarse localmente antes de usarse en cualquier lugar nuevo. Números en gris: modelo entrenado solo con sonido limpio.",
    "val.safetyReal": "Grabaciones cardíacas reales ({n})", "val.accepted": "{p} aceptadas", "val.rejected": "{k}/{n} rechazados",
    "val.limit1": "Se le escapa aproximadamente 1 de cada {n} soplos en la validación cruzada, sobre todo los débiles",
    "val.limit2": "Entrenado en una sola región (Brasil), sobre todo con niños, con un solo estetoscopio",
    "val.limit3": "En un conjunto distinto, de grabaciones sobre todo de adultos con otros dispositivos, el AUC bajó a {auc}",
    "val.limit4": "La validación cruzada con datos públicos no es el conjunto de prueba oculto del desafío",
    "val.limit5": "Sonidos rítmicos que no son latidos (por ejemplo, un metrónomo) pueden pasar el control de seguridad",
    "val.limit6": "Las puntuaciones anteriores no incluyen el control de latido, que pide grabar de nuevo alrededor del 4% de las grabaciones reales",
    "cond.clean": "Grabaciones originales", "cond.noise20": "Ruido de fondo, 20 dB de relación señal-ruido",
    "cond.noise10": "Ruido de fondo, 10 dB de relación señal-ruido", "cond.noise5": "Ruido de fondo, 5 dB de relación señal-ruido",
    "cond.phone": "Micrófono tipo teléfono (nada por debajo de 150 Hz)",
    "cond.rumble": "Ruido de manipulación, 10 dB (nunca visto en el entrenamiento)",
    "neg.white noise": "Ruido blanco (sintético)", "neg.pink-ish noise": "Ruido rosa (sintético)",
    "neg.steady tone 440 Hz": "Tono constante 440 Hz (sintético)", "neg.mains hum 50 Hz": "Zumbido de red 50 Hz (sintético)",
    "neg.speech-like babble": "Sonido parecido al habla (sintético)", "neg.silence with hiss": "Silencio con siseo (sintético)",
    "load.failTitle": "No se pudo cargar el modelo de IA",
    "load.failText": "Abra esta página a través de un servidor web (vea el README), no haciendo doble clic en el archivo.",
    "offline.ready": "✓ Funciona sin conexión",
  };

  const DICTS = { tr: TR, pt: PT, es: ES };
  const LOCALES = { en: "en-GB", tr: "tr-TR", pt: "pt-BR", es: "es-ES" };
  let lang = "en";
  try {
    const saved = localStorage.getItem("littleheart.lang");
    const browser = (navigator.language || "en").slice(0, 2);
    lang = saved || (DICTS[browser] ? browser : "en");
  } catch { /* storage unavailable: keep English */ }

  function t(key, vars) {
    let s = (DICTS[lang] && DICTS[lang][key]) ?? EN[key] ?? key;
    if (vars) for (const [k, v] of Object.entries(vars)) s = s.split("{" + k + "}").join(v);
    return s;
  }

  // Page text: English is whatever index.html contains, remembered before the first change.
  function apply() {
    document.documentElement.lang = lang;
    const pick = (el, attr, prop) => {
      const key = el.getAttribute(attr);
      if (el.dataset["en_" + prop] === undefined) el.dataset["en_" + prop] = prop === "placeholder" ? el.placeholder : el[prop];
      const text = lang === "en" ? (EN[key] ?? el.dataset["en_" + prop]) : (DICTS[lang][key] ?? EN[key] ?? el.dataset["en_" + prop]);
      if (prop === "placeholder") el.placeholder = text; else el[prop] = text;
    };
    document.querySelectorAll("[data-i18n]").forEach(el => pick(el, "data-i18n", "textContent"));
    document.querySelectorAll("[data-i18n-html]").forEach(el => pick(el, "data-i18n-html", "innerHTML"));
    document.querySelectorAll("[data-i18n-placeholder]").forEach(el => pick(el, "data-i18n-placeholder", "placeholder"));
  }

  function setLang(next) {
    lang = DICTS[next] ? next : "en";
    try { localStorage.setItem("littleheart.lang", lang); } catch { /* not saved; still switches */ }
    apply();
    document.dispatchEvent(new Event("langchange"));
```

```
  }

  return {
    t, apply, setLang,
    get lang() { return lang; },
    get locale() { return LOCALES[lang]; },
    keys: { EN, ...DICTS },
  };
})();
```

web/js/features.js

JavaScript port of features.py and the heartbeat check, so the browser computes identical numbers.

```
// Audio loading and log-mel spectrogram, a line-by-line port of ml/features.py.
// tests/parity.test.mjs checks that both versions produce the same numbers.

(function (root) {
  "use strict";

  // Reads a 16-bit PCM WAV file (the CirCor format). Returns null for anything else,
  // so the caller can fall back to the browser's decoder.
  function parseWav(buffer) {
    const view = new DataView(buffer);
    const tag = (o) => String.fromCharCode(view.getUint8(o), view.getUint8(o + 1), view.getUint8(o + 2), view.
getUint8(o + 3));
    if (buffer.byteLength < 44 || tag(0) !== "RIFF" || tag(8) !== "WAVE") return null;
    let offset = 12, fmt = null;
    while (offset + 8 <= buffer.byteLength) {
      const id = tag(offset), size = view.getUint32(offset + 4, true);
      if (id === "fmt ") {
        fmt = {
          format: view.getUint16(offset + 8, true),
          channels: view.getUint16(offset + 10, true),
          sampleRate: view.getUint32(offset + 12, true),
          bits: view.getUint16(offset + 22, true),
        };
      } else if (id === "data" && fmt) {
        if (fmt.format !== 1 || fmt.bits !== 16) return null;
        const n = Math.floor(Math.min(size, buffer.byteLength - offset - 8) / (2 * fmt.channels));
        const samples = new Float64Array(n);
        for (let i = 0; i < n; i++) {
          let sum = 0;
          for (let c = 0; c < fmt.channels; c++) {
            sum += view.getInt16(offset + 8 + 2 * (i * fmt.channels + c), true) / 32768;
          }
          samples[i] = sum / fmt.channels;
        }
        return { sampleRate: fmt.sampleRate, samples };
      }
      offset += 8 + size + (size % 2);
    }
    return null;
  }

  // Converts any audio the browser can decode (mic recording, mp3, 44.1 kHz wav...)
  // to mono at the model's sample rate.
  async function decodeToRate(buffer, sampleRate) {
    const wav = parseWav(buffer.slice(0));
    if (wav && wav.sampleRate === sampleRate) return wav.samples;
    const decoded = await new OfflineAudioContext(1, 1, 44100).decodeAudioData(buffer.slice(0));
    try {
      return await resample(decoded, sampleRate);
    } catch {
      // Some browsers refuse very low sample rates: render at 2x, then average sample pairs.
      const double = await resample(decoded, sampleRate * 2);
      const out = new Float64Array(Math.floor(double.length / 2));
      for (let i = 0; i < out.length; i++) out[i] = (double[2 * i] + double[2 * i + 1]) / 2;
      return out;
    }
  }

  async function resample(decoded, sampleRate) {
    const length = Math.ceil(decoded.duration * sampleRate);
    const offline = new OfflineAudioContext(1, length, sampleRate);
    const src = offline.createBufferSource();
    src.buffer = decoded;
    src.connect(offline.destination);
    src.start();
    const rendered = await offline.startRendering();
    return Float64Array.from(rendered.getChannelData(0));
  }

  function normalize(x) {
    let mean = 0;
    for (let i = 0; i < x.length; i++) mean += x[i];
    mean /= x.length;
    let variance = 0;
    for (let i = 0; i < x.length; i++) variance += (x[i] - mean) ** 2;
    const std = Math.sqrt(variance / x.length) + 1e-8;
    const out = new Float64Array(x.length);
    for (let i = 0; i < x.length; i++) out[i] = (x[i] - mean) / std;
    return out;
  }

  // In-place iterative radix-2 FFT on separate real/imaginary arrays.
  function fft(re, im) {
    const n = re.length;
    for (let i = 1, j = 0; i < n; i++) {
      let bit = n >> 1;
      for (; j & bit; bit >>= 1) j ^= bit;
```

```
      j ^= bit;
      if (i < j) {
        [re[i], re[j]] = [re[j], re[i]];
        [im[i], im[j]] = [im[j], im[i]];
      }
    }
    for (let len = 2; len <= n; len <<= 1) {
      const angle = (-2 * Math.PI) / len;
      for (let i = 0; i < n; i += len) {
        for (let k = 0; k < len / 2; k++) {
          const wr = Math.cos(angle * k), wi = Math.sin(angle * k);
          const ar = re[i + k + len / 2], ai = im[i + k + len / 2];
          const tr = ar * wr - ai * wi, ti = ar * wi + ai * wr;
          re[i + k + len / 2] = re[i + k] - tr;
          im[i + k + len / 2] = im[i + k] - ti;
          re[i + k] += tr;
          im[i + k] += ti;
        }
      }
    }
  }

  // Returns { data: Float32Array(nMels * nFrames) in row-major (mel, frame) order, nFrames }.
  function logMel(samples, cfg) {
    const { n_fft: nFft, hop, n_mels: nMels, eps, mel_filterbank: fb } = cfg;
    let x = normalize(samples);
    if (x.length < nFft) {
      const padded = new Float64Array(nFft);
      padded.set(x);
      x = padded;
    }
    const nFrames = 1 + Math.floor((x.length - nFft) / hop);
    const nBins = nFft / 2 + 1;
    const hann = new Float64Array(nFft);
    for (let i = 0; i < nFft; i++) hann[i] = 0.5 - 0.5 * Math.cos((2 * Math.PI * i) / nFft);

    const out = new Float32Array(nMels * nFrames);
    const re = new Float64Array(nFft), im = new Float64Array(nFft), power = new Float64Array(nBins);
    for (let f = 0; f < nFrames; f++) {
      for (let i = 0; i < nFft; i++) {
        re[i] = x[f * hop + i] * hann[i];
        im[i] = 0;
      }
      fft(re, im);
      for (let b = 0; b < nBins; b++) power[b] = re[b] * re[b] + im[b] * im[b];
      for (let m = 0; m < nMels; m++) {
        let sum = 0;
        const row = fb[m];
        for (let b = 0; b < nBins; b++) if (row[b] !== 0) sum += row[b] * power[b];
        out[m * nFrames + f] = Math.log10(sum + eps);
      }
    }
    return { data: out, nFrames };
  }

  function windowStarts(nFrames, cfg) {
    if (nFrames <= cfg.window_frames) return [0];
    const starts = [];
    for (let s = 0; s + cfg.window_frames <= nFrames; s += cfg.window_step) starts.push(s);
    return starts;
  }

  // One window, shape (nMels, windowFrames). Short recordings are repeated to fill it.
  function cutWindow(spec, start, cfg) {
    const { n_mels: nMels, window_frames: w } = cfg;
    const out = new Float32Array(nMels * w);
    for (let m = 0; m < nMels; m++) {
      for (let t = 0; t < w; t++) {
        out[m * w + t] = spec.data[m * spec.nFrames + ((start + t) % spec.nFrames)];
      }
    }
    return out;
  }

  // Port of beat_measures in ml/heartbeat_check.py: is there a heartbeat rhythm at all?
  function beatMeasures(spec, check) {
    const n = spec.nFrames;
    const env = new Float64Array(n);
    for (let f = 0; f < n; f++) {
      let sum = 0;
      for (let m = 0; m < check.low_bands; m++) sum += spec.data[m * n + f];
      env[f] = sum / check.low_bands;
    }
    let mean = 0;
    for (let f = 0; f < n; f++) mean += env[f];
    mean /= n;
    let denom = 0;
    for (let f = 0; f < n; f++) { env[f] -= mean; denom += env[f] * env[f]; }
    const depth = Math.sqrt(denom / Math.max(1, n));
    if (denom <= 0 || n <= check.max_lag + 1) return { rhythm: 0, depth };
    let rhythm = 0;
```

```
    for (let lag = check.min_lag; lag <= check.max_lag; lag++) {
      let r = 0;
      for (let f = 0; f + lag < n; f++) r += env[f] * env[f + lag];
      rhythm = Math.max(rhythm, r / denom);
    }
    return { rhythm, depth };
  }

  const api = { parseWav, decodeToRate, normalize, fft, logMel, windowStarts, cutWindow, beatMeasures };
  if (typeof module !== "undefined" && module.exports) module.exports = api;
  else root.LHFeatures = api;
})(typeof self !== "undefined" ? self : this);
```

web/js/model.js

Runs the ONNX model in the browser and applies the referral rule.

```
// Runs the exported CNN in the browser with ONNX Runtime Web and applies the
// same decision rule as ml/train.py (decide / patient_probs), plus the
// heartbeat safety check from ml/heartbeat_check.py.

(function (root) {
  "use strict";

  const ABSENT = 0, PRESENT = 1, UNKNOWN = 2;
  let session = null;
  let config = null;
  let beatCheck = null;
  let loading = null;

  function load() {
    loading = loading || (async () => {
      [config, beatCheck] = await Promise.all([
        fetch("model/config.json").then(r => r.json()),
        fetch("model/heartbeat_check.json").then(r => r.json()),
      ]);
      ort.env.wasm.wasmPaths = new URL("vendor/ort/", document.baseURI).href;
      ort.env.wasm.numThreads = 1;
      session = await ort.InferenceSession.create("model/murmur_cnn.onnx", { executionProviders: ["wasm"] });
    })();
    return loading;
  }

  // ONNX Runtime can only run one inference at a time, so calls wait in line.
  let queue = Promise.resolve();
  function analyzeRecording(samples) {
    const job = queue.then(() => analyzeNow(samples));
    queue = job.catch(() => {});
    return job;
  }

  // Analyses one recording: spectrogram, heartbeat check, 4-second windows, CNN, average.
  async function analyzeNow(samples) {
    await load();
    const f = config.features;
    const spec = LHFeatures.logMel(samples, f);
    const m = LHFeatures.beatMeasures(spec, beatCheck);
    const beat = { ...m, ok: m.rhythm >= beatCheck.rhythm_min && m.depth >= beatCheck.depth_min };

    const starts = LHFeatures.windowStarts(spec.nFrames, f);
    const size = f.n_mels * f.window_frames;
    const batch = new Float32Array(starts.length * size);
    starts.forEach((s, i) => batch.set(LHFeatures.cutWindow(spec, s, f), i * size));
    const input = new ort.Tensor("float32", batch, [starts.length, 1, f.n_mels, f.window_frames]);
    const { probs } = await session.run({ logmel: input });
    const windows = starts.map((s, i) => ({
      startSec: (s * f.hop) / f.sample_rate,
      probs: Array.from(probs.data.slice(i * 3, i * 3 + 3)),
    }));
    const mean = [0, 1, 2].map(c => windows.reduce((sum, w) => sum + w.probs[c], 0) / windows.length);
    return { spec, windows, probs: mean, beat, duration: samples.length / f.sample_rate };
  }

  // Label for one recording. A recording without a heartbeat rhythm is never
  // trusted as "murmur" or "no murmur": it must be recorded again.
  function recordingLabel(r) {
    const t = config.thresholds;
    if (!r.beat.ok) return UNKNOWN;
    if (r.probs[PRESENT] >= t.present) return PRESENT;
    if (r.probs[UNKNOWN] >= t.unknown) return UNKNOWN;
    return ABSENT;
  }

  // Child-level decision: Present if any valve is Present, else Unknown if
  // any recording is unclear, else Absent.
  function decide(results) {
    const labels = results.map(recordingLabel);
    const trusted = results.filter(r => r.beat.ok);
    const present = trusted.length ? Math.max(...trusted.map(r => r.probs[PRESENT])) : 0;
    let label = ABSENT;
    if (labels.includes(UNKNOWN)) label = UNKNOWN;
    if (labels.includes(PRESENT)) label = PRESENT;
    return { label, labels, present, noBeat: results.filter(r => !r.beat.ok).length, thresholds: config.
thresholds };
  }

  root.LHModel = {
    load, analyzeRecording, decide, recordingLabel, ABSENT, PRESENT, UNKNOWN,
    get config() { return config; },
    get beatCheck() { return beatCheck; },
  };
})(self);
```

web/js/draw.js

Spectrogram, murmur timeline, playback cursor and live waveform drawing.

```
// Drawing helpers shared by the screening, guided recording and quiz views.

const Draw = (() => {
  const COLOR_STOPS = [[13, 8, 35], [62, 24, 104], [158, 47, 127], [234, 105, 84], [252, 216, 141]];

  function colorAt(v) {
    const x = Math.min(0.9999, Math.max(0, v)) * (COLOR_STOPS.length - 1);
    const i = Math.floor(x), f = x - i;
    return COLOR_STOPS[i].map((c, k) => Math.round(c + (COLOR_STOPS[i + 1][k] - c) * f));
  }

  // Low pitches at the bottom, time from left to right.
  function spectrogram(canvas, spec) {
    const nMels = spec.data.length / spec.nFrames;
    canvas.width = spec.nFrames;
    canvas.height = nMels;
    const sorted = Float32Array.from(spec.data).sort();
    const lo = sorted[Math.floor(sorted.length * 0.3)];
    const hi = sorted[Math.floor(sorted.length * 0.995)];
    const ctx = canvas.getContext("2d");
    const img = ctx.createImageData(spec.nFrames, nMels);
    for (let m = 0; m < nMels; m++) {
      for (let f = 0; f < spec.nFrames; f++) {
        const [r, g, b] = colorAt((spec.data[m * spec.nFrames + f] - lo) / (hi - lo));
        const p = ((nMels - 1 - m) * spec.nFrames + f) * 4;
        img.data[p] = r; img.data[p + 1] = g; img.data[p + 2] = b; img.data[p + 3] = 255;
      }
    }
    ctx.putImageData(img, 0, 0);
  }

  function cssVar(name) {
    return getComputedStyle(document.documentElement).getPropertyValue(name);
  }

  // One bar per 4-second window: murmur likelihood, dashed line at the referral threshold.
  function timeline(canvas, result, threshold, caption) {
    const w = canvas.clientWidth || 600, h = 56;
    const dpr = window.devicePixelRatio || 1;
    canvas.width = w * dpr;
    canvas.height = h * dpr;
    const ctx = canvas.getContext("2d");
    ctx.scale(dpr, dpr);
    const x = (sec) => (sec / result.duration) * w;
    const y = (p) => h - 4 - p * (h - 14);

    ctx.strokeStyle = cssVar("--border");
    ctx.setLineDash([3, 3]);
    ctx.beginPath(); ctx.moveTo(0, y(threshold)); ctx.lineTo(w, y(threshold)); ctx.stroke();
    ctx.setLineDash([]);
    ctx.fillStyle = cssVar("--muted");
    ctx.font = "10px system-ui, sans-serif";
    ctx.fillText(caption, 4, 10);

    const span = Math.min(4, result.duration);
    result.windows.forEach(win => {
      const p = win.probs[1];
      ctx.fillStyle = p >= threshold ? cssVar("--bad") : cssVar("--ok");
      ctx.globalAlpha = 0.25 + 0.6 * p;
      const left = x(win.startSec + span / 2 - 1), right = x(Math.min(result.duration, win.startSec + span / 2 + 1));
      ctx.fillRect(left, y(p), Math.max(2, right - left - 1), h - 4 - y(p));
    });
    ctx.globalAlpha = 1;
  }

  // Play button with a moving cursor over the spectrogram.
  function playback(el, duration) {
    const audio = el.querySelector("audio");
    const btn = el.querySelector(".play");
    const head = el.querySelector(".playhead");
    let raf = null;
    const tick = () => {
      if (head) head.style.left = (100 * audio.currentTime / duration) + "%";
      if (!audio.paused) raf = requestAnimationFrame(tick);
    };
    btn.addEventListener("click", () => {
      document.querySelectorAll("audio").forEach(a => { if (a !== audio) a.pause(); });
      if (audio.paused) audio.play(); else audio.pause();
    });
    audio.addEventListener("play", () => { btn.textContent = "▮▮"; if (head) head.style.display = "block"; tick(); });
    audio.addEventListener("pause", () => { btn.textContent = "■"; cancelAnimationFrame(raf); });
    audio.addEventListener("ended", () => { btn.textContent = "■"; if (head) head.style.display = "none"; });
  }

  // Live microphone view: min/max envelope of the last few seconds.
```

```
  function wave(canvas, samples) {
    const w = canvas.clientWidth || 300, h = canvas.clientHeight || 70;
    const dpr = window.devicePixelRatio || 1;
    if (canvas.width !== w * dpr) { canvas.width = w * dpr; canvas.height = h * dpr; }
    const ctx = canvas.getContext("2d");
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    ctx.clearRect(0, 0, w, h);
    ctx.fillStyle = cssVar("--accent");
    let peak = 1e-4;
    for (let i = 0; i < samples.length; i++) peak = Math.max(peak, Math.abs(samples[i]));
    const per = Math.max(1, Math.floor(samples.length / w));
    for (let px = 0; px < w; px++) {
      let lo = 0, hi = 0;
      for (let i = px * per; i < Math.min(samples.length, (px + 1) * per); i++) {
        lo = Math.min(lo, samples[i]); hi = Math.max(hi, samples[i]);
      }
      const top = h / 2 - (hi / peak) * (h / 2 - 2), bottom = h / 2 - (lo / peak) * (h / 2 - 2);
      ctx.fillRect(px, top, 1, Math.max(1, bottom - top));
    }
  }

  return { spectrogram, timeline, playback, wave };
})();
```

web/js/app.js

Screening view: real cases, uploads, verdict, per-valve results.

```
// Screening view: real cases, uploads, results per child and per valve.
// Recording from a microphone lives in guided.js.

const $ = (sel) => document.querySelector(sel);
const pct = (x) => (I18N.lang === "tr" ? "%" + Math.round(x * 100) : Math.round(x * 100) + "%");
const t = (key, vars) => I18N.t(key, vars);
const LABEL_CLASS = ["ok", "bad", "warn"];

const App = (() => {
  let recordings = [];   // { id, site, name, url, result }
  let currentCase = null;
  let nextId = 1;

  const siteName = (site) => t("site." + site);

  // Adds all recordings first (so the result waits for every one of them),
  // then analyses them one by one. Results for recordings that were cleared
  // in the meantime (e.g. the user picked another case) are thrown away.
  async function addRecordings(items) {
    const recs = items.map(({ buffer, site, name }) => ({
      id: nextId++, site, name, buffer, url: URL.createObjectURL(new Blob([buffer])), result: null,
    }));
    recordings.push(...recs);
    renderAll();
    for (const rec of recs) {
      if (!recordings.includes(rec)) return;
      try {
        const samples = await LHFeatures.decodeToRate(rec.buffer, LHModel.config?.features.sample_rate || 4000);
        const result = await LHModel.analyzeRecording(samples);
        if (!recordings.includes(rec)) return;
        rec.result = result;
      } catch (err) {
        console.error(err);
        rec.error = true;
      }
      delete rec.buffer;
      renderAll();
    }
  }

  function replaceSite(site) {
    recordings.filter(r => r.site === site).forEach(r => URL.revokeObjectURL(r.url));
    recordings = recordings.filter(r => r.site !== site);
  }

  function clearRecordings() {
    recordings.forEach(r => URL.revokeObjectURL(r.url));
    recordings = [];
    currentCase = null;
    document.querySelectorAll("[data-case]").forEach(b => b.classList.remove("active"));
    renderAll();
  }

  function leaveDemoCase() {
    if (currentCase) clearRecordings();
  }

  // ------------------------------------------------------------ demo cases and uploads

  function renderDemoCases() {
    $("#demo-cases").innerHTML = METRICS.demo_cases.map((c, i) => `
      <button class="case${currentCase === c ? " active" : ""}" data-case="${i}">
        <span class="case-title">${t("demo.case", { n: i + 1 })}</span>
        <span class="case-meta">${t("age." + c.age)}, ${t("sex." + c.sex)} · ${t("demo.recs", { n: c.recordings.length })}</span>
      </button>`).join("");
    document.querySelectorAll("[data-case]").forEach(btn => btn.addEventListener("click", () => loadCase(+btn.dataset.case)));
  }

  async function loadCase(index) {
    clearRecordings();
    document.querySelectorAll("[data-case]").forEach(b => b.classList.toggle("active", +b.dataset.case === index));
    currentCase = METRICS.demo_cases[index];
    const caseRef = currentCase;
    if (window.innerWidth < 980) $("#verdict-card").scrollIntoView({ behavior: "smooth", block: "start" });
    const items = await Promise.all(caseRef.recordings.map(async r => ({
      buffer: await (await fetch("samples/" + r.file)).arrayBuffer(), site: r.location, name: r.file,
    })));
    if (currentCase !== caseRef) return;
    await addRecordings(items);
  }

  function siteFromName(name) {
    const m = name.match(/(?:^|[_\-\s])(AV|PV|TV|MV|Phc)(?:[_\-\s.]|$)/i);
    if (!m) return null;
```

```
    const s = m[1].toUpperCase();
    return s === "PHC" ? "Phc" : s;
  }

  function setupUpload() {
    const input = $("#file-input");
    const drop = $("#file-drop");
    const handle = async (files) => {
      leaveDemoCase();
      const order = ["AV", "PV", "TV", "MV"];
      const used = new Set(recordings.map(r => r.site));
      const items = [];
      for (const file of files) {
        const site = siteFromName(file.name) || order.find(s => !used.has(s)) || "Phc";
        used.add(site);
        items.push({ buffer: await file.arrayBuffer(), site, name: file.name });
      }
      await addRecordings(items);
    };
    input.addEventListener("change", () => { handle([...input.files]); input.value = ""; });
    drop.addEventListener("dragover", e => { e.preventDefault(); drop.classList.add("over"); });
    drop.addEventListener("dragleave", () => drop.classList.remove("over"));
    drop.addEventListener("drop", e => {
      e.preventDefault();
      drop.classList.remove("over");
      handle([...e.dataTransfer.files]);
    });
  }

  // ---------------------------------------------------------- results

  function decision() {
    const done = recordings.filter(r => r.result);
    if (!done.length || done.length < recordings.filter(r => !r.error).length) return null;
    const d = LHModel.decide(done.map(r => r.result));
    return { ...d, done, sites: done.map((r, i) => ({ site: r.site, label: d.labels[i], present: r.result.probs[
1] })) };
  }

  function renderAll() {
    renderChest();
    renderVerdict();
    renderSiteResults();
  }

  function siteLabel(site) {
    const here = recordings.filter(r => r.site === site);
    const done = here.filter(r => r.result);
    if (!done.length) return here.some(r => !r.error) ? "busy" : null;
    const labels = done.map(r => LHModel.recordingLabel(r.result));
    if (labels.includes(LHModel.PRESENT)) return LABEL_CLASS[1];
    if (labels.includes(LHModel.UNKNOWN)) return LABEL_CLASS[2];
    return LABEL_CLASS[0];
  }

  function renderChest() {
    document.querySelectorAll(".site").forEach(g => {
      const focus = g.classList.contains("focus") ? " focus" : "";
      g.setAttribute("class", "site " + (siteLabel(g.dataset.site) || "") + focus);
    });
  }

  function highlightSite(site) {
    document.querySelectorAll(".site").forEach(g => g.classList.toggle("focus", g.dataset.site === site));
  }

  function verdictHtml(icon, title, text) {
    return `<div class="verdict-icon" aria-hidden="true">${icon}</div>
      <div><div class="verdict-title">${title}</div><div class="verdict-text">${text}</div></div>`;
  }

  function renderVerdict() {
    const box = $("#verdict");
    const truth = $("#truth");
    const done = recordings.filter(r => r.result);
    const pending = recordings.filter(r => !r.result && !r.error).length;
    truth.hidden = true;
    $("#verdict-actions").hidden = !done.length || pending > 0;

    if (!recordings.length) {
      box.className = "verdict empty";
      box.innerHTML = verdictHtml("■", t("verdict.empty.title"), t("verdict.empty.text"));
      return;
    }
    if (!done.length) {
      box.className = "verdict empty";
      box.innerHTML = verdictHtml("…", t("verdict.busy.title"), t("verdict.busy.text"));
      return;
    }

    const d = LHModel.decide(done.map(r => r.result));
```

```
    const trusted = done.filter(r => r.result.beat.ok);
    const strongest = trusted.length ? trusted.reduce((a, b) => (b.result.probs[1] > a.result.probs[1] ? b : a)) : done[0];
    const unclear = [...new Set(done.filter((r, i) => d.labels[i] === LHModel.UNKNOWN).map(r => r.site))];
    const noBeat = [...new Set(done.filter(r => !r.result.beat.ok).map(r => r.site))];
    const more = pending ? " " + t("verdict.pending", { n: pending }) : "";

    if (d.label === LHModel.PRESENT) {
      box.className = "verdict bad";
      box.innerHTML = verdictHtml("!", t("verdict.refer.title"),
        t("verdict.refer.text", { site: siteName(strongest.site).toLocaleLowerCase(I18N.locale), p: pct(d.present) }) + more);
    } else if (d.label === LHModel.UNKNOWN && noBeat.length && done.every(r => !r.result.beat.ok)) {
      box.className = "verdict warn";
      box.innerHTML = verdictHtml("?", t("verdict.nobeat.title"), t("verdict.nobeat.text") + more);
    } else if (d.label === LHModel.UNKNOWN) {
      box.className = "verdict warn";
      const why = noBeat.length ? " " + t("verdict.unclear.nobeat", { sites: noBeat.join(", ") }) : "";
      box.innerHTML = verdictHtml("?", t("verdict.unclear.title"),
        t("verdict.unclear.text", { sites: unclear.join(", ") }) + why + more);
    } else {
      box.className = "verdict ok";
      box.innerHTML = verdictHtml("✓", t("verdict.clear.title"),
        t("verdict.clear.text", { p: pct(d.present), th: pct(d.thresholds.present) }) + more);
    }

    if (currentCase && !pending) {
      const c = currentCase;
      const truthLabel = ["Absent", "Present", "Unknown"].indexOf(c.truth);
      const agree = truthLabel === d.label;
      let details;
      if (c.truth === "Present") {
        details = t("truth.murmur", { sites: c.murmur_locations.join(", ") });
        if (c.timing !== "nan") details += " · " + t("timing." + c.timing);
        if (c.grade !== "nan") details += ", " + t("truth.grade", { g: c.grade });
      } else {
        details = c.truth === "Absent" ? t("truth.none") : t("truth.unknown");
      }
      truth.hidden = false;
      truth.className = "truth " + (agree ? "agree" : "disagree");
      truth.innerHTML = `<b>${t("truth.label")}</b> ${details}. <span>${agree ? t("truth.agree") : t("truth.disagree")}</span>`;
    }
  }

  function renderSiteResults() {
    const container = $("#site-results");
    const order = { AV: 0, PV: 1, TV: 2, MV: 3, Phc: 4 };
    const sorted = [...recordings].sort((a, b) => order[a.site] - order[b.site] || a.id - b.id);
    const existing = new Map([...container.children].map(el => [+el.dataset.id, el]));
    sorted.forEach(rec => {
      let el = existing.get(rec.id);
      const state = rec.error ? "error" : rec.result ? "done" : "busy";
      if (!el || el.dataset.state !== state || el.dataset.lang !== I18N.lang) {
        const fresh = buildSiteCard(rec, state);
        if (el) el.replaceWith(fresh); else container.appendChild(fresh);
        el = fresh;
      }
      container.appendChild(el);
      existing.delete(rec.id);
    });
    existing.forEach(el => el.remove());
  }

  function buildSiteCard(rec, state) {
    const el = document.createElement("div");
    el.className = "card site-card";
    el.dataset.id = rec.id;
    el.dataset.state = state;
    el.dataset.lang = I18N.lang;
    const title = `<div class="site-head"><h3>${siteName(rec.site)} <span class="site-code">${rec.site}</span></h3>`;

    if (state === "busy") {
      el.innerHTML = `${title}<span class="badge">${t("card.busy")}</span></div><div class="skeleton"></div>`;
      return el;
    }
    if (state === "error") {
      el.innerHTML = `${title}<span class="badge warn">${t("card.errorBadge")}</span></div><p class="small">${t("card.error")}</p>`;
      return el;
    }

    const r = rec.result;
    const th = LHModel.config.thresholds;
    const label = LHModel.recordingLabel(r);
    const badge = r.beat.ok ? t("label." + label) : t("card.nobeat");
    el.innerHTML = `${title}<span class="badge ${LABEL_CLASS[label]}">${badge}</span></div>
      <div class="spec-wrap">
        <canvas class="spec" aria-label="${t("card.specAria")}"></canvas>
        <div class="playhead"></div>
```

```
        <span class="axis-label top">1000 Hz</span><span class="axis-label bottom">25 Hz</span>
      </div>
      <canvas class="timeline" aria-label="${t("card.timelineAria")}"></canvas>
      <div class="site-foot">
        <button class="play" aria-label="${t("card.play")}">■</button>
        <span class="small muted">${r.duration.toFixed(1)} s · ${r.beat.ok ? t("card.beatOk") : t("card.beatNo")}
.
          ${t("card.murmur")} <b>${pct(r.probs[1])}</b> · ${t("card.unclear")} ${pct(r.probs[2])}</span>
      </div>
      <audio preload="none" src="${rec.url}"></audio>`;
    Draw.spectrogram(el.querySelector(".spec"), r.spec);
    requestAnimationFrame(() => Draw.timeline(el.querySelector(".timeline"), r, th.present, t("card.
timelineCaption")));
    Draw.playback(el, r.duration);
    return el;
  }

  // ------------------------------------------------------- setup

  function init() {
    setupUpload();
    renderDemoCases();
    renderAll();
    $("#print-btn").addEventListener("click", () => {
      $("#print-meta").textContent = t("print.meta", { date: new Date().toLocaleString(I18N.locale), n:
recordings.length });
      window.print();
    });
    $("#clear-btn").addEventListener("click", clearRecordings);
    document.querySelectorAll(".site").forEach(g => g.addEventListener("click", () => {
      $("#record-site").value = g.dataset.site;
      highlightSite(g.dataset.site);
    }));
    const caseFromUrl = parseInt(new URLSearchParams(location.search).get("case"), 10);
    if (caseFromUrl >= 1 && caseFromUrl <= METRICS.demo_cases.length) {
      document.querySelector('[data-source="demo"]').click();
      loadCase(caseFromUrl - 1);
    }
  }

  function relabel() {
    renderDemoCases();
    renderAll();
  }

  return {
    init, relabel, addRecordings, replaceSite, clearRecordings, leaveDemoCase, highlightSite, decision,
    get isDemo() { return currentCase !== null; },
  };
})();
```

web/js/guided.js

Guided recording with live heartbeat and sound-level check.

```
// Guided recording: walks the health worker through the four listening sites,
// shows the live sound with a heartbeat check, records 15 seconds per site and
// hands each recording to App for analysis.

const Guided = (() => {
  const SITES = ["AV", "PV", "TV", "MV"];
  const RECORD_SECONDS = 15;
  const GET_READY_SECONDS = 3;
  const LIVE_SECONDS = 6;

  // Runs on the audio thread and forwards raw microphone samples.
  const WORKLET = `
    class Tap extends AudioWorkletProcessor {
      process(inputs) {
        if (inputs[0] && inputs[0][0]) this.port.postMessage(inputs[0][0].slice(0));
        return true;
      }
    }
    registerProcessor("tap", Tap);`;

  let ctx = null, stream = null, node = null;
  let live = [];            // last LIVE_SECONDS of samples, for the quality check
  let take = null;          // samples of the recording in progress
  let queue = [];           // sites still to record
  let site = null, phase = "idle", phaseEnds = 0, total = 0, step = 0;
  let timer = null, liveCheck = null, liveState = { level: 0, beat: null };

  // ------------------------------------------------------------ audio

  async function openMic() {
    stream = await navigator.mediaDevices.getUserMedia({
      audio: { echoCancellation: false, noiseSuppression: false, autoGainControl: false },
    });
    ctx = new (window.AudioContext || window.webkitAudioContext)();
    const url = URL.createObjectURL(new Blob([WORKLET], { type: "application/javascript" }));
    await ctx.audioWorklet.addModule(url);
    URL.revokeObjectURL(url);
    node = new AudioWorkletNode(ctx, "tap");
    node.port.onmessage = (e) => onSamples(e.data);
    ctx.createMediaStreamSource(stream).connect(node);
  }

  function closeMic() {
    clearInterval(timer);
    clearInterval(liveCheck);
    if (node) node.port.onmessage = null;
    if (stream) stream.getTracks().forEach(tr => tr.stop());
    if (ctx) ctx.close();
    ctx = stream = node = null;
    live = [];
    take = null;
  }

  function onSamples(chunk) {
    live.push(chunk);
    let buffered = live.reduce((n, c) => n + c.length, 0);
    while (buffered - live[0].length > LIVE_SECONDS * ctx.sampleRate) buffered -= live.shift().length;
    if (phase === "recording") take.push(chunk);
  }

  function concat(chunks) {
    const out = new Float32Array(chunks.reduce((n, c) => n + c.length, 0));
    let o = 0;
    for (const c of chunks) { out.set(c, o); o += c.length; }
    return out;
  }

  // Quick resampling for the live check only: average blocks, then pick samples.
  function toModelRate(x, rate, target) {
    const ratio = rate / target;
    const width = Math.max(1, Math.round(ratio));
    const out = new Float64Array(Math.floor(x.length / ratio));
    for (let i = 0; i < out.length; i++) {
      const start = Math.floor(i * ratio);
      let sum = 0, n = 0;
      for (let j = start; j < Math.min(x.length, start + width); j++) { sum += x[j]; n++; }
      out[i] = n ? sum / n : 0;
    }
    return out;
  }

  // 16-bit PCM WAV, so the normal upload path (and playback) can use the take.
  function encodeWav(samples, rate) {
    const buf = new ArrayBuffer(44 + samples.length * 2);
    const v = new DataView(buf);
    const str = (o, s) => [...s].forEach((ch, i) => v.setUint8(o + i, ch.charCodeAt(0)));
    str(0, "RIFF"); v.setUint32(4, 36 + samples.length * 2, true); str(8, "WAVE");
```

```
    str(12, "fmt "); v.setUint32(16, 16, true); v.setUint16(20, 1, true); v.setUint16(22, 1, true);
    v.setUint32(24, rate, true); v.setUint32(28, rate * 2, true); v.setUint16(32, 2, true); v.setUint16(34, 16, true);
    str(36, "data"); v.setUint32(40, samples.length * 2, true);
    for (let i = 0; i < samples.length; i++) {
      v.setInt16(44 + i * 2, Math.max(-1, Math.min(1, samples[i])) * 32767, true);
    }
    return buf;
  }

  // ---------------------------------------------------------- live quality check

  function checkLive() {
    if (!ctx || !live.length) return;
    const x = concat(live);
    Draw.wave($("#live-wave"), x.subarray(Math.max(0, x.length - 3 * ctx.sampleRate)));
    const recent = x.subarray(Math.max(0, x.length - 0.3 * ctx.sampleRate));
    let sq = 0;
    for (let i = 0; i < recent.length; i++) sq += recent[i] * recent[i];
    liveState.level = Math.sqrt(sq / Math.max(1, recent.length));
    if (x.length >= 4 * ctx.sampleRate && LHModel.config) {
      const f = LHModel.config.features;
      const spec = LHFeatures.logMel(toModelRate(x, ctx.sampleRate, f.sample_rate), f);
      const m = LHFeatures.beatMeasures(spec, LHModel.beatCheck);
      liveState.beat = m.rhythm >= LHModel.beatCheck.rhythm_min && m.depth >= LHModel.beatCheck.depth_min;
    }
    renderLive();
  }

  function renderLive() {
    const chip = $("#beat-chip");
    let cls = "warn", text = t("live.searching");
    if (liveState.level < 0.002) { cls = "bad"; text = t("live.quiet"); }
    else if (liveState.beat) { cls = "ok"; text = t("live.beat"); }
    chip.className = "chip-status " + cls;
    chip.textContent = text;
    const db = 20 * Math.log10(liveState.level + 1e-9);
    $("#level-meter span").style.width = Math.max(0, Math.min(100, (db + 70) / 60 * 100)) + "%";
  }

  // ---------------------------------------------------------- steps

  function renderStep() {
    const left = Math.max(0, Math.ceil((phaseEnds - performance.now()) / 1000));
    $("#guide-step").textContent = total > 1
      ? t("guide.step", { n: step, total, site: t("site." + site) })
      : t("guide.single", { site: t("site." + site) });
    $("#guide-instruction").textContent = t("guide.place." + site);
    $("#countdown").textContent = phase === "ready" ? t("guide.ready", { s: left }) : t("guide.recording", { s: left });
    $("#countdown").className = "countdown " + phase;
  }

  function startSite(next, again = false) {
    if (!again) step += 1;
    site = next;
    App.highlightSite(site);
    phase = "ready";
    phaseEnds = performance.now() + GET_READY_SECONDS * 1000;
    renderStep();
  }

  function tick() {
    if (performance.now() < phaseEnds) return renderStep();
    if (phase === "ready") {
      phase = "recording";
      take = [];
      phaseEnds = performance.now() + RECORD_SECONDS * 1000;
      return renderStep();
    }
    if (phase === "recording") finishSite();
  }

  function finishSite() {
    const samples = concat(take);
    take = null;
    App.replaceSite(site);
    App.addRecordings([{ buffer: encodeWav(samples, ctx.sampleRate), site, name: t("guide.takeName", { site }) }]);
    nextSite();
  }

  function nextSite() {
    if (queue.length) return startSite(queue.shift());
    stop(t("guide.done"));
  }

  async function begin(sites) {
    const status = $("#record-status");
    try {
      await LHModel.load();
```

```
      await openMic();
    } catch (err) {
      console.error(err);
      status.textContent = t("guide.denied");
      closeMic();
      return;
    }
    App.leaveDemoCase();
    status.textContent = "";
    queue = sites.slice(1);
    total = sites.length;
    step = 0;
    $("#guide-idle").hidden = true;
    $("#guide-live").hidden = false;
    liveState = { level: 0, beat: null };
    startSite(sites[0]);
    timer = setInterval(tick, 200);
    liveCheck = setInterval(checkLive, 1000);
  }

  function stop(message) {
    phase = "idle";
    closeMic();
    $("#guide-idle").hidden = false;
    $("#guide-live").hidden = true;
    $("#record-status").textContent = message || "";
    App.highlightSite(null);
  }

  function init() {
    $("#guide-start").addEventListener("click", () => begin(SITES));
    $("#record-one").addEventListener("click", () => begin([$("#record-site").value]));
    $("#record-site").addEventListener("change", () => App.highlightSite($("#record-site").value));
    $("#guide-skip").addEventListener("click", () => { take = null; nextSite(); });
    $("#guide-redo").addEventListener("click", () => startSite(site, true));
    $("#guide-stop").addEventListener("click", () => stop(t("guide.stopped")));
  }

  return { init, relabel: () => { if (phase !== "idle") { renderStep(); renderLive(); } }, encodeWav };
})();
```

web/js/quiz.js

Train-your-ear mode.

```
// "Train your ear": the health worker listens to real recordings, decides
// murmur / no murmur, then sees the cardiologist's answer, the AI's answer
// (from a cross-validation model that never heard that child) and the spectrogram.

const Quiz = (() => {
  let index = 0;
  let answers = [];   // { you: bool, ai: bool } correctness per item
  let revealed = false;
  let audioUrl = null;

  const items = () => METRICS.quiz || [];

  function describe(item) {
    if (!item.murmur) return t("quiz.truthNone");
    const parts = [t("quiz.truthMurmur")];
    if (item.grade !== "nan") parts.push(t("truth.grade", { g: item.grade }));
    if (item.timing !== "nan") parts.push(t("timing." + item.timing));
    if (item.pitch !== "nan") parts.push(t("quiz.pitch." + item.pitch));
    if (item.quality !== "nan") parts.push(t("quiz.quality." + item.quality));
    return parts.join(" · ");
  }

  async function loadAudio(item) {
    const buf = await (await fetch("quiz/" + item.file)).arrayBuffer();
    if (audioUrl) URL.revokeObjectURL(audioUrl);
    audioUrl = URL.createObjectURL(new Blob([buf]));
    return buf;
  }

  async function render() {
    const card = $("#quiz-card");
    const list = items();
    renderScore();
    if (!list.length) { card.innerHTML = `<p>${t("quiz.none")}</p>`; return; }
    if (index >= list.length) return renderEnd(card);

    const item = list[index];
    revealed = false;
    card.innerHTML = `
      <div class="quiz-head">
        <span class="muted small">${t("quiz.progress", { n: index + 1, total: list.length })}</span>
        <span class="badge">${t("site." + item.location)}</span>
      </div>
      <div class="quiz-listen">
        <button class="play big" aria-label="${t("card.play")}">■</button>
        <div>
          <div class="quiz-question">${t("quiz.question")}</div>
          <div class="muted small">${t("quiz.hint")}</div>
        </div>
      </div>
      <div class="quiz-answers">
        <button class="btn answer" data-answer="1">${t("quiz.yes")}</button>
        <button class="btn answer secondary" data-answer="0">${t("quiz.no")}</button>
      </div>
      <div class="quiz-reveal" hidden></div>
      <audio preload="auto"></audio>`;
    // Answers work straight away; sound and spectrogram follow when the file arrives.
    const bufReady = loadAudio(item);
    card.querySelectorAll(".answer").forEach(b => b.addEventListener("click", () => answer(item, b.dataset.answer
=== "1", bufReady)));
    const audio = card.querySelector("audio");
    const play = card.querySelector(".play");
    play.addEventListener("click", async () => {
      await bufReady;
      if (audio.paused) audio.play(); else audio.pause();
    });
    audio.addEventListener("play", () => { play.textContent = "▮▮"; });
    audio.addEventListener("pause", () => { play.textContent = "■"; });
    await bufReady;
    if (items()[index] === item) audio.src = audioUrl;
  }

  function answer(item, saysMurmur, bufReady) {
    if (revealed) return;
    revealed = true;
    const you = saysMurmur === item.murmur;
    const ai = item.ai_flags === item.murmur;
    answers[index] = { you, ai };
    const card = $("#quiz-card");
    card.querySelectorAll(".answer").forEach(b => {
      b.disabled = true;
      if ((b.dataset.answer === "1") === saysMurmur) b.classList.add(you ? "right" : "wrong");
    });
    const box = card.querySelector(".quiz-reveal");
    box.hidden = false;
    box.innerHTML = `
      <div class="reveal-verdict ${you ? "ok" : "bad"}">${you ? t("quiz.right") : t("quiz.wrong")}</div>
```

```
      <p><b>${t("quiz.cardiologist")}</b> ${describe(item)}</p>
      <p><b>${t("quiz.ai")}</b> ${t(item.ai_flags ? "quiz.aiYes" : "quiz.aiNo", { p: pct(item.ai_present_prob) })}
        <span class="${ai ? "ok-text" : "bad-text"}">${ai ? "✓" : "✗"}</span></p>
      <div class="spec-wrap"><canvas class="spec"></canvas>
        <span class="axis-label top">1000 Hz</span><span class="axis-label bottom">25 Hz</span></div>
      <p class="muted small">${t(item.murmur ? "quiz.lookMurmur" : "quiz.lookClear")}</p>
      <button class="btn" id="quiz-next">${index + 1 < items().length ? t("quiz.next") : t("quiz.finish")}</
button>`;
    bufReady.then(buf => {
      const wav = LHFeatures.parseWav(buf);
      if (wav && LHModel.config) Draw.spectrogram(box.querySelector(".spec"), LHFeatures.logMel(wav.samples,
LHModel.config.features));
    });
    box.querySelector("#quiz-next").addEventListener("click", () => { index += 1; render(); });
    renderScore();
  }

  function renderScore() {
    const done = answers.filter(Boolean);
    const you = done.filter(a => a.you).length, ai = done.filter(a => a.ai).length;
    $("#scoreboard").innerHTML = `
      <div class="score-row"><span>${t("quiz.you")}</span><b>${you} / ${done.length}</b></div>
      <div class="score-row"><span>${t("quiz.aiName")}</span><b>${ai} / ${done.length}</b></div>
      <p class="muted small">${t("quiz.scoreNote")}</p>`;
  }

  function renderEnd(card) {
    const you = answers.filter(a => a && a.you).length, ai = answers.filter(a => a && a.ai).length;
    card.innerHTML = `
      <h2>${t("quiz.endTitle")}</h2>
      <p>${t("quiz.endText", { you, ai, total: items().length })}</p>
      <p class="muted small">${t("quiz.endNote")}</p>
      <button class="btn" id="quiz-again">${t("quiz.again")}</button>`;
    card.querySelector("#quiz-again").addEventListener("click", () => { index = 0; answers = []; render(); });
  }

  return { init: render, relabel: () => { if (!revealed) render(); else renderScore(); } };
})();
```

web/js/log.js

Screening log stored on the device, with CSV export.

```
// Screening log for a school or village screening day. Stored only in this
// browser (localStorage); nothing is sent anywhere.

const Log = (() => {
  const KEY = "littleheart.log.v1";
  let filter = "all";

  function load() {
    try { return JSON.parse(localStorage.getItem(KEY)) || []; } catch { return []; }
  }

  function store(entries) {
    try { localStorage.setItem(KEY, JSON.stringify(entries)); return true; } catch { return false; }
  }

  function saveCurrent() {
    const d = App.decision();
    if (!d) return;
    const input = $("#child-id");
    const entries = load();
    entries.unshift({
      id: Date.now().toString(36),
      child: input.value.trim() || t("log.unnamed", { n: entries.length + 1 }),
      date: new Date().toISOString(),
      label: d.label,
      present: Math.round(d.present * 100) / 100,
      sites: d.sites.map(s => ({ site: s.site, label: s.label })),
      demo: App.isDemo,
    });
    const btn = $("#save-btn");
    btn.textContent = store(entries) ? t("log.saved") : t("log.saveFailed");
    setTimeout(() => { btn.textContent = t("log.save"); }, 1800);
    input.value = "";
    render();
  }

  const labelText = (label) => t(["log.status0", "log.status1", "log.status2"][label]);

  function render() {
    const entries = load();
    const count = (l) => entries.filter(e => e.label === l).length;
    const stats = [
      [entries.length, t("log.statScreened"), ""],
      [count(1), t("log.statRefer"), "bad"],
      [count(2), t("log.statRerecord"), "warn"],
      [count(0), t("log.statClear"), "ok"],
    ];
    $("#log-stats").innerHTML = stats.map(([n, label, cls]) =>
      `<div class="card metric"><div class="metric-big ${cls}-num">${n}</div><h3>${label}</h3></div>`).join("");

    const shown = entries.filter(e => filter === "all" || String(e.label) === filter);
    const table = $("#log-table");
    if (!shown.length) {
      table.innerHTML = `<tr><td class="muted">${entries.length ? t("log.noneFilter") : t("log.empty")}</td></
tr>`;
      return;
    }
    table.innerHTML = `
      <thead><tr><th>${t("log.colChild")}</th><th>${t("log.colDate")}</th><th>${t("log.colResult")}</th>
        <th>${t("log.colSites")}</th><th></th></tr></thead>
      <tbody>${shown.map(e => `
        <tr>
          <td>${escapeHtml(e.child)}${e.demo ? ` <span class="badge">${t("log.demo")}</span>` : ""}</td>
          <td>${new Date(e.date).toLocaleString(I18N.locale, { dateStyle: "short", timeStyle: "short" })}</td>
          <td><span class="badge ${LABEL_CLASS[e.label]}">${labelText(e.label)}</span></td>
          <td class="small">${e.sites.map(s => `${s.site}: ${t("label." + s.label)}`).join(", ")}</td>
          <td><button class="link-btn" data-delete="${e.id}" aria-label="${t("log.delete")}">✕</button></td>
        </tr>`).join("")}</tbody>`;
    table.querySelectorAll("[data-delete]").forEach(b => b.addEventListener("click", () => {
      store(load().filter(e => e.id !== b.dataset.delete));
      render();
    }));
  }

  function escapeHtml(s) {
    return s.replace(/[&<>"']/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[
c]));
  }

  function exportCsv() {
    const rows = [["child", "date", "result", "murmur_likelihood", "sites", "demo_case"]];
    load().forEach(e => rows.push([e.child, e.date, ["no murmur", "refer", "record again"][e.label], e.present,
      e.sites.map(s => `${s.site}:${["no murmur", "murmur", "unclear"][s.label]}`).join(" "), e.demo ? "yes" :
"no"]));
    const csv = rows.map(r => r.map(v => `"${String(v).replace(/"/g, '""')}"`).join(",")).join("\r\n");
    const a = document.createElement("a");
    a.href = URL.createObjectURL(new Blob(["■" + csv], { type: "text/csv" }));
```

```
    a.download = `littleheart-screening-${new Date().toISOString().slice(0, 10)}.csv`;
    a.click();
    setTimeout(() => URL.revokeObjectURL(a.href), 1000);
  }

  function init() {
    $("#save-btn").addEventListener("click", saveCurrent);
    $("#log-export").addEventListener("click", exportCsv);
    $("#log-wipe").addEventListener("click", () => {
      if (confirm(t("log.confirmWipe"))) { store([]); render(); }
    });
    document.querySelectorAll("[data-filter]").forEach(b => b.addEventListener("click", () => {
      filter = b.dataset.filter;
      document.querySelectorAll("[data-filter]").forEach(x => x.classList.toggle("active", x === b));
      render();
    }));
    render();
  }

  return { init, relabel: render };
})();
```

web/js/validation.js

Validation tab: metrics, error analysis, robustness, external test.

```
// Validation tab: cross-validation results, error analysis, robustness to
// damaged audio, the external dataset test, the safety check and limits.

const Validation = (() => {
  let external = null;
  const fmt2 = (x) => (x === null || x === undefined ? "–" : x.toFixed(2));
  const pctOrDash = (x) => (x === null || x === undefined ? "–" : pct(x));

  function cards() {
    const m = METRICS, c = m.cnn, ds = m.dataset;
    const pm = (k) => `${pct(c[k].mean)} <small>± ${Math.round(c[k].std * 100)}</small>`;
    $("#val-intro").textContent = t("val.intro", { folds: m.folds, children: ds.cv_children, recordings: ds.recordings.toLocaleString(I18N.locale) });
    const list = [
      [c.weighted_accuracy.mean.toFixed(2) + ` <small>± ${c.weighted_accuracy.std.toFixed(2)}</small>`, t("val.card1"), t("val.card1Text")],
      [pm("sensitivity"), t("val.card2"), t("val.card2Text")],
      [pm("specificity"), t("val.card3"), t("val.card3Text")],
      [c.auc.mean.toFixed(2) + ` <small>± ${c.auc.std.toFixed(2)}</small>`, "AUC", t("val.card4Text")],
    ];
    $("#val-cards").innerHTML = list.map(([big, title, text]) =>
      `<div class="card metric"><div class="metric-big">${big}</div><h3>${title}</h3><p class="muted small">${text}</p></div>`).join("");
  }

  function compare() {
    const m = METRICS;
    const bars = [[t("val.barBaseline"), m.baseline.weighted_accuracy.mean, "base"]];
    if (m.comparison) bars.push([t("val.barClean"), m.comparison.cnn.weighted_accuracy.mean, "base2"]);
    bars.push([t("val.barOurs"), m.cnn.weighted_accuracy.mean, "ours"]);
    bars.push([t("val.barTop"), m.challenge_reference.top_hidden_test, "ref"]);
    $("#compare").innerHTML = bars.map(([name, v, cls]) => `
      <div class="cmp-row"><span class="cmp-name">${name}</span>
        <span class="cmp-track"><span class="cmp-bar ${cls}" style="width:${v * 100}%"></span></span>
        <span class="cmp-val">${v.toFixed(2)}</span></div>`).join("");
    $("#compare-note").textContent = t("val.compareNote");
  }

  function confusion() {
    const names = [t("label.0"), t("label.1"), t("label.2")];
    let html = `<div></div>${names.map(n => `<div class="cm-head">${n}</div>`).join("")}`;
    METRICS.confusion.forEach((row, ti) => {
      html += `<div class="cm-side">${names[ti]}</div>`;
      row.forEach((v, p) => { html += `<div class="cm-cell ${ti === p ? "good" : "bad"}">${v}</div>`; });
    });
    $("#confusion").innerHTML = html;
  }

  function grades() {
    const a = METRICS.analysis;
    if (!a) return;
    $("#by-grade").innerHTML = a.by_grade.map(g => {
      const w = (n) => (100 * n / g.n).toFixed(1) + "%";
      return `<div class="stack-row"><span class="stack-name">${t("truth.grade", { g: g.group })} <small>(n=${g.n})</small></span>
        <span class="stack"><span class="seg bad" style="width:${w(g.referred)}"></span><span class="seg warn" style="width:${w(g.rerecord)}"></span><span class="seg miss" style="width:${w(g.missed)}"></span></span>
        <span class="stack-val">${pct(g.referred / g.n)}</span></div>`;
    }).join("");
    const missed = a.by_grade.reduce((s, g) => s + g.missed, 0);
    const faint = (a.by_grade.find(g => g.group === "I/VI") || { missed: 0 }).missed;
    const loud = a.by_grade.find(g => g.group === "III/VI");
    $("#grade-note").textContent = t("val.gradeNote", {
      missed, faint, share: missed ? pct(faint / missed) : "0%",
      loud: loud ? pct(loud.referred / loud.n) : "–",
    });

    $("#by-age").innerHTML = `<thead><tr><th>${t("val.ageCol")}</th><th>${t("val.ageMurmur")}</th><th>${t("val.ageCaught")}</th>
      <th>${t("val.ageNone")}</th><th>${t("val.ageCleared")}</th></tr></thead><tbody>` +
      a.by_age.map(g => `<tr><td>${t("age." + g.group)}</td><td>${g.murmur}</td><td>${pctOrDash(g.sensitivity)}</td>
        <td>${g.no_murmur}</td><td>${pctOrDash(g.specificity)}</td></tr>`).join("") + "</tbody>";
  }

  function robustness() {
    const a = METRICS.analysis;
    if (!a) return;
    const old = METRICS.comparison && Object.fromEntries(METRICS.comparison.robustness.map(r => [r.condition, r]));
    $("#robust-lead").textContent = t(old ? "val.robustLeadCompare" : "val.robustLead");
    const cell = (r, k, fmt) => old ? `<span class="muted">${fmt(old[r.condition][k])} →</span> <b>${fmt(r[k])}</b>` : fmt(r[k]);
    $("#robustness").innerHTML = `<thead><tr><th>${t("val.robCond")}</th><th>${t("val.robScore")}</th><th>${t("val.card2")}</th>
      <th>${t("val.card3")}</th><th>${t("val.robBeat")}</th></tr></thead><tbody>` +
      a.robustness.map(r => `<tr><td>${t("cond." + r.condition)}</td><td>${cell(r, "weighted_accuracy", fmt2)}</
```

```
td>
        <td>${cell(r, "sensitivity", pct)}</td><td>${cell(r, "specificity", pct)}</td><td>${cell(r, "beat_pass", pct)}</td></tr>`).join("") +
      "</tbody>";
    const phone = a.robustness.find(r => r.condition === "phone");
    const rumble = a.robustness.find(r => r.condition === "rumble");
    $("#robust-note").textContent = old && phone && rumble
      ? t("val.robustNote", { before: pct(old.phone.sensitivity), after: pct(phone.sensitivity),
        rb: fmt2(old.rumble.weighted_accuracy), ra: fmt2(rumble.weighted_accuracy) }) : "";
  }

  async function externalTest() {
    if (!external) {
      try { external = await (await fetch("model/external.json")).json(); } catch { return; }
    }
    const old = METRICS.comparison && METRICS.comparison.external;
    const row = (name, r, o) => `<tr><td>${name}</td><td>${r.recordings}</td><td>${pct(r.abnormal_share)}</td>
      <td>${o ? `<span class="muted">${fmt2(o.auc)} →</span> ` : ""}<b>${fmt2(r.auc)}</b></td>
      <td>${pctOrDash(r.sensitivity)}</td><td>${pctOrDash(r.specificity)}</td><td>${pct(r.beat_pass)}</td></tr>`;
    $("#external").innerHTML = `<thead><tr><th>${t("val.extSet")}</th><th>${t("val.extN")}</th><th>${t("val.extAbn")}</th>
      <th>AUC</th><th>${t("val.extSens")}</th><th>${t("val.extSpec")}</th><th>${t("val.robBeat")}</th></tr></thead><tbody>` +
      Object.entries(external.subsets).map(([k, r]) => row(t("val.extSubset", { k: k.toUpperCase() }), r, old && old.subsets[k])).join("") +
      row(`<b>${t("val.extAll")}</b>`, external.overall, old && old.overall) + "</tbody>";
    $("#external-note").textContent = t("val.externalNote", { auc: fmt2(external.overall.auc), beat: pct(external.overall.beat_pass) });
    limits();
  }

  function safety() {
    const b = LHModel.beatCheck;
    if (!b) return;
    const rows = [[t("val.safetyReal", { n: b.real_recordings.toLocaleString(I18N.locale) }), t("val.accepted", { p: (b.real_pass_rate * 100).toFixed(1) + "%" }), true]];
    Object.entries(b.negatives).forEach(([name, v]) =>
      rows.push([t("neg." + name), t("val.rejected", { k: v.rejected, n: v.trials }), v.rejected === v.trials]));
    $("#safety").innerHTML = rows.map(([name, result, good]) =>
      `<div class="safety-row"><span>${name}</span><b class="${good ? "ok-text" : "warn-text"}">${result}</b></div>`).join("");
  }

  function limits() {
    const c = METRICS.cnn;
    const items = [
      t("val.limit1", { n: Math.max(2, Math.round(1 / Math.max(0.01, 1 - c.sensitivity.mean))) }),
      t("val.limit2"),
      external ? t("val.limit3", { auc: fmt2(external.overall.auc) }) : null,
      t("val.limit4"),
      t("val.limit5"),
      t("val.limit6"),
    ].filter(Boolean);
    $("#limits").innerHTML = items.map(x => `<li>${x}</li>`).join("");
  }

  function render() {
    cards(); compare(); confusion(); grades(); robustness(); safety(); limits();
    externalTest();
  }

  return { render, relabel: render, safety };
})();
```

web/js/main.js

Start-up: tabs, language, modules, offline support.

```
// Starts the app: tabs, language, modules, model loading and offline support.

(function main() {
  function showTab(name) {
    document.querySelectorAll(".tab").forEach(tb => tb.classList.toggle("active", tb.dataset.tab === name));
    document.querySelectorAll(".panel").forEach(p => p.classList.toggle("active", p.id === "tab-" + name));
  }

  document.querySelectorAll(".tab").forEach(tab => tab.addEventListener("click", () => {
    showTab(tab.dataset.tab);
    history.replaceState(null, "", location.search + "#" + tab.dataset.tab);
  }));
  const fromUrl = location.hash.slice(1);
  if (document.getElementById("tab-" + fromUrl)) showTab(fromUrl);

  document.querySelectorAll(".source").forEach(btn => btn.addEventListener("click", () => {
    document.querySelectorAll(".source").forEach(b => b.classList.toggle("active", b === btn));
    document.querySelectorAll(".source-panel").forEach(p =>
      p.classList.toggle("active", p.id === "source-" + btn.dataset.source));
  }));

  const langSelect = $("#lang");
  langSelect.value = I18N.lang;
  langSelect.addEventListener("change", () => I18N.setLang(langSelect.value));
  I18N.apply();

  App.init();
  Guided.init();
  Quiz.init();
  Log.init();
  Validation.render();

  document.addEventListener("langchange", () => {
    App.relabel(); Guided.relabel(); Quiz.relabel(); Log.relabel(); Validation.relabel();
  });

  LHModel.load().then(() => Validation.safety()).catch(err => {
    console.error(err);
    $("#verdict").innerHTML = `<div class="verdict-icon">!</div><div><div class="verdict-title">${t("load.
failTitle")}</div>
      <div class="verdict-text">${t("load.failText")}</div></div>`;
  });

  // Offline support: cache the app, model and recordings after the first visit.
  if ("serviceWorker" in navigator && (location.protocol === "https:" || location.hostname === "localhost")) {
    navigator.serviceWorker.register("sw.js").then(() => navigator.serviceWorker.ready).then(() => {
      const el = $("#offline-status");
      el.hidden = false;
      el.textContent = t("offline.ready");
      el.dataset.i18n = "offline.ready";
    }).catch(err => console.warn("Offline mode unavailable", err));
  }
})();
```

web/sw.js

Service worker that keeps the app working offline.

```
// Service worker: keeps LittleHeart working without internet after the first visit.
// Files are served from the cache straight away and refreshed in the background.

const CACHE = "littleheart-v3";
importScripts("model/metrics.js");   // provides METRICS, which lists the demo and quiz recordings

const FILES = [
  "./", "index.html", "manifest.webmanifest", "css/style.css",
  "js/i18n.js", "js/features.js", "js/model.js", "js/draw.js", "js/app.js", "js/guided.js",
  "js/quiz.js", "js/log.js", "js/validation.js", "js/main.js",
  "vendor/ort/ort.wasm.min.js", "vendor/ort/ort-wasm-simd-threaded.wasm", "vendor/ort/ort-wasm-simd-threaded.mjs",
  "model/murmur_cnn.onnx", "model/config.json", "model/heartbeat_check.json", "model/metrics.js", "model/external.
json",
  "icons/icon.svg", "icons/icon-192.png", "icons/icon-512.png",
  ...METRICS.demo_cases.flatMap(c => c.recordings.map(r => "samples/" + r.file)),
  ...(METRICS.quiz || []).map(q => "quiz/" + q.file),
];

self.addEventListener("install", (event) => {
  event.waitUntil(caches.open(CACHE).then(cache => cache.addAll(FILES)).then(() => self.skipWaiting()));
});

self.addEventListener("activate", (event) => {
  event.waitUntil(caches.keys()
    .then(keys => Promise.all(keys.filter(k => k !== CACHE).map(k => caches.delete(k))))
    .then(() => self.clients.claim()));
});

self.addEventListener("fetch", (event) => {
  const url = new URL(event.request.url);
  if (event.request.method !== "GET" || url.origin !== location.origin) return;
  event.respondWith(caches.open(CACHE).then(async cache => {
    const cached = await cache.match(event.request, { ignoreSearch: true });
    const fresh = fetch(event.request).then(res => {
      if (res.ok) cache.put(event.request, res.clone());
      return res;
    }).catch(() => cached);
    return cached || fresh;
  }));
});
```

web/css/style.css

Visual design, light/dark themes, phone and print layouts.

```
:root {
  --bg: #f7f5f3;
  --surface: #ffffff;
  --surface-2: #f2efec;
  --text: #1d1a1c;
  --muted: #675f63;
  --border: #e6e0dc;
  --accent: #c2364f;
  --accent-soft: #fbe9ec;
  --ok: #2e8f62;
  --ok-soft: #e4f4ec;
  --warn: #b7791f;
  --warn-soft: #fbf1df;
  --bad: #c62f3f;
  --bad-soft: #fbe7e8;
  --torso: #f1e6e0;
  --shadow: 0 1px 2px rgba(30, 20, 25, .05), 0 6px 20px rgba(30, 20, 25, .05);
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --bg: #141113;
    --surface: #1e1a1d;
    --surface-2: #282326;
    --text: #efe9ec;
    --muted: #a99fa4;
    --border: #3a3337;
    --accent: #f06b82;
    --accent-soft: #3a1f26;
    --ok: #4cc38a;
    --ok-soft: #173326;
    --warn: #e9b04f;
    --warn-soft: #3a2e17;
    --bad: #f0606a;
    --bad-soft: #3d1c20;
    --torso: #2c2528;
    --shadow: none;
  }
}

* { box-sizing: border-box; }
body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font: 16px/1.55 system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}
.wrap { max-width: 1160px; margin: 0 auto; padding: 0 16px; }
h1 { font-size: clamp(1.7rem, 3.6vw, 2.4rem); line-height: 1.15; margin: 0 0 .6rem; letter-spacing: -.01em; }
h2 { font-size: 1.05rem; margin: 0 0 .7rem; }
h3 { font-size: .98rem; margin: 0 0 .4rem; }
p { margin: 0 0 .75rem; }
code { background: var(--surface-2); padding: 1px 5px; border-radius: 4px; font-size: .85em; }
.muted { color: var(--muted); }
.small { font-size: .85rem; }
.mt { margin-top: 1.4rem; }
.ok-text { color: var(--ok); }
.warn-text { color: var(--warn); }
.bad-text { color: var(--bad); }

/* header */
.site-header { background: var(--surface); border-bottom: 1px solid var(--border); position: sticky; top: 0; z-
index: 10; }
.header-row { display: flex; align-items: center; justify-content: space-between; gap: 12px; min-height: 60px;
flex-wrap: wrap; }
.brand { display: flex; align-items: center; gap: 10px; font-weight: 750; font-size: 1.15rem; }
.logo { width: 28px; height: 28px; fill: var(--accent); }
.logo-line { fill: none; stroke: var(--surface); stroke-width: 2; stroke-linejoin: round; stroke-linecap: round; }
.tabs { display: flex; gap: 4px; overflow-x: auto; }
.tab { border: 0; background: none; color: var(--muted); font: inherit; font-size: .95rem; padding: 8px 12px;
border-radius: 8px; cursor: pointer; white-space: nowrap; }
.tab:hover { color: var(--text); }
.tab.active { background: var(--accent-soft); color: var(--accent); font-weight: 650; }

/* layout */
.panel { display: none; padding: 28px 0 8px; }
.panel.active { display: block; }
.intro { max-width: 760px; margin-bottom: 24px; }
.intro p { color: var(--muted); font-size: 1.05rem; }
.card { background: var(--surface); border: 1px solid var(--border); border-radius: 14px; padding: 20px; box-
shadow: var(--shadow); min-width: 0; }
.grid { display: grid; gap: 20px; margin-bottom: 20px; }
.grid.two { grid-template-columns: 1fr 1fr; }
.grid.three { grid-template-columns: repeat(3, 1fr); }
.grid.four { grid-template-columns: repeat(4, 1fr); }
.screen-grid { display: grid; grid-template-columns: 380px 1fr; gap: 20px; align-items: start; }
.results-col { display: grid; gap: 16px; min-width: 0; }
```

```
 84
 85  /* input card */
 86  .source-tabs { display: flex; background: var(--surface-2); border-radius: 10px; padding: 3px; margin-bottom:
     12px; }
 87  .source { flex: 1; border: 0; background: none; font: inherit; font-size: .9rem; color: var(--muted); padding:
     7px; border-radius: 8px; cursor: pointer; }
 88  .source.active { background: var(--surface); color: var(--text); font-weight: 650; box-shadow: 0 1px 2px rgba(0,0,
     0,.08); }
 89  .source-panel { display: none; }
 90  .source-panel.active { display: block; }
 91  .demo-cases { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
 92  .case { text-align: left; border: 1px solid var(--border); background: var(--surface); color: var(--text); border-
     radius: 10px; padding: 8px 10px; cursor: pointer; font: inherit; }
 93  .case:hover { border-color: var(--accent); }
 94  .case.active { border-color: var(--accent); background: var(--accent-soft); }
 95  .case-title { display: block; font-weight: 650; font-size: .92rem; }
 96  .case-meta { display: block; font-size: .78rem; color: var(--muted); }
 97  .file-drop { display: grid; place-items: center; border: 2px dashed var(--border); border-radius: 12px; padding:
     26px 12px; cursor: pointer; text-align: center; color: var(--muted); }
 98  .file-drop.over, .file-drop:hover { border-color: var(--accent); color: var(--text); }
 99  .file-drop input { display: none; }
100  .record-row { display: flex; gap: 8px; }
101  select { font: inherit; color: var(--text); background: var(--surface); border: 1px solid var(--border); border-
     radius: 8px; padding: 8px; flex: 1; min-width: 0; }
102  .btn { font: inherit; font-weight: 650; border: 0; border-radius: 10px; padding: 9px 14px; background: var(--
     accent); color: #fff; cursor: pointer; white-space: nowrap; }
103  .btn:disabled { opacity: .6; cursor: default; }
104
105  /* chest */
106  .chest-wrap { display: grid; grid-template-columns: 170px 1fr; gap: 12px; align-items: center; }
107  .chest { width: 100%; }
108  .torso { fill: var(--torso); }
109  .sternum { stroke: var(--border); stroke-width: 6; stroke-linecap: round; }
110  .heart-shape { fill: var(--accent-soft); stroke: var(--accent); stroke-width: 1; opacity: .8; }
111  .site { cursor: pointer; }
112  .site circle { fill: var(--surface); stroke: var(--muted); stroke-width: 2; transition: fill .3s, stroke .3s; }
113  .site text { font-size: 9px; font-weight: 700; text-anchor: middle; dominant-baseline: central; fill: var(--text);
      pointer-events: none; }
114  .site.focus circle { stroke: var(--accent); stroke-width: 3; }
115  .site.ok circle { fill: var(--ok-soft); stroke: var(--ok); }
116  .site.bad circle { fill: var(--bad-soft); stroke: var(--bad); }
117  .site.warn circle { fill: var(--warn-soft); stroke: var(--warn); }
118  .site.busy circle { stroke: var(--accent); stroke-dasharray: 4 3; animation: spin 1.2s linear infinite; transform-
     origin: center; transform-box: fill-box; }
119  @keyframes spin { to { transform: rotate(360deg); } }
120  .site-legend { list-style: none; padding: 0; margin: 0; display: grid; gap: 6px; font-size: .8rem; color: var(--
     muted); }
121  .site-legend b { color: var(--text); display: inline-block; width: 26px; }
122
123  /* verdict */
124  .verdict-card { scroll-margin-top: 76px; }
125  .verdict { display: flex; gap: 14px; align-items: center; border-radius: 12px; padding: 14px; background: var(--
     surface-2); }
126  .verdict-icon { flex: none; width: 46px; height: 46px; border-radius: 50%; display: grid; place-items: center;
     font-size: 1.4rem; font-weight: 800; background: var(--surface); }
127  .verdict-title { font-weight: 750; font-size: 1.12rem; line-height: 1.3; }
128  .verdict-text { color: var(--muted); font-size: .92rem; margin-top: 2px; }
129  .verdict.ok { background: var(--ok-soft); } .verdict.ok .verdict-icon { color: var(--ok); }
130  .verdict.bad { background: var(--bad-soft); } .verdict.bad .verdict-icon { color: var(--bad); }
131  .verdict.warn { background: var(--warn-soft); } .verdict.warn .verdict-icon { color: var(--warn); }
132  .verdict-actions { display: flex; gap: 8px; margin-top: 12px; flex-wrap: wrap; }
133  .verdict-actions[hidden] { display: none; }
134  .btn-ghost { font: inherit; font-size: .88rem; font-weight: 600; border: 1px solid var(--border); background: var(
     --surface); color: var(--text); border-radius: 9px; padding: 7px 12px; cursor: pointer; }
135  .btn-ghost:hover { border-color: var(--accent); color: var(--accent); }
136  .print-only { display: none; }
137  .truth { margin-top: 12px; font-size: .9rem; padding: 10px 12px; border-radius: 10px; border: 1px solid var(--
     border); }
138  .truth span { font-weight: 700; margin-left: 4px; }
139  .truth.agree span { color: var(--ok); }
140  .truth.disagree span { color: var(--bad); }
141
142  /* site cards */
143  .site-results { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
144  .site-card { padding: 14px; }
145  .site-head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px; }
146  .site-head h3 { margin: 0; }
147  .site-code { color: var(--muted); font-weight: 500; font-size: .8rem; }
148  .badge { font-size: .75rem; font-weight: 700; padding: 3px 9px; border-radius: 99px; background: var(--surface-2);
      color: var(--muted); }
149  .badge.ok { background: var(--ok-soft); color: var(--ok); }
150  .badge.bad { background: var(--bad-soft); color: var(--bad); }
151  .badge.warn { background: var(--warn-soft); color: var(--warn); }
152  .spec-wrap { position: relative; border-radius: 8px; overflow: hidden; }
153  .spec { display: block; width: 100%; height: 110px; image-rendering: auto; }
154  .playhead { display: none; position: absolute; top: 0; bottom: 0; width: 2px; background: #fff; box-shadow: 0 0
     4px rgba(0,0,0,.6); }
155  .axis-label { position: absolute; left: 6px; font-size: 10px; color: rgba(255,255,255,.75); }
156  .axis-label.top { top: 3px; } .axis-label.bottom { bottom: 3px; }
157  .timeline { display: block; width: 100%; height: 56px; margin-top: 4px; }
158  .site-foot { display: flex; align-items: center; gap: 10px; margin-top: 6px; }
```

```
.play { width: 32px; height: 32px; border-radius: 50%; border: 1px solid var(--border); background: var(--surface); color: var(--text); cursor: pointer; font-size: .8rem; flex: none; }
.play:hover { border-color: var(--accent); color: var(--accent); }
.skeleton { height: 170px; border-radius: 8px; background: linear-gradient(90deg, var(--surface-2), var(--border), var(--surface-2)); background-size: 200% 100%; animation: shimmer 1.2s infinite; }
@keyframes shimmer { to { background-position: -200% 0; } }

.disclaimer { margin-top: 24px; font-size: .85rem; color: var(--muted); border-left: 3px solid var(--warn); padding: 8px 12px; background: var(--surface); border-radius: 6px; }

/* validation */
.metric { text-align: center; }
.metric-big { font-size: 2rem; font-weight: 800; color: var(--accent); line-height: 1.2; }
.metric-big small { font-size: .9rem; font-weight: 600; color: var(--muted); }
.compare { display: grid; gap: 12px; margin: 8px 0 12px; }
.cmp-row { display: grid; grid-template-columns: 190px 1fr 44px; gap: 10px; align-items: center; font-size: .9rem; }
.cmp-track { height: 14px; background: var(--surface-2); border-radius: 7px; overflow: hidden; }
.cmp-bar { display: block; height: 100%; border-radius: 7px; }
.cmp-bar.base { background: var(--muted); }
.cmp-bar.ours { background: var(--accent); }
.cmp-bar.ref { background: var(--border); background-image: repeating-linear-gradient(45deg, transparent 0 5px, var(--surface) 5px 8px); border: 1px solid var(--muted); }
.cmp-val { font-weight: 700; font-variant-numeric: tabular-nums; }
.confusion { display: grid; grid-template-columns: auto repeat(3, 1fr); gap: 6px; font-size: .85rem; }
.cm-head { text-align: center; color: var(--muted); font-weight: 650; }
.cm-side { color: var(--muted); font-weight: 650; display: flex; align-items: center; }
.cm-cell { text-align: center; padding: 12px 4px; border-radius: 8px; font-weight: 750; font-size: 1.2rem; font-variant-numeric: tabular-nums; }
.cm-cell.good { background: var(--ok-soft); }
.cm-cell.bad { background: var(--bad-soft); }
.steps, .checks { margin: 0; padding-left: 20px; display: grid; gap: 8px; font-size: .93rem; }
.safety-card { margin-bottom: 20px; }
.safety { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 8px 24px; }
.safety-row { display: flex; justify-content: space-between; gap: 12px; padding: 6px 0; border-bottom: 1px solid var(--border); font-size: .9rem; }
.status-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; }
.status-grid ul { margin: 0; padding-left: 18px; display: grid; gap: 6px; font-size: .9rem; }

.footer { color: var(--muted); font-size: .85rem; padding: 28px 16px 40px; display: flex; justify-content: space-between; gap: 12px; flex-wrap: wrap; }
.offline-status { color: var(--ok); font-weight: 600; }

/* header language picker */
.lang { flex: none; width: auto; font-size: .8rem; padding: 3px 6px; margin-left: 4px; border-radius: 6px; }

/* guided recording */
.btn.wide { width: 100%; padding: 12px; font-size: 1rem; }
.one-site { margin-top: 10px; font-size: .88rem; }
.one-site summary { color: var(--accent); cursor: pointer; margin-bottom: 8px; }
.guide-live { border: 1px solid var(--border); border-radius: 12px; padding: 12px; background: var(--surface-2); }
.guide-step { font-weight: 750; }
.guide-instruction { font-size: .9rem; margin: 2px 0 8px; }
.live-wave { width: 100%; height: 70px; display: block; background: var(--surface); border-radius: 8px; }
.live-row { display: flex; align-items: center; gap: 10px; margin-top: 8px; flex-wrap: wrap; }
.chip-status { font-size: .8rem; font-weight: 700; padding: 3px 10px; border-radius: 99px; background: var(--surface); }
.chip-status.ok { background: var(--ok-soft); color: var(--ok); }
.chip-status.warn { background: var(--warn-soft); color: var(--warn); }
.chip-status.bad { background: var(--bad-soft); color: var(--bad); }
.level { flex: 1; min-width: 60px; height: 8px; border-radius: 4px; background: var(--surface); overflow: hidden; }
.level span { display: block; height: 100%; width: 0; background: var(--ok); transition: width .2s; }
.countdown { font-size: .85rem; font-weight: 700; font-variant-numeric: tabular-nums; }
.countdown.recording { color: var(--bad); }
.countdown.recording::before { content: "● "; }
.guide-actions { display: flex; gap: 6px; margin-top: 10px; flex-wrap: wrap; }
.child-id { font: inherit; font-size: .88rem; color: var(--text); background: var(--surface); border: 1px solid var(--border); border-radius: 9px; padding: 7px 10px; flex: 1; min-width: 150px; }

/* train your ear */
.train-grid { display: grid; grid-template-columns: 1fr 320px; gap: 20px; align-items: start; }
.train-side { display: grid; gap: 20px; }
.quiz-head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 14px; }
.quiz-listen { display: flex; gap: 16px; align-items: center; margin-bottom: 16px; }
.play.big { width: 64px; height: 64px; font-size: 1.3rem; border-width: 2px; }
.quiz-question { font-size: 1.25rem; font-weight: 750; }
.quiz-answers { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
.btn.answer { padding: 14px; font-size: 1rem; }
.btn.secondary { background: var(--surface-2); color: var(--text); border: 1px solid var(--border); }
.btn.answer:disabled { opacity: .45; cursor: default; }
.btn.answer.right { opacity: 1; background: var(--ok); color: #fff; }
.btn.answer.wrong { opacity: 1; background: var(--bad); color: #fff; }
.quiz-reveal { margin-top: 16px; padding-top: 14px; border-top: 1px solid var(--border); }
.quiz-reveal .spec-wrap { margin: 8px 0; }
.reveal-verdict { font-size: 1.15rem; font-weight: 800; margin-bottom: 6px; }
.reveal-verdict.ok { color: var(--ok); }
.reveal-verdict.bad { color: var(--bad); }
.scoreboard .score-row { display: flex; justify-content: space-between; padding: 6px 0; border-bottom: 1px solid var(--border); font-size: 1.05rem; }
.tips { margin: 0; padding-left: 18px; display: grid; gap: 8px; font-size: .9rem; }
```

```
238
239 /* screening log */
240 .log-toolbar { display: flex; justify-content: space-between; gap: 10px; flex-wrap: wrap; margin-bottom: 12px; }
241 .filter, .log-actions { display: flex; gap: 6px; flex-wrap: wrap; }
242 .chip { font: inherit; font-size: .85rem; border: 1px solid var(--border); background: var(--surface); color: var(
    --text); border-radius: 99px; padding: 5px 12px; cursor: pointer; }
243 .chip.active { background: var(--accent-soft); border-color: var(--accent); color: var(--accent); font-weight:
    650; }
244 .table-wrap { overflow-x: auto; }
245 .log-table, .data-table { width: 100%; border-collapse: collapse; font-size: .9rem; }
246 .log-table th, .data-table th { text-align: left; color: var(--muted); font-weight: 650; font-size: .8rem;
    padding: 6px 8px; border-bottom: 1px solid var(--border); }
247 .log-table td, .data-table td { padding: 8px; border-bottom: 1px solid var(--border); vertical-align: top; }
248 .data-table td:not(:first-child), .data-table th:not(:first-child) { text-align: right; white-space: nowrap; }
249 .link-btn { border: 0; background: none; color: var(--muted); cursor: pointer; font-size: 1rem; }
250 .link-btn:hover { color: var(--bad); }
251 .bad-num { color: var(--bad) !important; }
252 .warn-num { color: var(--warn) !important; }
253 .ok-num { color: var(--ok) !important; }
254
255 /* validation: stacked bars */
256 .stack-chart { display: grid; gap: 10px; margin: 6px 0 10px; }
257 .stack-row { display: grid; grid-template-columns: 120px 1fr 44px; gap: 10px; align-items: center; font-size: .
    9rem; }
258 .stack { display: flex; height: 16px; border-radius: 8px; overflow: hidden; background: var(--surface-2); }
259 .seg.bad { background: var(--bad); }
260 .seg.warn { background: var(--warn); }
261 .seg.miss { background: var(--border); }
262 .stack-val { font-weight: 700; font-variant-numeric: tabular-nums; }
263 .legend-row { display: flex; gap: 14px; flex-wrap: wrap; font-size: .8rem; color: var(--muted); }
264 .sw { display: inline-block; width: 10px; height: 10px; border-radius: 2px; margin-right: 5px; vertical-align: -
    1px; }
265 .sw.bad { background: var(--bad); } .sw.warn { background: var(--warn); } .sw.miss { background: var(--border); }
266 .cmp-bar.base2 { background: color-mix(in srgb, var(--accent) 45%, var(--surface-2)); }
267
268 @media print {
269   .site-header, .intro, .input-card, .verdict-actions, .play, .footer, .truth { display: none !important; }
270   .print-only { display: block; margin-top: 10px; }
271   body { background: #fff; }
272   .screen-grid { grid-template-columns: 1fr; }
273   .site-results { grid-template-columns: 1fr 1fr; }
274   .card { box-shadow: none; break-inside: avoid; }
275   .spec, .timeline { print-color-adjust: exact; -webkit-print-color-adjust: exact; }
276 }
277 @media (max-width: 980px) {
278   .train-grid { grid-template-columns: 1fr; }
279   .screen-grid { grid-template-columns: 1fr; }
280   .grid.four { grid-template-columns: 1fr 1fr; }
281   .grid.three { grid-template-columns: 1fr 1fr; }
282 }
283 @media (max-width: 680px) {
284   .grid.two, .grid.three, .status-grid, .site-results { grid-template-columns: 1fr; }
285   .cmp-row { grid-template-columns: 1fr 44px; }
286   .cmp-name { grid-column: 1 / -1; }
287   .chest-wrap { grid-template-columns: 120px 1fr; }
288 }
289 @media (max-width: 420px) {
290   .grid.four { grid-template-columns: 1fr; }
291   .case-meta { font-size: .72rem; }
292 }
```

tests/test_pipeline.py

Automated tests, including Python vs JavaScript parity and translations.

```
"""Run with:  python -m pytest tests"""

import json
import shutil
import subprocess
import sys
from pathlib import Path

import numpy as np
import pytest

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "ml"))

import features as F  # noqa: E402
from heartbeat_check import beat_measures  # noqa: E402
from train import ABSENT, PRESENT, UNKNOWN, decide, weighted_accuracy  # noqa: E402

SAMPLES = sorted((ROOT / "web" / "samples").glob("*.wav"))
CONFIG = ROOT / "web" / "model" / "config.json"
CHECK = ROOT / "web" / "model" / "heartbeat_check.json"


def test_mel_filters_cover_range_in_order():
    fb = F.MEL_FB
    assert fb.shape == (F.N_MELS, F.N_FFT // 2 + 1)
    peaks = fb.argmax(axis=1)
    assert np.all(np.diff(peaks) >= 0), "filters should go from low to high pitch"
    assert np.all(fb.max(axis=1) > 0), "every filter should catch some frequency"


def test_pure_tone_lands_in_matching_band():
    t = np.arange(F.SAMPLE_RATE * 2) / F.SAMPLE_RATE
    for freq in (60, 200, 600):
        spec = F.log_mel(np.sin(2 * np.pi * freq * t))
        band = spec.mean(axis=1).argmax()
        centers = F.mel_to_hz(np.linspace(F.hz_to_mel(F.F_MIN), F.hz_to_mel(F.F_MAX), F.N_MELS + 2))[1:-1]
        assert abs(centers[band] - freq) < 40, f"{freq} Hz tone went to band centred at {centers[band]:.0f} Hz"


def test_loudness_does_not_change_spectrogram():
    rng = np.random.default_rng(0)
    x = rng.normal(size=8000)
    assert np.allclose(F.log_mel(x), F.log_mel(x * 7.5), atol=1e-4)


def test_short_recording_is_repeated_to_fill_window():
    spec = np.arange(F.N_MELS * 100, dtype=np.float32).reshape(F.N_MELS, 100)
    w = F.cut_window(spec, 0)
    assert w.shape == (F.N_MELS, F.WINDOW_FRAMES)
    assert np.array_equal(w[:, 100:200], spec)


def test_weighted_accuracy_matches_challenge_definition():
    # 2 present (1 correct), 1 unknown (correct), 3 absent (2 correct)
    y_true = [PRESENT, PRESENT, UNKNOWN, ABSENT, ABSENT, ABSENT]
    y_pred = [PRESENT, ABSENT, UNKNOWN, ABSENT, ABSENT, PRESENT]
    expected = (5 * 1 + 3 * 1 + 1 * 2) / (5 * 2 + 3 * 1 + 1 * 3)
    assert weighted_accuracy(y_true, y_pred) == pytest.approx(expected)


def test_decision_rule_order():
    probs = np.array([[0.9, 0.9], [0.1, 0.9], [0.1, 0.1]])  # (present, unknown) per child
    assert list(decide(probs, 0.5, 0.5)) == [PRESENT, UNKNOWN, ABSENT]


@pytest.mark.skipif(not SAMPLES or not CONFIG.exists(), reason="run ml/train.py first")
@pytest.mark.skipif(shutil.which("node") is None, reason="Node.js not installed")
def test_javascript_features_match_python():
    for wav in SAMPLES[:3]:
        out = subprocess.run(["node", str(ROOT / "tests" / "js_logmel.cjs"), str(wav), str(CONFIG)],
                             capture_output=True, text=True, check=True)
        js = json.loads(out.stdout)
        py = F.log_mel(F.load_wav(wav))
        assert js["nFrames"] == py.shape[1]
        assert js["starts"] == F.window_starts(py.shape[1])
        diff = np.abs(np.array(js["data"]).reshape(py.shape) - py).max()
        assert diff < 1e-3, f"{wav.name}: max difference {diff}"
        rhythm, depth = beat_measures(py)
        assert js["beat"]["rhythm"] == pytest.approx(rhythm, abs=1e-3)
        assert js["beat"]["depth"] == pytest.approx(depth, abs=1e-3)


@pytest.mark.skipif(not CHECK.exists(), reason="run ml/heartbeat_check.py first")
def test_heartbeat_check_rejects_non_heart_sounds_and_accepts_demo_cases():
    limits = json.loads(CHECK.read_text())
    passes = lambda m: m[0] >= limits["rhythm_min"] and m[1] >= limits["depth_min"]
```

```
    rng = np.random.default_rng(123)
    t = np.arange(F.SAMPLE_RATE * 10) / F.SAMPLE_RATE
    assert not passes(beat_measures(F.log_mel(rng.normal(size=len(t)))))
    assert not passes(beat_measures(F.log_mel(np.sin(2 * np.pi * 440 * t))))
    accepted = [passes(beat_measures(F.log_mel(F.load_wav(w)))) for w in SAMPLES]
    assert np.mean(accepted) >= 0.9


@pytest.mark.skipif(not CONFIG.exists(), reason="run ml/train.py first")
def test_exported_model_outputs_probabilities():
    ort = pytest.importorskip("onnxruntime")
    session = ort.InferenceSession(str(ROOT / "web" / "model" / "murmur_cnn.onnx"))
    x = np.random.default_rng(1).normal(-1, 1, (3, 1, F.N_MELS, F.WINDOW_FRAMES)).astype(np.float32)
    probs = session.run(None, {"logmel": x})[0]
    assert probs.shape == (3, 3)
    assert np.allclose(probs.sum(axis=1), 1, atol=1e-5)


def test_add_noise_hits_requested_snr():
    import evaluation as E
    rng = np.random.default_rng(3)
    x = np.sin(2 * np.pi * 100 * np.arange(40000) / F.SAMPLE_RATE)
    for snr in (20, 10, 5):
        noise = E.add_noise(x, snr, rng) - x
        measured = 10 * np.log10(np.mean(x ** 2) / np.mean(noise ** 2))
        assert measured == pytest.approx(snr, abs=0.3)


def test_augmented_training_window_has_model_shape():
    import evaluation as E
    rng = np.random.default_rng(4)
    audio = (rng.normal(size=F.SAMPLE_RATE * 9) * 3000).astype(np.int16)
    short = audio[:F.SAMPLE_RATE * 2]
    for a in (audio, short):
        for _ in range(5):
            assert E.augmented_window(a, rng).shape == (F.N_MELS, F.WINDOW_FRAMES)


def test_breakdown_counts_referred_rerecord_missed():
    import evaluation as E
    kids = [{"label": PRESENT, "pred": PRESENT, "grade": "I/VI"},
            {"label": PRESENT, "pred": ABSENT, "grade": "I/VI"},
            {"label": PRESENT, "pred": UNKNOWN, "grade": "III/VI"},
            {"label": ABSENT, "pred": PRESENT, "grade": "nan"}]
    out = E.breakdown(kids, "grade", ["I/VI", "II/VI", "III/VI"], PRESENT, UNKNOWN, ABSENT)
    assert out == [{"group": "I/VI", "n": 2, "referred": 1, "rerecord": 0, "missed": 1},
                   {"group": "III/VI", "n": 1, "referred": 0, "rerecord": 1, "missed": 0}]


@pytest.mark.skipif(shutil.which("node") is None, reason="Node.js not installed")
def test_every_text_is_translated():
    out = subprocess.run(["node", str(ROOT / "tests" / "i18n_check.cjs")], capture_output=True, text=True)
    assert out.returncode == 0, out.stdout


@pytest.mark.skipif(shutil.which("node") is None, reason="Node.js not installed")
def test_browser_wav_encoder_round_trips(tmp_path):
    script = tmp_path / "enc.cjs"
    script.write_text(
        "const fs=require('fs'),vm=require('vm');"
        "const ctx=vm.createContext({});"
        f"vm.runInContext(fs.readFileSync({json.dumps(str(ROOT / 'web' / 'js' / 'guided.js'))},'utf8')+';this.
G=Guided;',ctx);"
        "const x=new Float32Array(48000);for(let i=0;i<x.length;i++)x[i]=0.5*Math.sin(2*Math.PI*220*i/48000);"
        f"fs.writeFileSync({json.dumps(str(tmp_path / 'out.wav'))},Buffer.from(ctx.G.encodeWav(x,48000)));")
    subprocess.run(["node", str(script)], check=True)
    from scipy.io import wavfile
    rate, y = wavfile.read(tmp_path / "out.wav")
    assert rate == 48000 and len(y) == 48000
    assert np.abs(y).max() == pytest.approx(0.5 * 32767, rel=0.01)
```

tests/js_logmel.cjs

Node.js helper used by the parity test.

```
// Helper for tests/test_pipeline.py: prints the JavaScript log-mel of a WAV file as JSON.
// Usage: node tests/js_logmel.cjs <file.wav> <config.json>
const fs = require("fs");
const path = require("path");
const F = require(path.join(__dirname, "..", "web", "js", "features.js"));

const [wavPath, configPath] = process.argv.slice(2);
const buf = fs.readFileSync(wavPath);
const wav = F.parseWav(buf.buffer.slice(buf.byteOffset, buf.byteOffset + buf.byteLength));
const cfg = JSON.parse(fs.readFileSync(configPath, "utf8")).features;
const spec = F.logMel(wav.samples, cfg);
const starts = F.windowStarts(spec.nFrames, cfg);
const check = JSON.parse(fs.readFileSync(path.join(path.dirname(configPath), "heartbeat_check.json"), "utf8"));
const beat = F.beatMeasures(spec, check);
process.stdout.write(JSON.stringify({ nFrames: spec.nFrames, data: Array.from(spec.data), starts, beat }));
```

tests/i18n_check.cjs

Checks that every text exists in all four languages.

```
// Checks that every text key used by the page and the scripts exists in every
// language, with the same {placeholders}. Prints problems; exit code 1 if any.
const fs = require("fs");
const path = require("path");
const vm = require("vm");

const web = path.join(__dirname, "..", "web");
const ctx = vm.createContext({ localStorage: undefined, navigator: { language: "en" } });
vm.runInContext(fs.readFileSync(path.join(web, "js", "i18n.js"), "utf8") + "\nthis.I18N = I18N;", ctx);
const { EN, tr, pt, es } = ctx.I18N.keys;
const langs = { tr, pt, es };
const problems = [];

const html = fs.readFileSync(path.join(web, "index.html"), "utf8");
const htmlKeys = [...html.matchAll(/data-i18n(?:-html|-placeholder)?="([^"]+)"/g)].map(m => m[1]);

const jsKeys = new Set();
for (const f of fs.readdirSync(path.join(web, "js"))) {
  if (f === "i18n.js") continue;
  const src = fs.readFileSync(path.join(web, "js", f), "utf8");
  for (const m of src.matchAll(/\bt\("([^"]+)"/g)) if (!m[1].endsWith(".")) jsKeys.add(m[1]);
}
const families = {
  "site.": ["AV", "PV", "TV", "MV", "Phc"], "label.": ["0", "1", "2"],
  "age.": ["Neonate", "Infant", "Child", "Adolescent", "nan"], "sex.": ["Female", "Male"],
  "timing.": ["Early-systolic", "Mid-systolic", "Late-systolic", "Holosystolic"],
  "quiz.pitch.": ["Low", "Medium", "High"], "quiz.quality.": ["Blowing", "Harsh", "Musical"],
  "cond.": ["clean", "noise20", "noise10", "noise5", "phone", "rumble"],
  "neg.": ["white noise", "pink-ish noise", "steady tone 440 Hz", "mains hum 50 Hz", "speech-like babble",
"silence with hiss"],
  "guide.place.": ["AV", "PV", "TV", "MV"], "log.status": ["0", "1", "2"],
};
for (const [prefix, ends] of Object.entries(families)) ends.forEach(e => jsKeys.add(prefix + e));

for (const key of jsKeys) {
  if (!(key in EN)) problems.push(`EN missing script key: ${key}`);
}
for (const [name, dict] of Object.entries(langs)) {
  for (const key of htmlKeys) if (!(key in dict)) problems.push(`${name} missing page key: ${key}`);
  for (const key of jsKeys) if (!(key in dict)) problems.push(`${name} missing script key: ${key}`);
  for (const [key, text] of Object.entries(EN)) {
    if (!(key in dict)) continue;
    const vars = (s) => [...s.matchAll(/\{(\w+)\}/g)].map(m => m[1]).sort().join(",");
    if (vars(text) !== vars(dict[key])) problems.push(`${name} placeholders differ for ${key}`);
  }
}

console.log(problems.length ? problems.join("\n") : `ok: ${htmlKeys.length} page keys, ${jsKeys.size} script keys,
 4 languages`);
process.exit(problems.length ? 1 : 0);
```

docs/make_pdfs.py

Generates this PDF and the one-page description.

```
"""
Builds the two PDFs required for the UnivaBio submission:

  docs/LittleHeart_Project_Description.pdf   one page
  docs/LittleHeart_Code.pdf                  all source code with line numbers

Run after training:  python docs/make_pdfs.py
"""

import json
from pathlib import Path

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import mm
from reportlab.platypus import (Image, PageBreak, Paragraph, Preformatted,
                                SimpleDocTemplate, Spacer, Table, TableStyle)

ROOT = Path(__file__).resolve().parent.parent
DOCS = ROOT / "docs"
ACCENT = colors.HexColor("#c2364f")
GREY = colors.HexColor("#675f63")

CODE_FILES = [
    ("ml/download_data.py", "Downloads the CirCor and PhysioNet 2016 datasets from PhysioNet's open-data mirror."),
    ("ml/features.py", "Turns a recording into a log-mel spectrogram and 4-second windows."),
    ("ml/train.py", "Cross-validated training (clean or noise-robust), evaluation, ONNX export, demo and quiz sets."),
    ("ml/evaluation.py", "Damaged-audio test conditions, training augmentation, error analysis, quiz selection."),
    ("ml/heartbeat_check.py", "Safety check calibration: refuse recordings without a heartbeat rhythm."),
    ("ml/external_validation.py", "Tests the exported model on the PhysioNet 2016 dataset it never saw."),
    ("ml/compare_models.py", "Adds the clean-trained model's results for before/after comparison."),
    ("web/index.html", "Page structure: screening, ear training, screening log, validation and education tabs."),
    ("web/js/i18n.js", "Translations (English, Turkish, Portuguese, Spanish)."),
    ("web/js/features.js", "JavaScript port of features.py and the heartbeat check, so the browser computes identical numbers."),
    ("web/js/model.js", "Runs the ONNX model in the browser and applies the referral rule."),
    ("web/js/draw.js", "Spectrogram, murmur timeline, playback cursor and live waveform drawing."),
    ("web/js/app.js", "Screening view: real cases, uploads, verdict, per-valve results."),
    ("web/js/guided.js", "Guided recording with live heartbeat and sound-level check."),
    ("web/js/quiz.js", "Train-your-ear mode."),
    ("web/js/log.js", "Screening log stored on the device, with CSV export."),
    ("web/js/validation.js", "Validation tab: metrics, error analysis, robustness, external test."),
    ("web/js/main.js", "Start-up: tabs, language, modules, offline support."),
    ("web/sw.js", "Service worker that keeps the app working offline."),
    ("web/css/style.css", "Visual design, light/dark themes, phone and print layouts."),
    ("tests/test_pipeline.py", "Automated tests, including Python vs JavaScript parity and translations."),
    ("tests/js_logmel.cjs", "Node.js helper used by the parity test."),
    ("tests/i18n_check.cjs", "Checks that every text exists in all four languages."),
    ("docs/make_pdfs.py", "Generates this PDF and the one-page description."),
]


def styles():
    base = getSampleStyleSheet()
    return {
        "title": ParagraphStyle("t", parent=base["Title"], fontSize=21, textColor=ACCENT, spaceAfter=1),
        "tagline": ParagraphStyle("tg", parent=base["Normal"], alignment=TA_CENTER, fontSize=9.5, textColor=GREY, spaceAfter=5),
        "h": ParagraphStyle("h", parent=base["Heading2"], fontSize=10.2, textColor=ACCENT, spaceBefore=4, spaceAfter=1),
        "body": ParagraphStyle("b", parent=base["Normal"], fontSize=8.3, leading=10.7),
        "small": ParagraphStyle("s", parent=base["Normal"], fontSize=7.3, leading=9, textColor=GREY),
        "code": ParagraphStyle("c", parent=base["Code"], fontSize=6.8, leading=8.3, leftIndent=0),
    }


def build_description(m, ext):
    s = styles()
    c, b, ds = m["cnn"], m["baseline"], m["dataset"]
    pct = lambda x: f"{round(x * 100)}%"
    rob = {r["condition"]: r for r in m["analysis"]["robustness"]}
    old = {r["condition"]: r for r in m["comparison"]["robustness"]} if "comparison" in m else None
    grades = {g["group"]: g for g in m["analysis"]["by_grade"]}
    missed = sum(g["missed"] for g in grades.values())
    faint = grades.get("I/VI", {"missed": 0})["missed"]
    loud = grades.get("III/VI")
    phone = (f"With a phone-like microphone, training on randomly damaged audio raised murmurs caught from "
             f"{pct(old['phone']['sensitivity'])} to <b>{pct(rob['phone']['sensitivity'])}</b>. ") if old else ""

    left = [
        Paragraph("The problem", s["h"]),
        Paragraph("About 1 in 100 babies is born with a heart defect, and rheumatic heart disease affects tens of millions of "
```

```
                  "people, most of them young and living in low- and middle-income countries. Both are often first noticed as a "
                  "heart murmur. But recognising a murmur takes years of training, and the echocardiogram that confirms it is "
                  "scarce exactly where these diseases are most common.", s["body"]),
        Paragraph("Our solution", s["h"]),
        Paragraph("<b>LittleHeart</b> lets a community health worker, school nurse or GP record a child's heart at the four "
                  "valve areas and instantly learn: <i>refer for echocardiogram</i>, <i>no murmur detected</i>, or <i>recording "
                  "unclear, record again</i>, so scarce echocardiograms go to the children who need them.", s["body"]),
        Paragraph("Key features", s["h"]),
        Paragraph("• <b>Guided recording</b>: step-by-step through the four sites with a live heartbeat and sound-level check.<br/>"
                  "• <b>Explainable results</b>: chest diagram, spectrogram per valve and a murmur-likelihood timeline.<br/>"
                  "• <b>Train your ear</b>: health workers practise on 18 real recordings against the AI.<br/>"
                  "• <b>Screening log</b> for school or village screening days, with CSV export.<br/>"
                  "• <b>Works offline</b>, installs like an app, and runs privately in the browser: no audio is uploaded.<br/>"
                  "• <b>4 languages</b> (English, Turkish, Portuguese, Spanish); real held-out cases; printable referral note.<br/>"
                  "• <b>Transparent validation tab</b> with an honest \"working / not yet / limits\" table.", s["body"]),
        Paragraph("Honest limits", s["h"]),
        Paragraph("A screening aid, not a diagnosis or medical device. It misses some (mostly faint) murmurs, was trained on one "
                  "population and one stethoscope, cannot tell innocent from pathological murmurs, and has not been tested with "
                  "real phone adapters or prospectively. All of this is stated in the app.", s["body"]),
    ]
    right = [
        Paragraph("How the AI works", s["h"]),
        Paragraph(f"Trained on the CirCor DigiScope dataset (PhysioNet): {ds['recordings']:,} recordings from {ds['children']} "
                  "Brazilian children, labelled by cardiologists. Each recording becomes a log-mel spectrogram (25-1000 Hz), cut "
                  "into 4-second windows that a small convolutional network (about 61,000 weights, PyTorch, run in the browser "
                  "with ONNX Runtime Web) scores as murmur / no murmur / unclear. During training, audio is randomly damaged with "
                  "noise and phone-like filtering. A <b>heartbeat safety check</b> refuses recordings with no 40-220 bpm rhythm, "
                  "so the AI never guesses on noise or speech.", s["body"]),
        Paragraph("Results", s["h"]),
        Paragraph(f"{m['folds']}-fold cross-validation, split by child, {ds['cv_children']} children: PhysioNet 2022 murmur score "
                  f"<b>{c['weighted_accuracy']['mean']:.2f} ± {c['weighted_accuracy']['std']:.2f}</b> (baseline "
                  f"{b['weighted_accuracy']['mean']:.2f}; the best challenge team scored 0.78 on a harder hidden test set, not a "
                  f"like-for-like comparison). It caught <b>{pct(c['sensitivity']['mean'])}</b> of children with murmurs and "
                  f"cleared <b>{pct(c['specificity']['mean'])}</b> of those without (AUC {c['auc']['mean']:.2f}). "
                  f"Of {missed} murmurs missed, {faint} were the faintest grade; loud grade III/VI murmurs were referred "
                  f"{pct(loud['referred'] / loud['n']) if loud else '-'} of the time. " + phone, s["body"]),
        Paragraph("External test", s["h"]),
        Paragraph(f"On the PhysioNet 2016 database (3,240 recordings, mostly adults, other devices, never used in training), "
                  f"the murmur score separated normal from abnormal hearts with AUC <b>{ext['overall']['auc']:.2f}</b>, and only "
                  f"{pct(ext['overall']['beat_pass'])} of recordings passed the heartbeat check. Its labels are not murmurs, but "
                  "the result shows LittleHeart must be validated locally before use in a new setting.", s["body"]),
        Paragraph("Next steps", s["h"]),
        Paragraph("Test with low-cost stethoscope adapters, validate with children in a new region, add innocent-vs-"
                  "pathological classification with echocardiography-labelled data, and pilot with a school RHD-screening "
                  "programme.", s["body"]),
    ]

    story = [Paragraph("LittleHeart", s["title"]),
             Paragraph("AI heart murmur screening for children · UnivaBio 2026: AI for Human Health", s["tagline"])]
    cols = Table([[left, right]], colWidths=[92 * mm, 92 * mm])
    cols.setStyle(TableStyle([("VALIGN", (0, 0), (-1, -1), "TOP"), ("LEFTPADDING", (0, 0), (-1, -1), 0),
                              ("RIGHTPADDING", (0, 0), (0, 0), 4 * mm)]))
    story += [cols, Spacer(1, 2.5 * mm)]
    shot = DOCS / "screenshot.png"
    if shot.exists():
        img = Image(str(shot))
        img._restrictSize(150 * mm, 72 * mm)
        story.append(img)
    story += [Spacer(1, 1.5 * mm),
              Paragraph("Tech: Python, PyTorch, scikit-learn, NumPy (training) · HTML, CSS, JavaScript, ONNX Runtime Web (app) · "
                        "Data: CirCor DigiScope (Oliveira et al. 2022) and PhysioNet/CinC Challenge 2016 (Liu et
```

```
    al. 2016), PhysioNet.",
                          s["small"])]

    doc = SimpleDocTemplate(str(DOCS / "LittleHeart_Project_Description.pdf"), pagesize=A4, leftMargin=13 * mm,
                            rightMargin=13 * mm, topMargin=10 * mm, bottomMargin=9 * mm, title="LittleHeart -
    Project Description")
    doc.build(story)
    return doc


def build_code_pdf():
    s = styles()
    story = [Paragraph("LittleHeart: source code", s["title"]),
             Paragraph("UnivaBio 2026 · AI for Human Health", s["tagline"]), Spacer(1, 4 * mm),
             Paragraph("Project structure", s["h"])]
    for path, purpose in CODE_FILES:
        story.append(Paragraph(f"<b>{path}</b>: {purpose}", s["body"]))
    story.append(Paragraph("Generated, not listed: web/model/murmur_cnn.onnx (trained network), config.json (
    spectrogram "
                           "settings and thresholds), metrics.json/.js (validation results), external.json (
    external test), "
                           "heartbeat_check.json, web/samples and web/quiz (recordings from the dataset). Third-
    party: "
                           "web/vendor/ort (ONNX Runtime Web, MIT licence).", s["body"]))
    for path, purpose in CODE_FILES:
        story += [PageBreak(), Paragraph(path, s["h"]), Paragraph(purpose, s["small"]), Spacer(1, 2 * mm)]
        lines = (ROOT / path).read_text(encoding="utf-8").splitlines()
        story.append(Preformatted("\n".join(f"{i:4d}  {line}" for i, line in enumerate(lines, 1)),
                                  s["code"], maxLineLength=120, newLineChars="      "))

    def footer(canvas, doc):
        canvas.setFont("Helvetica", 7)
        canvas.setFillColor(colors.grey)
        canvas.drawRightString(A4[0] - 13 * mm, 8 * mm, f"LittleHeart · page {doc.page}")

    doc = SimpleDocTemplate(str(DOCS / "LittleHeart_Code.pdf"), pagesize=A4, leftMargin=13 * mm, rightMargin=13 *
    mm,
                            topMargin=12 * mm, bottomMargin=14 * mm, title="LittleHeart - Source Code")
    doc.build(story, onFirstPage=footer, onLaterPages=footer)
    return doc


if __name__ == "__main__":
    metrics = json.loads((ROOT / "web" / "model" / "metrics.json").read_text())
    external = json.loads((ROOT / "web" / "model" / "external.json").read_text())
    d = build_description(metrics, external)
    c = build_code_pdf()
    print(f"Description: {d.page} page(s); code listing: {c.page} page(s)")
```